Veda ERP

API Source Code

78 Route Files | 13 Modules

Next.js App Router | TypeScript

MongoDB + JWT Authentication

Table of Contents

/api/admin/fix-indexes/route.ts 110 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB } from '@/lib/db'
import { Stock, Production } from '@/lib/models'
import mongoose from 'mongoose'
import { requireAdmin } from '@/lib/auth'

// Force dynamic — never cache
export const dynamic = 'force-dynamic'
export const revalidate = 0

// GET /api/admin/fix-indexes
// One-shot fix: drops stale indexes from collections that no longer have those
// fields in their schema. Specifically:
//   - stocks.brickType_1   (old Stock schema had a brickType field — now removed)
//   - any other indexes not in the current schema
//
// Safe to call multiple times. Returns before/after index lists for audit.
// Admin-only — mutates database structure.
export async function GET() {
  const collections: Record<string, unknown> = {}
  const result: { collections: Record<string, unknown>; timestamp: string; error?: unknown } = {
    collections,
    timestamp: new Date().toISOString(),
  }

  try {
    const auth = await requireAdmin()
    if (auth instanceof NextResponse) return auth

    await connectDB()

    // ── Stock collection ──────────────────────────────────────
    const stockCollection = mongoose.connection.collection('stocks')
    const stockIndexesBefore = await stockCollection.indexes()

    // Drop every index that's not _id_ and not in the current schema.
    // Current StockSchema indexes: _id_, date_-1
    const stockValidIndexes = new Set(['_id_', 'date_-1'])
    const stockStaleIndexes = stockIndexesBefore.filter(
      (idx) => !stockValidIndexes.has(idx.name || '')
    )

    const stockDropped: string[] = []
    for (const idx of stockStaleIndexes) {
      try {
        await stockCollection.dropIndex(idx.name || '')
        stockDropped.push(idx.name || '')
      } catch (err) {
        // Index may already be gone — log and continue
        stockDropped.push(`${idx.name || ''} (drop failed: ${(err as Error).message})`)
      }
    }

    // Recreate the date index (idempotent — Mongoose would do this on next
    // syncIndexes call anyway, but we do it explicitly here for safety).
    try {
      await stockCollection.createIndex({ date: -1 }, { name: 'date_-1' })
    } catch {
      // Already exists — fine
    }

    const stockIndexesAfter = await stockCollection.indexes()

    result.collections.stocks = {
      indexesBefore: stockIndexesBefore.map((i) => i.name || ''),
      stale: stockStaleIndexes.map((i) => i.name || ''),
      dropped: stockDropped,
      indexesAfter: stockIndexesAfter.map((i) => i.name || ''),
    }

    // ── Production collection (defensive) ─────────────────────
    const prodCollection = mongoose.connection.collection('productions')
    const prodIndexesBefore = await prodCollection.indexes()

    // Drop any non-_id_ indexes that might be stale from old schemas
    // (e.g. customerName_1, address_1 if those were ever indexed)
    const prodValidIndexes = new Set(['_id_', 'date_-1', 'customerId_1'])
    const prodStaleIndexes = prodIndexesBefore.filter(
      (idx) => !prodValidIndexes.has(idx.name || '')
    )

    const prodDropped: string[] = []
    for (const idx of prodStaleIndexes) {
```

```
      try {
        await prodCollection.dropIndex(idx.name || '')
        prodDropped.push(idx.name || '')
      } catch (err) {
        prodDropped.push(`${idx.name || ''} (drop failed: ${(err as Error).message})`)
      }
    }

    const prodIndexesAfter = await prodCollection.indexes()

    result.collections.productions = {
      indexesBefore: prodIndexesBefore.map((i) => i.name || ''),
      stale: prodStaleIndexes.map((i) => i.name || ''),
      dropped: prodDropped,
      indexesAfter: prodIndexesAfter.map((i) => i.name || ''),
    }

    return NextResponse.json(result)
  } catch (err) {
    result.error = err instanceof Error ? {
      message: err.message,
      stack: err.stack?.split('\n').slice(0, 10),
    } : String(err)
    return NextResponse.json(result, { status: 500 })
  }
}
```

/api/admin/sync-all-stock/route.ts 67 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB } from '@/lib/db'
import { Production } from '@/lib/models'
import { syncStockForDates } from '@/lib/sync-stock'
import { requireAdmin } from '@/lib/auth'

// Force dynamic — never cache
export const dynamic = 'force-dynamic'
export const revalidate = 0

// GET /api/admin/sync-all-stock
// One-shot backfill: aggregates ALL existing production dates and syncs them
// to the Stock collection. Used after schema changes or to populate Stock
// Overview retroactively for historical production data.
//
// Returns a summary: how many dates were processed, how many succeeded,
// how many failed, and the per-date results.
// Admin-only — triggers full-table scans.
export async function GET() {
  const result: Record<string, unknown> = {
    timestamp: new Date().toISOString(),
  }

  try {
    const auth = await requireAdmin()
    if (auth instanceof NextResponse) return auth

    await connectDB()

    // Get all distinct production dates
    const dates: string[] = await Production.distinct('date')
    result.datesFound = dates.length
    result.dates = dates

    // Sync each date, tracking success/failure
    const succeeded: string[] = []
    const failed: Array<{ date: string; error: string }> = []

    for (const date of dates) {
      try {
        // Use syncStockForDates for a single date — it already handles errors
        // internally but we want per-date reporting here so we call the single
        // date version indirectly.
        await syncStockForDates([date])
        succeeded.push(date)
      } catch (err) {
        failed.push({
          date,
          error: err instanceof Error ? err.message : String(err),
        })
      }
    }

    result.succeeded = succeeded.length
    result.failed = failed.length
    result.failures = failed

    return NextResponse.json(result)
  } catch (err) {
    result.error = err instanceof Error ? {
      message: err.message,
      stack: err.stack?.split('\n').slice(0, 10),
    } : String(err)
    return NextResponse.json(result, { status: 500 })
  }
}
```

/api/ai/config/route.ts 121 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB } from '@/lib/db'
import { AiConfig } from '@/lib/models'
import { requireSession, requireAdmin } from '@/lib/auth'

// Force dynamic
export const dynamic = 'force-dynamic'
export const revalidate = 0

// GET /api/ai/config
// Returns the AI config. The API key is MASKED (only last 4 chars
// shown) so the client can display "sk-...abcd" without exposing the full
// secret. Any logged-in user can read this — they need to know if AI is
// enabled to show/hide the AI buttons in the UI.
export async function GET() {
  try {
    const session = await requireSession()
    if (session instanceof NextResponse) return session

    await connectDB()
    const config = await AiConfig.findOne().lean()

    if (!config) {
      // No config yet — return defaults (AI disabled, no key)
      return NextResponse.json({
        provider: 'openai',
        enabled: false,
        model: 'gpt-4o-mini',
        hasKey: false,
        keyMasked: '',
      })
    }

    const key = String((config as Record<string, unknown>).openaiApiKey || '')
    // Mask differently based on provider so user knows which kind of key is saved
    const provider = String((config as Record<string, unknown>).provider || 'openai')
    const masked = key.length > 8
      ? provider === 'groq'
        ? `gsk_...${key.slice(-4)}`
        : `sk-...${key.slice(-4)}`
      : ''

    return NextResponse.json({
      provider,
      enabled: !!(config as Record<string, unknown>).enabled,
      model: String((config as Record<string, unknown>).model || 'gpt-4o-mini'),
      hasKey: key.length > 0,
      keyMasked: masked,
    })
  } catch (error) {
    console.error('Error fetching AI config:', error)
    return NextResponse.json({ error: 'Failed to fetch AI config' }, { status: 500 })
  }
}

// PUT /api/ai/config
// Updates the AI config. Admin-only — operators/accountants cannot change
// the API key (would be a security hole).
// Body: { provider?: 'openai'|'groq', openaiApiKey?: string, enabled?: boolean, model?: string }
export async function PUT(request: Request) {
  try {
    const auth = await requireAdmin()
    if (auth instanceof NextResponse) return auth

    await connectDB()

    const body = await request.json()
    const update: Record<string, unknown> = {}

    if (body.provider === 'openai' || body.provider === 'groq') {
      update.provider = body.provider
    }
    if (typeof body.openaiApiKey === 'string') {
      // Allow empty string to clear the key
      update.openaiApiKey = body.openaiApiKey.trim()
    }
    if (typeof body.enabled === 'boolean') {
      update.enabled = body.enabled
    }
    if (typeof body.model === 'string' && body.model.trim()) {
      update.model = body.model.trim()
    }
```

```
    // Upsert: if no config doc exists, create one; otherwise update in place
    const existing = await AiConfig.findOne()
    let saved
    if (existing) {
      Object.entries(update).forEach(([k, v]) => existing.set(k, v))
      saved = await existing.save()
    } else {
      saved = await AiConfig.create({
        provider: 'openai',
        openaiApiKey: '',
        enabled: false,
        model: 'gpt-4o-mini',
        ...update,
      })
    }

    // Return masked — never echo the full key back
    const key = String((saved as Record<string, unknown>).openaiApiKey || '')
    const provider = String((saved as Record<string, unknown>).provider || 'openai')
    const masked = key.length > 8
      ? provider === 'groq'
        ? `gsk_...${key.slice(-4)}`
        : `sk-...${key.slice(-4)}`
      : ''

    return NextResponse.json({
      provider,
      enabled: !!(saved as Record<string, unknown>).enabled,
      model: String((saved as Record<string, unknown>).model || 'gpt-4o-mini'),
      hasKey: key.length > 0,
      keyMasked: masked,
    })
  } catch (error) {
    console.error('Error updating AI config:', error)
    return NextResponse.json({ error: 'Failed to update AI config' }, { status: 500 })
  }
}
```

/api/ai/parse/route.ts 150 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB } from '@/lib/db'
import { AiConfig } from '@/lib/models'
import { getSession } from '@/lib/auth'
import { AI_MODULE_MAP, buildSystemPrompt, coerceFieldValue } from '@/lib/ai-schemas'
import OpenAI from 'openai'

// Force dynamic
export const dynamic = 'force-dynamic'
export const revalidate = 0

// POST /api/ai/parse
// Body: { module: string, text: string }
// Returns: { fields: Record<string, unknown>, raw: Record<string, unknown> }
//
// Calls OpenAI with the module's system prompt + user input and returns the
// extracted field values, coerced to the correct types per the schema.
//
// Any logged-in user can call this — the AI button is shown to everyone,
// but disabled in the UI if no OpenAI key is configured.
export async function POST(request: Request) {
  try {
    await connectDB()

    // Auth gate — any logged-in user can use AI
    const session = await getSession()
    if (!session) {
      return NextResponse.json(
        { error: 'Unauthorized — please log in' },
        { status: 401 }
      )
    }

    const body = await request.json()
    const { module: moduleKey, text } = body as { module?: string; text?: string }

    if (!moduleKey || typeof moduleKey !== 'string') {
      return NextResponse.json(
        { error: 'module is required' },
        { status: 400 }
      )
    }
    if (!text || typeof text !== 'string' || !text.trim()) {
      return NextResponse.json(
        { error: 'text is required' },
        { status: 400 }
      )
    }

    const schema = AI_MODULE_MAP[moduleKey]
    if (!schema) {
      return NextResponse.json(
        { error: `Unknown module: ${moduleKey}` },
        { status: 400 }
      )
    }

    // ── Load AI config (API key + model + provider) ──────────────
    const configDoc = await AiConfig.findOne().lean()
    const apiKey = String((configDoc as Record<string, unknown> | null)?.openaiApiKey || '')
    const enabled = !!(configDoc as Record<string, unknown> | null)?.enabled
    const provider = String((configDoc as Record<string, unknown> | null)?.provider || 'openai')
    const model = String((configDoc as Record<string, unknown> | null)?.model || 'gpt-4o-mini')

    if (!enabled) {
      return NextResponse.json(
        { error: 'AI features are disabled. Ask an admin to enable them in Admin Panel.' },
        { status: 403 }
      )
    }
    if (!apiKey) {
      return NextResponse.json(
        { error: 'No AI API key configured. Ask an admin to add one in Admin Panel.' },
        { status: 403 }
      )
    }

    // ── Build OpenAI-compatible client ──────────────────────────
    // Groq exposes an OpenAI-compatible endpoint at https://api.groq.com/openai/v1
    // — same request/response shapes, just a different baseURL + key prefix (gsk_).
    // OpenAI SDK supports this via the baseURL option.
    //
    // PERFORMANCE: we set a short timeout so the user doesn't wait forever if
```

```
    // the API is slow. Groq typically responds in 500-1500ms; OpenAI in 1-3s.
    // We also use a lower max_tokens for form extraction (responses are tiny
    // JSON objects, no need for 800 tokens).
    const clientOptions: ConstructorParameters<typeof OpenAI>[0] = {
      apiKey,
      timeout: 12_000, // 12s hard cap — fail fast instead of hanging
      maxRetries: 1,   // one retry on transient errors only
    }
    if (provider === 'groq') {
      clientOptions.baseURL = 'https://api.groq.com/openai/v1'
    }
    // For OpenAI, we omit baseURL — the SDK defaults to api.openai.com/v1
    const client = new OpenAI(clientOptions)
    const systemPrompt = buildSystemPrompt(schema)

    // Build a JSON schema for structured output. Each field is optional
    // (the AI omits keys it couldn't extract), so we use additionalProperties
    // false + a permissive object with all keys as optional strings/numbers.
    //
    // PERFORMANCE: temperature=0 for deterministic, fast reasoning. Groq's
    // llama-3.3-70b is incredibly fast at temp=0 — often <800ms.
    // max_tokens=500 is plenty for form fields (response is a tiny JSON).
    const response = await client.chat.completions.create({
      model,
      messages: [
        { role: 'system', content: systemPrompt },
        { role: 'user', content: text.trim() },
      ],
      temperature: 0, // deterministic — fastest + most consistent extraction
      max_tokens: 500,
      response_format: { type: 'json_object' },
    })

    const rawContent = response.choices[0]?.message?.content || '{}'
    let rawJson: Record<string, unknown>
    try {
      rawJson = JSON.parse(rawContent)
    } catch {
      console.error('[POST /api/ai/parse] Failed to parse AI response as JSON:', rawContent)
      return NextResponse.json(
        { error: 'AI returned invalid JSON. Please try rephrasing your input.' },
        { status: 502 }
      )
    }

    // ── Coerce values to correct types + filter unknown keys ──────────
    const coerced: Record<string, unknown> = {}
    for (const field of schema.fields) {
      if (field.key in rawJson) {
        const value = coerceFieldValue(field, rawJson[field.key])
        if (value !== undefined) {
          coerced[field.key] = value
        }
      }
    }

    return NextResponse.json({
      fields: coerced,
      raw: rawJson,
    })
  } catch (error) {
    console.error('Error in /api/ai/parse:', error)
    const message = error instanceof Error ? error.message : 'Failed to parse input'
    return NextResponse.json({ error: message }, { status: 500 })
  }
}
```

/api/ai/test/route.ts 121 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB } from '@/lib/db'
import { AiConfig } from '@/lib/models'
import { getSession } from '@/lib/auth'
import OpenAI from 'openai'

// Force dynamic
export const dynamic = 'force-dynamic'
export const revalidate = 0

// POST /api/ai/test
// Admin-only. Sends a tiny "ping" request to the configured AI provider
// using the saved key+model. Returns 200 with latency if the call succeeds,
// or 4xx/5xx with a human-readable error if it fails.
//
// This is called by the "Test Connection" button in the Admin Panel so the
// admin can immediately see whether their key+model+endpoint combination
// actually works, instead of having to navigate to a form, open the AI
// widget, type something, and try to interpret the failure.
export async function POST() {
  try {
    await connectDB()

    const session = await getSession()
    if (!session || session.role !== 'admin') {
      return NextResponse.json(
        { error: 'Unauthorized — only admins can test AI configuration' },
        { status: 403 }
      )
    }

    const configDoc = await AiConfig.findOne().lean()
    const apiKey = String((configDoc as Record<string, unknown> | null)?.openaiApiKey || '')
    const enabled = !!(configDoc as Record<string, unknown> | null)?.enabled
    const provider = String((configDoc as Record<string, unknown> | null)?.provider || 'openai')
    const model = String((configDoc as Record<string, unknown> | null)?.model || 'gpt-4o-mini')

    if (!apiKey) {
      return NextResponse.json(
        { ok: false, error: 'No API key configured. Please save an API key first.' },
        { status: 400 }
      )
    }
    if (!enabled) {
      return NextResponse.json(
        { ok: false, error: 'AI is disabled. Toggle it on and save first.' },
        { status: 400 }
      )
    }

    // Build the OpenAI-compatible client — Groq uses a different baseURL
    const clientOptions: ConstructorParameters<typeof OpenAI>[0] = {
      apiKey,
      timeout: 10_000,
      maxRetries: 0, // no retries on test — fail fast
    }
    if (provider === 'groq') {
      clientOptions.baseURL = 'https://api.groq.com/openai/v1'
    }
    const client = new OpenAI(clientOptions)

    const t0 = Date.now()
    // Minimal ping — "Reply with OK" with json_object response_format so we
    // exercise the same code path as the real parse endpoint.
    const response = await client.chat.completions.create({
      model,
      messages: [
        {
          role: 'system',
          content:
            'You are a connectivity-test bot. Reply with a JSON object: {"status":"ok"}. Do not say anything else.',
        },
        { role: 'user', content: 'ping' },
      ],
      temperature: 0,
      max_tokens: 50,
      response_format: { type: 'json_object' },
    })
    const latencyMs = Date.now() - t0

    const content = response.choices[0]?.message?.content || ''
    let parsedOk = false
    try {
```

```
        const j = JSON.parse(content)
        parsedOk = j?.status === 'ok' || typeof j === 'object'
      } catch {
        parsedOk = false
      }

      return NextResponse.json({
        ok: true,
        provider,
        model,
        latencyMs,
        responsePreview: content.slice(0, 120),
        parsedOk,
        message: `Success — ${provider === 'groq' ? 'Groq' : 'OpenAI'} responded in ${latencyMs}ms using model "${model}".`,
      })
    } catch (error) {
      console.error('Error in /api/ai/test:', error)

      // Try to extract a useful message from OpenAI SDK errors. The SDK throws
      // APIError subclasses that have .status, .message, and .error.code.
      let message = 'Unknown error'
      let status = 500
      if (error instanceof Error) {
        message = error.message
        // Common patterns:
        //  - "401 Incorrect API key provided"
        //  - "404 The model ... does not exist"
        //  - "429 Rate limit exceeded"
        const m = message.match(/^(\d{3})\s/)
        if (m) status = Number(m[1])
      }
      return NextResponse.json(
        { ok: false, error: message, status },
        { status }
      )
    }
  }
```

/api/auth/forgot-password/request-otp/route.ts 161 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB } from '@/lib/db'
import { User, PasswordReset } from '@/lib/models'
import bcrypt from 'bcryptjs'
import { sendMail, buildOtpEmail, isEmailConfigured } from '@/lib/email'

// ─────────────────────────────────────────────────────────────────────
// POST /api/auth/forgot-password/request-otp
//
// Body: { email: string }
//
// Generates a 6-digit OTP, stores it (bcrypt-hashed) in the PasswordReset
// collection with a 10-minute expiry, and emails it to the user.
//
// SECURITY:
//   • Always returns 200 with a generic message, even if the email does not
//     exist in our user database. This prevents user enumeration via response
//     status / timing. (We still send the OTP only if the user exists; if
//     they don't, we silently skip the send step.)
//   • OTP is hashed with bcrypt before storage — DB leak does not reveal OTPs.
//   • Old unused OTPs for the same email are marked `used` so only the latest
//     one is valid.
//   • Rate-limited: at most 1 OTP per email per 60 seconds (enforced via the
//     createdAt index on PasswordReset).
// ─────────────────────────────────────────────────────────────────────

const OTP_TTL_MINUTES = 10
const RESEND_COOLDOWN_SECONDS = 60

function generateOtp(): string {
  // 6-digit numeric OTP — cryptographically random.
  // node:crypto.randomInt gives a uniform distribution (no modulo bias).
  // eslint-disable-next-line @typescript-eslint/no-require-imports
  const { randomInt } = require('crypto') as { randomInt: (min: number, max: number) => number }
  const n = randomInt(0, 1_000_000)
  return n.toString().padStart(6, '0')
}

export async function POST(request: Request) {
  try {
    // Validate input FIRST — so bad input is rejected with 400 even if DB
    // is temporarily unavailable.
    const body = await request.json().catch(() => ({}))
    const email = typeof body?.email === 'string' ? body.email.toLowerCase().trim() : ''

    // Basic email format check
    if (!email || !/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email) || email.length > 256) {
      return NextResponse.json(
        { error: 'Please enter a valid email address.' },
        { status: 400 }
      )
    }

    await connectDB()

    // Rate-limit check — was an OTP issued for this email in the last 60s?
    const cooldownAt = new Date(Date.now() - RESEND_COOLDOWN_SECONDS * 1000)
    const recent = await PasswordReset.findOne({
      email,
      createdAt: { $gt: cooldownAt },
    }).sort({ createdAt: -1 })
    if (recent) {
      const secsLeft = Math.ceil(
        (recent.createdAt.getTime() + RESEND_COOLDOWN_SECONDS * 1000 - Date.now()) / 1000
      )
      return NextResponse.json(
        {
          error: `Please wait ${secsLeft}s before requesting another OTP.`,
          cooldownSeconds: secsLeft,
        },
        { status: 429 }
      )
    }

    // Look up the user. We DON'T reveal whether the email exists in the
    // response — but we only SEND an OTP if the user exists. This is the
    // standard trade-off: a sophisticated attacker could still infer
    // existence from response timing (DB lookup time) or email delivery
    // delays, but the typical enumeration attack (visible error messages)
    // is blocked.
    const user = await User.findOne({ email })
    if (!user) {
      // Silently succeed — but don't send any email. Attacker learns nothing.
```

```
      return NextResponse.json({
        message: 'If the email exists in our system, an OTP has been sent.',
        email,
        expiryMinutes: OTP_TTL_MINUTES,
      })
    }

    if (!user.active) {
      // Same — don't reveal that the account is disabled. Silently succeed.
      return NextResponse.json({
        message: 'If the email exists in our system, an OTP has been sent.',
        email,
        expiryMinutes: OTP_TTL_MINUTES,
      })
    }

    // Invalidate any previous unused OTPs for this email
    await PasswordReset.updateMany(
      { email, used: false },
      { $set: { used: true } }
    )

    // Generate + hash OTP
    const otp = generateOtp()
    const otpHash = await bcrypt.hash(otp, 10)
    const expiresAt = new Date(Date.now() + OTP_TTL_MINUTES * 60 * 1000)

    await PasswordReset.create({
      email,
      otpHash,
      expiresAt,
      verified: false,
      used: false,
      attempts: 0,
    })

    // Send the email
    const { subject, text, html } = buildOtpEmail({
      name: user.name,
      otp,
      expiryMinutes: OTP_TTL_MINUTES,
    })
    const result = await sendMail({ to: email, subject, text, html })

    // In dev mode (no SMTP configured), surface the OTP so the developer can
    // test the flow locally. In production, this is `undefined`.
    const devPreview = !result.delivered && result.devPreview
      ? result.devPreview
      : undefined

    if (!result.delivered && !devPreview) {
      // SMTP was configured but send failed — surface the error to the user
      // so they can contact admin. Don't expose internal details.
      return NextResponse.json(
        { error: 'Failed to send OTP email. Please try again or contact admin.' },
        { status: 500 }
      )
    }

    return NextResponse.json({
      message: devPreview
        ? '[DEV] OTP logged to server console. Configure EMAIL_USER + EMAIL_PASS for production.'
        : 'OTP sent to your email.',
      email,
      expiryMinutes: OTP_TTL_MINUTES,
      emailConfigured: isEmailConfigured(),
      // Only include devPreview when SMTP is NOT configured (dev mode)
      ...(devPreview ? { devPreview } : {}),
    })
  } catch (error) {
    console.error('Error in request-otp:', error)
    return NextResponse.json(
      { error: 'Failed to send OTP. Please try again.' },
      { status: 500 }
    )
  }
}
```

/api/auth/forgot-password/reset/route.ts 161 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB } from '@/lib/db'
import { User, PasswordReset } from '@/lib/models'
import bcrypt from 'bcryptjs'
import { jwtVerify } from 'jose'

// ─────────────────────────────────────────────────────────────────
// POST /api/auth/forgot-password/reset
//
// Body: { email: string, resetToken: string, newPassword: string, confirmPassword: string }
//
// Validates the resetToken (issued by /verify-otp), checks that the
// corresponding PasswordReset doc is still in `verified && !used` state,
// then updates the user's password.
//
// SECURITY:
//   • resetToken is a signed JWT with `purpose: 'password-reset'` — cannot
//     be reused as a login token.
//   • The doc must be marked `verified: true` and `used: false`. After a
//     successful reset, the doc is marked `used: true` so the same token
//     cannot be replayed.
//   • newPassword must match confirmPassword (client enforces this too).
//   • Password min length 6 (matches the /api/users route policy).
//   • bcrypt rounds: 12 (matches the rest of the auth system).
// ─────────────────────────────────────────────────────────────────

const PASSWORD_MIN_LENGTH = 6
const PASSWORD_MAX_LENGTH = 256

function getSecret(): Uint8Array {
  const raw = process.env.JWT_SECRET
  if (!raw || raw.length < 16) {
    return new TextEncoder().encode('veda-dev-only-ephemeral-secret-do-not-use-in-prod')
  }
  return new TextEncoder().encode(raw)
}

export async function POST(request: Request) {
  try {
    // Validate input FIRST.
    const body = await request.json().catch(() => ({}))
    const email = typeof body?.email === 'string' ? body.email.toLowerCase().trim() : ''
    const resetToken = typeof body?.resetToken === 'string' ? body.resetToken.trim() : ''
    const newPassword = typeof body?.newPassword === 'string' ? body.newPassword : ''
    const confirmPassword = typeof body?.confirmPassword === 'string' ? body.confirmPassword : ''

    if (!email || !resetToken) {
      return NextResponse.json(
        { error: 'Missing email or reset token.' },
        { status: 400 }
      )
    }

    if (!newPassword || !confirmPassword) {
      return NextResponse.json(
        { error: 'Please enter and confirm your new password.' },
        { status: 400 }
      )
    }
    if (newPassword !== confirmPassword) {
      return NextResponse.json(
        { error: 'Passwords do not match.' },
        { status: 400 }
      )
    }
    if (newPassword.length < PASSWORD_MIN_LENGTH) {
      return NextResponse.json(
        { error: `Password must be at least ${PASSWORD_MIN_LENGTH} characters long.` },
        { status: 400 }
      )
    }
    if (newPassword.length > PASSWORD_MAX_LENGTH) {
      return NextResponse.json(
        { error: 'Password is too long.' },
        { status: 400 }
      )
    }

    await connectDB()

    // Verify the resetToken
    let payload: { email?: string; docId?: string; purpose?: string }
    try {
```

```
      const { payload: p } = await jwtVerify(resetToken, getSecret())
      payload = p as unknown as typeof payload
    } catch {
      return NextResponse.json(
        { error: 'Invalid or expired reset token. Please restart the forgot-password flow.' },
        { status: 401 }
      )
    }

    if (payload.purpose !== 'password-reset') {
      return NextResponse.json(
        { error: 'Invalid reset token.' },
        { status: 401 }
      )
    }
    if (!payload.email || payload.email !== email) {
      return NextResponse.json(
        { error: 'Email does not match the reset token.' },
        { status: 401 }
      )
    }
    if (!payload.docId) {
      return NextResponse.json(
        { error: 'Malformed reset token.' },
        { status: 401 }
      )
    }

    // Verify the PasswordReset doc is still in verified && !used state
    const doc = await PasswordReset.findById(payload.docId)
    if (!doc || !doc.verified || doc.used) {
      return NextResponse.json(
        { error: 'Reset session is no longer valid. Please restart the forgot-password flow.' },
        { status: 401 }
      )
    }
    if (doc.email !== email) {
      return NextResponse.json(
        { error: 'Email mismatch in reset record.' },
        { status: 401 }
      )
    }

    // Find the user
    const user = await User.findOne({ email })
    if (!user) {
      return NextResponse.json(
        { error: 'User not found. Please contact admin.' },
        { status: 404 }
      )
    }
    if (!user.active) {
      return NextResponse.json(
        { error: 'Account is disabled. Please contact admin.' },
        { status: 403 }
      )
    }

    // Update the password
    const newHash = await bcrypt.hash(newPassword, 12)
    await User.updateOne({ _id: user._id }, { $set: { password: newHash } })

    // Mark the PasswordReset doc as used so the token can't be replayed
    await PasswordReset.updateOne({ _id: doc._id }, { $set: { used: true } })

    return NextResponse.json({
      message: 'Password updated successfully. You can now log in with your new password.',
      email,
    })
  } catch (error) {
    console.error('Error in reset-password:', error)
    return NextResponse.json(
      { error: 'Failed to reset password. Please try again.' },
      { status: 500 }
    )
  }
}
```

/api/auth/forgot-password/verify-otp/route.ts 136 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB } from '@/lib/db'
import { PasswordReset } from '@/lib/models'
import bcrypt from 'bcryptjs'
import { SignJWT } from 'jose'

// ─────────────────────────────────────────────────────────────────────
// POST /api/auth/forgot-password/verify-otp
//
// Body: { email: string, otp: string }
//
// Validates the OTP against the latest unused PasswordReset doc for that email.
// On success: marks the doc as `verified` and returns a short-lived JWT
// `resetToken` (10 min) that the client must include in the subsequent
// /reset call. The resetToken encodes the email + the docId so the reset
// endpoint can confirm the OTP was actually verified.
//
// SECURITY:
//   • bcrypt compare against the stored hash (no plaintext OTP in DB).
//   • Max 5 wrong attempts per OTP doc — after that the doc is invalidated
//     and the user must request a new OTP.
//   • OTP expiry (10 min) is enforced.
//   • resetToken is signed with the same JWT_SECRET as login tokens — it's
//     a different `purpose` claim so it cannot be reused as a login token.
// ─────────────────────────────────────────────────────────────────────

const MAX_ATTEMPTS = 5

function getSecret(): Uint8Array {
  const raw = process.env.JWT_SECRET
  if (!raw || raw.length < 16) {
    return new TextEncoder().encode('veda-dev-only-ephemeral-secret-do-not-use-in-prod')
  }
  return new TextEncoder().encode(raw)
}

export async function POST(request: Request) {
  try {
    // Validate input FIRST.
    const body = await request.json().catch(() => ({}))
    const email = typeof body?.email === 'string' ? body.email.toLowerCase().trim() : ''
    const otp = typeof body?.otp === 'string' ? body.otp.trim() : ''

    if (!email || !/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email)) {
      return NextResponse.json({ error: 'Invalid email.' }, { status: 400 })
    }
    if (!otp || !/^\d{6}$/.test(otp)) {
      return NextResponse.json(
        { error: 'OTP must be a 6-digit number.' },
        { status: 400 }
      )
    }

    await connectDB()

    // Find the latest unused, unverified PasswordReset doc for this email
    const doc = await PasswordReset.findOne({
      email,
      used: false,
      verified: false,
    }).sort({ createdAt: -1 })

    if (!doc) {
      return NextResponse.json(
        { error: 'No active OTP found. Please request a new OTP.' },
        { status: 404 }
      )
    }

    // Expiry check
    if (doc.expiresAt.getTime() < Date.now()) {
      await PasswordReset.updateOne({ _id: doc._id }, { $set: { used: true } })
      return NextResponse.json(
        { error: 'OTP has expired. Please request a new OTP.' },
        { status: 410 }
      )
    }

    // Attempt-limit check
    if (doc.attempts >= MAX_ATTEMPTS) {
      await PasswordReset.updateOne({ _id: doc._id }, { $set: { used: true } })
      return NextResponse.json(
        { error: 'Too many wrong attempts. Please request a new OTP.' },
```

```
          { status: 429 }
        )
      }

      // bcrypt verify
      const ok = await bcrypt.compare(otp, doc.otpHash).catch(() => false)
      if (!ok) {
        // Increment attempts
        await PasswordReset.updateOne(
          { _id: doc._id },
          { $inc: { attempts: 1 } }
        )
        const attemptsLeft = MAX_ATTEMPTS - (doc.attempts + 1)
        return NextResponse.json(
          {
            error: `Wrong OTP. ${attemptsLeft} attempt(s) remaining.`,
            attemptsLeft: Math.max(0, attemptsLeft),
          },
          { status: 401 }
        )
      }

      // Success — mark as verified
      await PasswordReset.updateOne(
        { _id: doc._id },
        { $set: { verified: true } }
      )

      // Issue a short-lived resetToken (10 min) tied to this doc
      const resetToken = await new SignJWT({
        email,
        docId: doc._id.toString(),
        purpose: 'password-reset',
      })
        .setProtectedHeader({ alg: 'HS256' })
        .setExpirationTime('10m')
        .setIssuedAt()
        .sign(getSecret())

      return NextResponse.json({
        message: 'OTP verified. You can now set a new password.',
        resetToken,
        email,
      })
    } catch (error) {
      console.error('Error in verify-otp:', error)
      return NextResponse.json(
        { error: 'Failed to verify OTP. Please try again.' },
        { status: 500 }
      )
    }
  }
```

/api/auth/init/route.ts 102 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB } from '@/lib/db'
import { User, Company } from '@/lib/models'
import bcrypt from 'bcryptjs'

// ─────────────────────────────────────────────────────────────────────
// POST /api/auth/init
//
// Seeds the very first admin user + company record. This is intended to be
// called ONCE during initial ERP setup, from the login screen's "Initialize
// system" button.
//
// SECURITY:
//   The endpoint refuses to run if any User documents already exist in the
//   database (so it cannot be used to inject a second admin after setup).
//   We previously had no other gate — anyone hitting this endpoint before
//   the legitimate admin could claim the admin@veda.com account.
//
//   We now additionally support an optional FIRST_RUN_KEY env var. If set,
//   the caller must supply it via the X-First-Run-Key header or ?key=...
//   query param. This lets a deployment lock down initialization.
// ─────────────────────────────────────────────────────────────────────
export async function POST(request: Request) {
  try {
    await connectDB()

    // Optional env-var gate
    const expectedKey = process.env.FIRST_RUN_KEY
    if (expectedKey && expectedKey.length > 0) {
      const url = new URL(request.url)
      const headerKey = request.headers.get('x-first-run-key') || ''
      const queryKey = url.searchParams.get('key') || ''
      if (headerKey !== expectedKey && queryKey !== expectedKey) {
        return NextResponse.json(
          { error: 'Unauthorized — first-run key required' },
          { status: 403 }
        )
      }
    }

    // Refuse if users already exist — prevents re-init / account injection
    const existingCount = await User.countDocuments({})
    if (existingCount > 0) {
      return NextResponse.json(
        { message: 'Users already exist. Initialization skipped.' },
        { status: 400 }
      )
    }

    // If JWT_SECRET is not set, warn loudly — admin won't be able to log in
    if (!process.env.JWT_SECRET || process.env.JWT_SECRET.length < 16) {
      console.error(
        'FATAL: JWT_SECRET environment variable is missing or too short. ' +
        'The admin user will be created but no one will be able to log in. ' +
        'Set JWT_SECRET (e.g. `openssl rand -hex 32`) before initializing.'
      )
    }

    const hashedPassword = await bcrypt.hash('admin123', 12)

    const admin = await User.create({
      name: 'Admin',
      email: 'dataanalogydirector@gmail.com',
      password: hashedPassword,
      role: 'admin',
    })

    // Initialize company with Veda branding
    const existingCompany = await Company.findOne({})
    if (!existingCompany) {
      const vedaLogoSvg = `<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 200 200" fill="none"><rect width="200" height="200" rx="40" fill="#059669
      const vedaLogoDataUrl = `${Buffer.from(vedaLogoSvg).toString('base64')}`

      await Company.create({
        name: 'Veda Enterprises',
        tagline: 'Building the future, one brick at a time',
        logoUrl: vedaLogoDataUrl,
        setupComplete: true,
      })
    }

    return NextResponse.json(
      {
```

```
        message: 'Default admin user created successfully',
        user: {
          id: admin._id.toString(),
          name: admin.name,
          email: admin.email,
          role: admin.role,
        },
      },
      { status: 201 }
    )
  } catch (error) {
    console.error('Error initializing admin user:', error)
    return NextResponse.json(
      { error: 'Failed to initialize admin user' },
      { status: 500 }
    )
  }
}
```

/api/auth/login/route.ts 115 lines

```
import { NextResponse } from 'next/server'
import { connectDB, toObject } from '@/lib/db'
import { User } from '@/lib/models'
import bcrypt from 'bcryptjs'
import { signToken } from '@/lib/auth'

// Precomputed dummy hash — used when user not found so timing matches a real
// bcrypt.compare call. Prevents user enumeration via response timing.
// This is the bcrypt hash of the string "dummy" with 10 rounds.
const DUMMY_HASH = '$2a$10$CwTycUXWue0Thq9StjUM0uJ8eVjP3wW3P3lF9lJl9eVjP3wW3P3lF'

export async function POST(request: Request) {
  try {
    await connectDB()

    const body = await request.json()
    const { email, password } = body

    if (!email || !password) {
      return NextResponse.json(
        { error: 'Email and password are required' },
        { status: 400 }
      )
    }
    if (typeof email !== 'string' || typeof password !== 'string') {
      return NextResponse.json(
        { error: 'Invalid credentials format' },
        { status: 400 }
      )
    }
    // Cap input length to prevent abuse
    if (email.length > 256 || password.length > 256) {
      return NextResponse.json(
        { error: 'Input too long' },
        { status: 400 }
      )
    }

    const user = await User.findOne({ email: email.toLowerCase().trim() })

    if (!user) {
      // Run bcrypt against a dummy hash so timing matches the "user found,
      // wrong password" case. This prevents user enumeration.
      await bcrypt.compare(password, DUMMY_HASH).catch(() => {})
      return NextResponse.json(
        { error: 'Invalid email or password' },
        { status: 401 }
      )
    }

    if (!user.active) {
      // Still run bcrypt to keep timing constant
      await bcrypt.compare(password, DUMMY_HASH).catch(() => {})
      return NextResponse.json(
        { error: 'Account is disabled' },
        { status: 401 }
      )
    }

    const isValid = await bcrypt.compare(password, user.password)

    if (!isValid) {
      return NextResponse.json(
        { error: 'Invalid email or password' },
        { status: 401 }
      )
    }

    const token = await signToken({
      userId: user._id.toString(),
      email: user.email,
      role: user.role,
      name: user.name,
    })

    const response = NextResponse.json(
      {
        message: 'Login successful',
        user: {
          id: user._id.toString(),
          name: user.name,
          email: user.email,
          role: user.role,
```

```
        },
      },
      { status: 200 }
    )

    // Determine cookie security. In production we default to Secure:true
    // (the cookie only travels over HTTPS). Behind a proxy that terminates
    // TLS (Vercel, Caddy), we trust x-forwarded-proto.
    const xfp = request.headers.get('x-forwarded-proto')
    const isHttps =
      process.env.NODE_ENV === 'production' ||
      xfp === 'https' ||
      request.url.startsWith('https')

    response.cookies.set('token', token, {
      httpOnly: true,
      secure: isHttps,
      sameSite: 'lax',
      maxAge: 86400,
      path: '/',
    })

    return response
  } catch (error) {
    console.error('Error during login:', error)
    return NextResponse.json(
      { error: 'Login failed' },
      { status: 500 }
    )
  }
}
```

/api/auth/me/route.ts 15 lines

```ts
import { NextResponse } from 'next/server'
import { getSession } from '@/lib/auth'

export async function GET() {
  try {
    const session = await getSession()
    if (!session) {
      return NextResponse.json({ error: 'Not authenticated' }, { status: 401 })
    }
    return NextResponse.json({ user: session })
  } catch {
    return NextResponse.json({ error: 'Not authenticated' }, { status: 401 })
  }
}
```

/api/auth/reset-admin/route.ts 112 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB } from '@/lib/db'
import { User } from '@/lib/models'
import bcrypt from 'bcryptjs'
import { timingSafeEqualStr } from '@/lib/auth'

// ─────────────────────────────────────────────────────────────────────
// EMERGENCY ADMIN PASSWORD RESET
//
// WHY THIS EXISTS:
//   The backup-export route strips User.password for security. When a backup
//   is restored, the sanitizeRow() helper replaces missing passwords with a
//   random placeholder. The user cannot know this random password, so they
//   get locked out of their own ERP after a restore.
//
// WHAT THIS DOES:
//   - Resets the password of the admin@veda.com user (or the first admin
//     user if admin@veda.com doesn't exist) back to "admin123".
//   - Re-activates the account if it was disabled.
//   - Returns enough info for the user to log back in.
//
// SECURITY:
//   This endpoint REQUIRES the env var EMERGENCY_RESET_KEY to be set, and
//   the caller must supply it via the X-Emergency-Reset-Key header (or
//   ?key=... query param). Comparison is timing-safe.
//
//   If EMERGENCY_RESET_KEY is NOT set in the environment, the endpoint
//   refuses to run — returning 503. This is the inverse of the previous
//   behaviour (which was "open by default"), and prevents anonymous
//   attackers from resetting the admin password to a known value.
// ─────────────────────────────────────────────────────────────────────
export async function POST(request: Request) {
  try {
    await connectDB()

    // Require env-var gate — refuse to run if not set
    const expectedKey = process.env.EMERGENCY_RESET_KEY
    if (!expectedKey || expectedKey.length < 8) {
      return NextResponse.json(
        {
          error:
            'Emergency reset is disabled. Set the EMERGENCY_RESET_KEY environment variable (>=8 chars) to enable this endpoint.',
        },
        { status: 503 }
      )
    }

    const url = new URL(request.url)
    const headerKey = request.headers.get('x-emergency-reset-key') || ''
    const queryKey = url.searchParams.get('key') || ''

    // Timing-safe comparison — both header and query are checked, but a
    // passing match on EITHER is enough. Both must fail to reject.
    const headerOk = headerKey.length > 0 && timingSafeEqualStr(headerKey, expectedKey)
    const queryOk = queryKey.length > 0 && timingSafeEqualStr(queryKey, expectedKey)
    if (!headerOk && !queryOk) {
      return NextResponse.json(
        { error: 'Unauthorized — emergency reset key required' },
        { status: 403 }
      )
    }

    // Find admin@veda.com first; fall back to any admin user
    let admin = await User.findOne({ email: 'admin@veda.com' })
    let usedFallback = false
    if (!admin) {
      admin = await User.findOne({ role: 'admin' })
      usedFallback = true
    }

    if (!admin) {
      // No admin user exists — create one with default credentials
      const hashedPassword = await bcrypt.hash('admin123', 12)
      const newAdmin = await User.create({
        name: 'Admin',
        email: 'admin@veda.com',
        password: hashedPassword,
        role: 'admin',
        active: true,
      })
      return NextResponse.json({
        message: 'No admin user found. Created a new admin with default credentials.',
        credentials: { email: 'admin@veda.com', password: 'admin123' },
```

```
        userId: newAdmin._id.toString(),
        action: 'created',
      })
    }

    // Reset password and re-activate
    const hashedPassword = await bcrypt.hash('admin123', 12)
    admin.password = hashedPassword
    admin.active = true
    await admin.save()

    return NextResponse.json({
      message: 'Admin password reset to default. Please login and change it immediately.',
      credentials: { email: admin.email, password: 'admin123' },
      userId: admin._id.toString(),
      action: 'reset',
      note: usedFallback
        ? `admin@veda.com not found — reset the first admin user (${admin.email}) instead.`
        : undefined,
    })
  } catch (error) {
    console.error('Error resetting admin password:', error)
    return NextResponse.json(
      { error: 'Failed to reset admin password: ' + (error instanceof Error ? error.message : 'unknown error') },
      { status: 500 }
    )
  }
}
```

/api/bills/[id]/route.ts — 190 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB, toObject } from '@/lib/db'
import { Bill, Payment } from '@/lib/models'
import { requireSession, requireAdmin } from '@/lib/auth'
import { normalizeDate } from '@/lib/date-utils'

// GET single bill
export async function GET(
  _request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const session = await requireSession()
    if (session instanceof NextResponse) return session

    await connectDB()

    const { id } = await params
    const bill = await Bill.findById(id).lean()
    if (!bill) {
      return NextResponse.json({ error: 'Bill not found' }, { status: 404 })
    }
    return NextResponse.json({ bill: toObject(bill) })
  } catch {
    return NextResponse.json({ error: 'Failed to fetch bill' }, { status: 500 })
  }
}

// PUT — update bill
export async function PUT(
  request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const session = await requireSession()
    if (session instanceof NextResponse) return session

    await connectDB()

    const { id } = await params
    const body = await request.json()
    // Normalize date to canonical YYYY-MM-DD
    // (handles dd-mm-yyyy, dd/mm/yyyy, Excel serials, Date objects, etc.)
    if (body.date) body.date = normalizeDate(body.date)
    if (body.dueDate) body.dueDate = normalizeDate(body.dueDate)

    const existing = await Bill.findById(id)
    if (!existing) {
      return NextResponse.json({ error: 'Bill not found' }, { status: 404 })
    }

    // Recalculate amounts if items changed
    const items = Array.isArray(body.items) ? body.items : existing.items
    const subTotal = items.reduce((sum: number, item: Record<string, number>) => sum + (Number(item.amount) || 0), 0)
    const discountPercent = body.discountPercent !== undefined ? Number(body.discountPercent) : existing.discountPercent
    const discountAmount = body.discountAmount !== undefined ? Number(body.discountAmount) : (subTotal * discountPercent / 100)
    const taxableAmount = subTotal - discountAmount

    const cgstPercent = body.cgstPercent !== undefined ? Number(body.cgstPercent) : existing.cgstPercent
    const cgstAmount = taxableAmount * cgstPercent / 100
    const sgstPercent = body.sgstPercent !== undefined ? Number(body.sgstPercent) : existing.sgstPercent
    const sgstAmount = taxableAmount * sgstPercent / 100
    const igstPercent = body.igstPercent !== undefined ? Number(body.igstPercent) : existing.igstPercent
    const igstAmount = taxableAmount * igstPercent / 100

    const totalBeforeRound = taxableAmount + cgstAmount + sgstAmount + igstAmount
    const grandTotal = Math.round(totalBeforeRound)
    const roundOff = grandTotal - totalBeforeRound

    const paidAmount = body.paidAmount !== undefined ? Number(body.paidAmount) : existing.paidAmount
    const balanceAmount = grandTotal - paidAmount

    // customerId resolution: explicit null means "unlink", undefined means "no change"
    const customerId = body.customerId !== undefined ? (body.customerId || null) : existing.customerId

    const updateData: Record<string, unknown> = {
      ...body,
      customerId,
      items,
      subTotal,
      discountPercent,
      discountAmount,
      taxableAmount,
```

```
      cgstPercent, cgstAmount,
      sgstPercent, sgstAmount,
      igstPercent, igstAmount,
      roundOff,
      grandTotal,
      paidAmount,
      balanceAmount,
    }

    if (body.status !== undefined) {
      updateData.status = body.status
    } else if (paidAmount >= grandTotal && grandTotal > 0) {
      updateData.status = 'paid'
    } else if (paidAmount > 0) {
      updateData.status = 'partial'
    }

    const updated = await Bill.findByIdAndUpdate(id, updateData, { new: true })

    // ── Auto-sync Payment ──────────────────────────────────────
    // Keep the Payment mirror in sync with the bill's paidAmount + customer.
    // Three cases:
    //   1. paidAmount > 0 && customerId set  → upsert Payment (create or update)
    //   2. paidAmount === 0 || customerId null → delete any existing synced Payment
    //   3. customer changed → delete old Payment, create new under new customer
    // SKIPPED for quotations — quotations are not real invoices, so they
    // never have a "paid amount" to mirror into the Payments module. Any
    // stale Payment from a previously-invoiced record (e.g. billType was
    // changed from 'sales' to 'quotation') is still cleaned up below.
    const isQuotation = (updateData.billType || existing.billType) === 'quotation'
    try {
      const existingPayment = await Payment.findOne({ billId: existing._id })

      if (customerId && paidAmount > 0 && !isQuotation) {
        const paymentType = body.paymentMode || existing.paymentMode || 'Cash'
        const remarks = `Auto-synced from bill ${existing.billNumber}`
        if (existingPayment) {
          // Update in place — handles amount / customer / mode changes
          await Payment.findByIdAndUpdate(existingPayment._id, {
            customerId,
            paymentType,
            amount: paidAmount,
            date: body.date || existing.date,
            remarks,
          })
        } else {
          // Customer re-linked or first time paidAmount set — create fresh
          await Payment.create({
            customerId,
            paymentType,
            amount: paidAmount,
            date: body.date || existing.date,
            remarks,
            billId: existing._id,
            billNumber: existing.billNumber,
          })
        }
      } else {
        // paidAmount went to 0 OR customer was unlinked → remove the mirror
        if (existingPayment) {
          await Payment.findByIdAndDelete(existingPayment._id)
        }
      }
    } catch (syncErr) {
      // Sync failure is logged but does NOT fail the bill update — the bill
      // is the source of truth, the Payment is a convenience mirror.
      console.error('Bill → Payment auto-sync failed on update:', syncErr)
    }

    return NextResponse.json({ bill: toObject(updated) })
  } catch (error) {
    console.error('Error updating bill:', error)
    return NextResponse.json({ error: 'Failed to update bill' }, { status: 500 })
  }
}

// DELETE bill — admin-only (per canPerform map, only admin can delete bills)
export async function DELETE(
  _request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const auth = await requireAdmin()
    if (auth instanceof NextResponse) return auth

    await connectDB()
```

```
    const { id } = await params
    const deleted = await Bill.findByIdAndDelete(id)
    if (!deleted) {
      return NextResponse.json({ error: 'Bill not found' }, { status: 404 })
    }

    // Cascade: remove the auto-synced Payment mirror (if any). Manual payments
    // created via /api/payments have billId = null and are never touched here.
    try {
      await Payment.deleteMany({ billId: deleted._id })
    } catch (syncErr) {
      console.error('Bill → Payment cascade delete failed:', syncErr)
    }

    return NextResponse.json({ message: 'Bill deleted successfully' })
  } catch {
    return NextResponse.json({ error: 'Failed to delete bill' }, { status: 500 })
  }
}
```

/api/bills/route.ts 169 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB, toObject } from '@/lib/db'
import { Bill, Company, Payment } from '@/lib/models'
import { requireSession } from '@/lib/auth'
import { normalizeDate } from '@/lib/date-utils'

// Force dynamic — never cache bill list responses
export const dynamic = 'force-dynamic'
export const revalidate = 0

// GET — list all bills (with optional filter by type/status/search)
export async function GET(request: Request) {
  try {
    const session = await requireSession()
    if (session instanceof NextResponse) return session

    await connectDB()

    const { searchParams } = new URL(request.url)
    const billType = searchParams.get('billType')
    const status = searchParams.get('status')
    const search = searchParams.get('search')?.trim()

    // Build query — search matches billNumber OR party name (toName) OR
    // customer phone (toPhone). Case-insensitive regex.
    const query: Record<string, unknown> = {}
    if (billType) query.billType = billType
    if (status) query.status = status
    if (search) {
      // Escape regex metacharacters to prevent ReDoS
      const safe = search.replace(/[.*+?^${}()|[\]\\]/g, '\\$&')
      query.$or = [
        { billNumber: { $regex: safe, $options: 'i' } },
        { toName: { $regex: safe, $options: 'i' } },
        { toPhone: { $regex: safe, $options: 'i' } },
      ]
    }

    // Cap at 50 when searching (dropdown UX), otherwise return all
    const limit = search ? 50 : 0
    const bills = await Bill.find(query)
      .sort({ createdAt: -1 })
      .limit(limit)
      .lean()
    const res = NextResponse.json({ bills: toObject(bills) })
    res.headers.set('Cache-Control', 'no-store, no-cache, must-revalidate, max-age=0')
    return res
  } catch (error) {
    console.error('Error fetching bills:', error)
    return NextResponse.json({ error: 'Failed to fetch bills' }, { status: 500 })
  }
}

// POST — create new bill
export async function POST(request: Request) {
  try {
    const session = await requireSession()
    if (session instanceof NextResponse) return session

    await connectDB()

    const body = await request.json()
    // Normalize date to canonical YYYY-MM-DD
    // (handles dd-mm-yyyy, dd/mm/yyyy, Excel serials, Date objects, etc.)
    if (body.date) body.date = normalizeDate(body.date)
    if (body.dueDate) body.dueDate = normalizeDate(body.dueDate)

    // Generate bill number: BILL-YYYYMM-0001 for normal bills,
    // QUO-YYYYMM-0001 for quotations (separate prefix + counter).
    const isQuotation = body.billType === 'quotation'
    const prefix = isQuotation ? 'QUO' : 'BILL'
    const countQuery = isQuotation ? { billType: 'quotation' } : {}
    const count = await Bill.countDocuments(countQuery)
    const now = new Date()
    const yyyymm = `${now.getFullYear()}${String(now.getMonth() + 1).padStart(2, '0')}`
    const billNumber = `${prefix}-${yyyymm}-${String(count + 1).padStart(4, '0')}`

    // Get company info for "from" fields (defaults)
    const company = await Company.findOne({})
    const fromName = body.fromName || company?.name || 'Veda Enterprises'
    const fromAddress = body.fromAddress || [company?.address, company?.city, company?.state, company?.pincode].filter(Boolean).join(', ')
    const fromGst = body.fromGst || company?.gstNumber || ''
    const fromPhone = body.fromPhone || company?.phone || ''
```

```
    // Calculate amounts if items present
    const items = Array.isArray(body.items) ? body.items : []
    const subTotal = items.reduce((sum: number, item: Record<string, number>) => sum + (Number(item.amount) || 0), 0)
    const discountPercent = Number(body.discountPercent) || 0
    const discountAmount = Number(body.discountAmount) || (subTotal * discountPercent / 100)
    const taxableAmount = subTotal - discountAmount

    const cgstPercent = Number(body.cgstPercent) || 0
    const cgstAmount = Number(body.cgstAmount) || (taxableAmount * cgstPercent / 100)
    const sgstPercent = Number(body.sgstPercent) || 0
    const sgstAmount = Number(body.sgstAmount) || (taxableAmount * sgstPercent / 100)
    const igstPercent = Number(body.igstPercent) || 0
    const igstAmount = Number(body.igstAmount) || (taxableAmount * igstPercent / 100)

    const totalBeforeRound = taxableAmount + cgstAmount + sgstAmount + igstAmount
    const grandTotal = Math.round(totalBeforeRound)
    const roundOff = grandTotal - totalBeforeRound

    const paidAmount = Number(body.paidAmount) || 0
    const balanceAmount = grandTotal - paidAmount

    // customerId is optional — may be null when the bill is for a walk-in party
    const customerId = body.customerId || null

    const bill = await Bill.create({
      billNumber,
      billType: body.billType || 'sales',
      date: body.date || new Date().toISOString().split('T')[0],
      dueDate: body.dueDate || '',
      customerId,
      fromName, fromAddress, fromGst, fromPhone,
      toName: body.toName,
      toAddress: body.toAddress || '',
      toGst: body.toGst || '',
      toPhone: body.toPhone || '',
      items,
      subTotal,
      discountPercent,
      discountAmount,
      taxableAmount,
      cgstPercent, cgstAmount,
      sgstPercent, sgstAmount,
      igstPercent, igstAmount,
      roundOff,
      grandTotal,
      paidAmount,
      balanceAmount,
      paymentMode: body.paymentMode || 'Cash',
      notes: body.notes || '',
      terms: body.terms || company?.terms || '',
      status: paidAmount >= grandTotal && grandTotal > 0 ? 'paid' : (paidAmount > 0 ? 'partial' : 'draft'),
      createdBy: session.name,
    })

    // Auto-sync Payment: if a customer is linked AND there's a non-zero paid
    // amount, create a corresponding Payment row so the receipt shows up in
    // the Payments module without manual entry. The Payment carries billId so
    // future updates / deletes on this Bill propagate atomically.
    // SKIPPED for quotations — quotations are not real invoices, so there's
    // no "paid amount" to mirror into the Payments module.
    if (customerId && paidAmount > 0 && !isQuotation) {
      try {
        await Payment.create({
          customerId,
          paymentType: body.paymentMode || 'Cash',
          amount: paidAmount,
          date: bill.date,
          remarks: `Auto-synced from bill ${bill.billNumber}`,
          billId: bill._id,
          billNumber: bill.billNumber,
        })
      } catch (syncErr) {
        // Sync failure is logged but must NOT fail the bill creation — the
        // bill is the source of truth, the Payment is a convenience mirror.
        console.error('Bill → Payment auto-sync failed on create:', syncErr)
      }
    }

    return NextResponse.json({ bill: toObject(bill) }, { status: 201 })
  } catch (error) {
    console.error('Error creating bill:', error)
    return NextResponse.json({ error: 'Failed to create bill' }, { status: 500 })
  }
}
```

/api/cement-purchase/[id]/route.ts 105 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB, toObject } from '@/lib/db'
import { CementPurchase } from '@/lib/models'
import { requireSession, requireRole } from '@/lib/auth'
import { normalizeDate } from '@/lib/date-utils'

// Force dynamic — never cache individual responses
export const dynamic = 'force-dynamic'
export const revalidate = 0

// Whitelist of updatable fields. Must match CementPurchaseSchema in src/lib/models.ts.
const FIELDS = [
  'date',
  'vendorName',
  'itemName',
  'quantity',
  'rate',
  'totalAmount',
  'paidAmount',
  'transportationCharge',
  'gst',
  'remarks',
] as const

// GET /api/cement-purchase/[id] — fetch a single cement-purchase entry
export async function GET(
  _request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const session = await requireSession()
    if (session instanceof NextResponse) return session

    await connectDB()
    const { id } = await params
    const record = await CementPurchase.findById(id).lean()
    if (!record) {
      return NextResponse.json({ error: 'cementPurchase entry not found' }, { status: 404 })
    }
    return NextResponse.json({ cementPurchase: toObject(record) })
  } catch (error) {
    console.error('Error fetching cement-purchase entry:', error)
    return NextResponse.json({ error: 'Failed to fetch cement-purchase entry' }, { status: 500 })
  }
}

// PUT /api/cement-purchase/[id] — update a single cement-purchase entry
export async function PUT(
  request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const session = await requireRole(['admin', 'operator', 'accountant'])
    if (session instanceof NextResponse) return session

    await connectDB()
    const { id } = await params
    const body = await request.json()
    // Normalize date to canonical YYYY-MM-DD
    // (handles dd-mm-yyyy, dd/mm/yyyy, Excel serials, Date objects, etc.)
    if (body.date) body.date = normalizeDate(body.date)

    const updateData: Record<string, unknown> = {}
    for (const field of FIELDS) {
      if (body[field] !== undefined) {
        updateData[field] = body[field]
      }
    }

    const record = await CementPurchase.findByIdAndUpdate(id, updateData, { new: true })
    if (!record) {
      return NextResponse.json({ error: 'cementPurchase entry not found' }, { status: 404 })
    }

    return NextResponse.json({ cementPurchase: toObject(record) })
  } catch (error) {
    console.error('Error updating cement-purchase entry:', error)
    return NextResponse.json({ error: 'Failed to update cement-purchase entry' }, { status: 500 })
  }
}

// DELETE /api/cement-purchase/[id] — delete a single cement-purchase entry
export async function DELETE(
```

```
  _request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const session = await requireRole(['admin', 'operator', 'accountant'])
    if (session instanceof NextResponse) return session

    await connectDB()
    const { id } = await params

    const record = await CementPurchase.findByIdAndDelete(id)
    if (!record) {
      return NextResponse.json({ error: 'cementPurchase entry not found' }, { status: 404 })
    }

    return NextResponse.json({ message: 'cementPurchase entry deleted successfully' })
  } catch (error) {
    console.error('Error deleting cement-purchase entry:', error)
    return NextResponse.json({ error: 'Failed to delete cement-purchase entry' }, { status: 500 })
  }
}
```

/api/cement-purchase/bulk-delete/route.ts 61 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB } from '@/lib/db'
import { CementPurchase } from '@/lib/models'
import { getSession } from '@/lib/auth'

export const dynamic = 'force-dynamic'

// POST /api/cement-purchase/bulk-delete
// Body: { ids: string[] }
// Gated behind admin/operator session — accountants are read-only.
export async function POST(request: Request) {
  try {
    await connectDB()

    const session = await getSession()
    if (!session) {
      return NextResponse.json(
        { error: 'Unauthorized — please log in' },
        { status: 401 }
      )
    }
    if (session.role === 'accountant') {
      return NextResponse.json(
        { error: 'Forbidden — accountants cannot delete cement purchase entries' },
        { status: 403 }
      )
    }

    const body = await request.json()
    const { ids } = body as { ids?: string[] }

    if (!Array.isArray(ids) || ids.length === 0) {
      return NextResponse.json(
        { error: 'ids array is required and must be non-empty' },
        { status: 400 }
      )
    }

    const result = await CementPurchase.deleteMany({ _id: { $in: ids } })

    if (result.deletedCount === 0) {
      return NextResponse.json(
        { error: 'No matching cement purchase entries found — they may have been already deleted' },
        { status: 404 }
      )
    }

    return NextResponse.json({
      message: `${result.deletedCount} cement purchase entr${result.deletedCount === 1 ? 'y' : 'ies'} deleted`,
      deletedCount: result.deletedCount,
      requestedCount: ids.length,
    })
  } catch (error) {
    console.error('Error bulk-deleting cement purchases:', error)
    return NextResponse.json(
      { error: 'Failed to bulk-delete cement purchase entries' },
      { status: 500 }
    )
  }
}
```

/api/cement-purchase/route.ts 57 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB, toObject } from '@/lib/db'
import { CementPurchase } from '@/lib/models'
import { requireSession, requireRole } from '@/lib/auth'
import { normalizeDate } from '@/lib/date-utils'

// Force dynamic — never cache list responses
export const dynamic = 'force-dynamic'
export const revalidate = 0

export async function GET() {
  try {
    const session = await requireSession()
    if (session instanceof NextResponse) return session

    await connectDB()
    const records = await CementPurchase.find({}).sort({ date: -1 }).lean()
    return NextResponse.json({ cementPurchases: records.map(toObject) })
  } catch (error) {
    console.error('Error fetching cement purchases:', error)
    return NextResponse.json({ error: 'Failed to fetch cement purchases' }, { status: 500 })
  }
}

export async function POST(request: Request) {
  try {
    const session = await requireRole(['admin', 'operator', 'accountant'])
    if (session instanceof NextResponse) return session

    await connectDB()
    const body = await request.json()
    // Normalize date to canonical YYYY-MM-DD
    // (handles dd-mm-yyyy, dd/mm/yyyy, Excel serials, Date objects, etc.)
    if (body.date) body.date = normalizeDate(body.date)
    if (!body.date || !body.vendorName || !body.quantity || !body.rate) {
      return NextResponse.json({ error: 'Date, vendor name, quantity and rate are required' }, { status: 400 })
    }
    const totalAmount = Number(body.quantity) * Number(body.rate)
    const record = await CementPurchase.create({
      date: body.date,
      vendorName: body.vendorName,
      itemName: body.itemName || '',
      quantity: Number(body.quantity),
      rate: Number(body.rate),
      totalAmount,
      paidAmount: Number(body.paidAmount) || 0,
      transportationCharge: Number(body.transportationCharge) || 0,
      gst: Number(body.gst) || 0,
      remarks: body.remarks || '',
    })
    return NextResponse.json({ cementPurchase: toObject(record) }, { status: 201 })
  } catch (error) {
    console.error('Error creating cement purchase:', error)
    return NextResponse.json({ error: 'Failed to create cement purchase' }, { status: 500 })
  }
}
```

/api/company/route.ts 160 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB, toObject } from '@/lib/db'
import { Company } from '@/lib/models'
import { requireSession, requireAdmin } from '@/lib/auth'

// ── Default contact info for Veda Enterprises ──────────────────────
//
// These are sensible defaults used in two scenarios:
//   1. Brand-new install with no Company record yet — we seed these so the
//      footer / invoices show real contact info from day one.
//   2. Existing install where the Company record was created by an older
//      version of the seed (with empty phone/address). On every GET we
//      backfill any of these fields that are still empty, so existing
//      deployments get the new contact info without needing a manual
//      migration script.
//
// If an admin has already filled in their own phone/address/email through
// Settings, those user-supplied values take precedence and we never
// overwrite them.
const VEDA_DEFAULTS = {
  name: 'Veda Enterprises',
  tagline: 'Paver Block ERP',
  address: 'Purushottampur, Muzaffarpur',
  city: 'Muzaffarpur',
  state: 'Bihar',
  pincode: '842002',
  phone: '9572831213',
  email: 'vedaenterprises@gmail.com',
}

// ── Legacy value migrations ────────────────────────────────────────
//
// When we rename something globally (e.g. "Paper Block ERP" → "Paver Block
// ERP"), existing MongoDB records still hold the OLD value because the
// backfill logic above only touches EMPTY fields. This map silently
// upgrades known stale strings to their current value on every GET, so
// every existing deployment gets the rename without a manual migration
// script.
//
// To add a new migration: { field: 'tagName', from: 'old value', to: 'new value' }
// Keep the `from` comparison case-insensitive and trimmed so we catch
// "Paper Block ERP", "paper block erp", " Paper Block ERP " etc.
const LEGACY_MIGRATIONS: Array<{ field: keyof typeof VEDA_DEFAULTS; from: string; to: string }> = [
  { field: 'tagline', from: 'Paper Block ERP', to: 'Paver Block ERP' },
]

export async function GET() {
  try {
    const session = await requireSession()
    if (session instanceof NextResponse) return session

    await connectDB()

    let company = await Company.findOne({})
    if (!company) {
      // Brand-new install — create with full defaults.
      company = await Company.create({
        ...VEDA_DEFAULTS,
        setupComplete: false,
      })
    } else {
      // Existing install — backfill any empty contact fields with the
      // Veda defaults. This is what makes the footer / invoices show the
      // right phone + address for deployments that were created before
      // these defaults existed.
      let needsUpdate = false
      const patch: Record<string, string> = {}
      for (const key of Object.keys(VEDA_DEFAULTS) as (keyof typeof VEDA_DEFAULTS)[]) {
        const current = (company as any)[key]
        if (!current || String(current).trim() === '') {
          patch[key] = VEDA_DEFAULTS[key]
          needsUpdate = true
        }
      }

      // Legacy value migration — if a field holds a known stale value
      // (e.g. tagline was "Paper Block ERP" before the rename to "Paver
      // Block ERP"), silently upgrade it. This runs AFTER the empty-field
      // backfill above so we don't accidentally re-set a value the admin
      // had intentionally cleared.
      for (const migration of LEGACY_MIGRATIONS) {
        const current = (company as any)[migration.field]
        if (typeof current === 'string' && current.trim().toLowerCase() === migration.from.toLowerCase()) {
```

```
          patch[migration.field] = migration.to
          needsUpdate = true
        }
      }

      if (needsUpdate) {
        company = await Company.findByIdAndUpdate(
          company._id,
          { $set: patch },
          { new: true }
        )
      }
    }

    return NextResponse.json({ company: toObject(company) })
  } catch (error) {
    console.error('Error fetching company:', error)
    return NextResponse.json(
      { error: 'Failed to fetch company settings' },
      { status: 500 }
    )
  }
}

export async function PUT(request: Request) {
  try {
    const auth = await requireAdmin()
    if (auth instanceof NextResponse) return auth

    await connectDB()
    const body = await request.json()

    let company = await Company.findOne({})
    if (!company) {
      company = await Company.create({
        ...VEDA_DEFAULTS,
        setupComplete: false,
      })
    }

    const fields = [
      'name', 'tagline', 'address', 'city', 'state', 'pincode',
      'phone', 'email', 'gstNumber', 'panNumber', 'logoUrl',
      'primaryColor', 'bankName', 'bankAccount', 'bankIfsc',
      'invoicePrefix', 'dispatchPrefix', 'orderPrefix', 'terms',
      'signatureName', 'setupComplete',
    ]

    const updateData: Record<string, unknown> = {}
    for (const field of fields) {
      if (body[field] !== undefined) {
        updateData[field] = body[field]
      }
    }

    // Auto-detect setup completion
    const name = (body.name !== undefined ? body.name : company.name) as string
    const address = (body.address !== undefined ? body.address : company.address) as string
    const phone = (body.phone !== undefined ? body.phone : company.phone) as string
    const gstNumber = (body.gstNumber !== undefined ? body.gstNumber : company.gstNumber) as string

    if (name && address && phone && gstNumber) {
      updateData.setupComplete = true
    }

    const updated = await Company.findByIdAndUpdate(company._id, updateData, { new: true })

    return NextResponse.json({ company: toObject(updated) })
  } catch (error) {
    console.error('Error updating company:', error)
    return NextResponse.json(
      { error: 'Failed to update company settings' },
      { status: 500 }
    )
  }
}
```

/api/customer-payment/[id]/route.ts 100 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB, toObject } from '@/lib/db'
import { CustomerPayment } from '@/lib/models'
import { requireSession, requireRole } from '@/lib/auth'
import { normalizeDate } from '@/lib/date-utils'

// Force dynamic — never cache individual responses
export const dynamic = 'force-dynamic'
export const revalidate = 0

// Whitelist of updatable fields. Must match CustomerPaymentSchema in src/lib/models.ts.
const FIELDS = [
  'date',
  'name',
  'address',
  'amount',
  'remarks',
] as const

// GET /api/customer-payment/[id] — fetch a single customer-payment entry
export async function GET(
  _request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const session = await requireSession()
    if (session instanceof NextResponse) return session

    await connectDB()
    const { id } = await params
    const record = await CustomerPayment.findById(id).lean()
    if (!record) {
      return NextResponse.json({ error: 'customerPayment entry not found' }, { status: 404 })
    }
    return NextResponse.json({ customerPayment: toObject(record) })
  } catch (error) {
    console.error('Error fetching customer-payment entry:', error)
    return NextResponse.json({ error: 'Failed to fetch customer-payment entry' }, { status: 500 })
  }
}

// PUT /api/customer-payment/[id] — update a single customer-payment entry
export async function PUT(
  request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const session = await requireRole(['admin', 'operator', 'accountant'])
    if (session instanceof NextResponse) return session

    await connectDB()
    const { id } = await params
    const body = await request.json()
    // Normalize date to canonical YYYY-MM-DD
    // (handles dd-mm-yyyy, dd/mm/yyyy, Excel serials, Date objects, etc.)
    if (body.date) body.date = normalizeDate(body.date)

    const updateData: Record<string, unknown> = {}
    for (const field of FIELDS) {
      if (body[field] !== undefined) {
        updateData[field] = body[field]
      }
    }

    const record = await CustomerPayment.findByIdAndUpdate(id, updateData, { new: true })
    if (!record) {
      return NextResponse.json({ error: 'customerPayment entry not found' }, { status: 404 })
    }

    return NextResponse.json({ customerPayment: toObject(record) })
  } catch (error) {
    console.error('Error updating customer-payment entry:', error)
    return NextResponse.json({ error: 'Failed to update customer-payment entry' }, { status: 500 })
  }
}

// DELETE /api/customer-payment/[id] — delete a single customer-payment entry
export async function DELETE(
  _request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const session = await requireRole(['admin', 'operator', 'accountant'])
```

```
    if (session instanceof NextResponse) return session

    await connectDB()
    const { id } = await params

    const record = await CustomerPayment.findByIdAndDelete(id)
    if (!record) {
      return NextResponse.json({ error: 'customerPayment entry not found' }, { status: 404 })
    }

    return NextResponse.json({ message: 'customerPayment entry deleted successfully' })
  } catch (error) {
    console.error('Error deleting customer-payment entry:', error)
    return NextResponse.json({ error: 'Failed to delete customer-payment entry' }, { status: 500 })
  }
}
```

/api/customer-payment/bulk-delete/route.ts 63 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB } from '@/lib/db'
import { CustomerPayment } from '@/lib/models'
import { getSession } from '@/lib/auth'

// Force dynamic — never cache
export const dynamic = 'force-dynamic'

// POST /api/customer-payment/bulk-delete
// Body: { ids: string[] }
// Deletes customer payment entries whose `_id` is in `ids`.
// Gated behind admin/operator session — accountants are read-only.
export async function POST(request: Request) {
  try {
    await connectDB()

    const session = await getSession()
    if (!session) {
      return NextResponse.json(
        { error: 'Unauthorized — please log in' },
        { status: 401 }
      )
    }
    if (session.role === 'accountant') {
      return NextResponse.json(
        { error: 'Forbidden — accountants cannot delete customer payment entries' },
        { status: 403 }
      )
    }

    const body = await request.json()
    const { ids } = body as { ids?: string[] }

    if (!Array.isArray(ids) || ids.length === 0) {
      return NextResponse.json(
        { error: 'ids array is required and must be non-empty' },
        { status: 400 }
      )
    }

    const result = await CustomerPayment.deleteMany({ _id: { $in: ids } })

    if (result.deletedCount === 0) {
      return NextResponse.json(
        { error: 'No matching customer payment entries found — they may have been already deleted' },
        { status: 404 }
      )
    }

    return NextResponse.json({
      message: `${result.deletedCount} customer payment entr${result.deletedCount === 1 ? 'y' : 'ies'} deleted`,
      deletedCount: result.deletedCount,
      requestedCount: ids.length,
    })
  } catch (error) {
    console.error('Error bulk-deleting customer payments:', error)
    return NextResponse.json(
      { error: 'Failed to bulk-delete customer payment entries' },
      { status: 500 }
    )
  }
}
```

/api/customer-payment/route.ts 51 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB, toObject } from '@/lib/db'
import { CustomerPayment } from '@/lib/models'
import { requireSession, requireRole } from '@/lib/auth'
import { normalizeDate } from '@/lib/date-utils'

// Force dynamic — never cache list responses
export const dynamic = 'force-dynamic'
export const revalidate = 0

export async function GET() {
  try {
    const session = await requireSession()
    if (session instanceof NextResponse) return session

    await connectDB()
    const records = await CustomerPayment.find({}).sort({ date: -1 }).lean()
    return NextResponse.json({ customerPayments: records.map(toObject) })
  } catch (error) {
    console.error('Error fetching customer payments:', error)
    return NextResponse.json({ error: 'Failed to fetch customer payments' }, { status: 500 })
  }
}

export async function POST(request: Request) {
  try {
    const session = await requireRole(['admin', 'operator', 'accountant'])
    if (session instanceof NextResponse) return session

    await connectDB()
    const body = await request.json()
    // Normalize date to canonical YYYY-MM-DD
    // (handles dd-mm-yyyy, dd/mm/yyyy, Excel serials, Date objects, etc.)
    if (body.date) body.date = normalizeDate(body.date)
    if (!body.date || !body.name || !body.amount) {
      return NextResponse.json({ error: 'Date, name and amount are required' }, { status: 400 })
    }
    const record = await CustomerPayment.create({
      date: body.date,
      name: body.name,
      address: body.address || '',
      amount: Number(body.amount),
      remarks: body.remarks || '',
    })
    return NextResponse.json({ customerPayment: toObject(record) }, { status: 201 })
  } catch (error) {
    console.error('Error creating customer payment:', error)
    return NextResponse.json({ error: 'Failed to create customer payment' }, { status: 500 })
  }
}
```

/api/customers/[id]/bill-history/route.ts 142 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB, toObject } from '@/lib/db'
import { Customer, Production, Dispatch, Bill, Order, Payment } from '@/lib/models'
import { requireSession } from '@/lib/auth'

// Force dynamic — history changes after every mutation.
export const dynamic = 'force-dynamic'
export const revalidate = 0

// Product field → human-readable label map. Used to flatten a Production
// doc into billable line items. Quantities of 0 are skipped downstream.
const PRODUCT_FIELDS: Array<{ key: string; label: string; hsn: string }> = [
  { key: 'cement',         label: 'Cement (bags)',         hsn: '2523' },
  { key: 'zigZagGrey80',   label: 'Zig Zag Grey 80mm',    hsn: '6810' },
  { key: 'zigZagRed80',    label: 'Zig Zag Red 80mm',     hsn: '6810' },
  { key: 'zigZagYellow80', label: 'Zig Zag Yellow 80mm',  hsn: '6810' },
  { key: 'zigZagGrey60',   label: 'Zig Zag Grey 60mm',    hsn: '6810' },
  { key: 'zigZagRed60',    label: 'Zig Zag Red 60mm',     hsn: '6810' },
  { key: 'zigZagYellow60', label: 'Zig Zag Yellow 60mm',  hsn: '6810' },
  { key: 'chequreTile',    label: 'Chequre Tile',         hsn: '6810' },
  { key: 'curveStone',     label: 'Curve Stone',          hsn: '6810' },
  { key: 'dumbleGrey80',   label: 'Dumble Grey 80mm',     hsn: '6810' },
  { key: 'dumbleRed80',    label: 'Dumble Red 80mm',      hsn: '6810' },
  { key: 'dumbleYellow80', label: 'Dumble Yellow 80mm',   hsn: '6810' },
]

// GET /api/customers/[id]/bill-history
// Returns everything needed to generate a bill for this customer:
//   - customer record (for auto-filling party details)
//   - production records (product-wise quantities per date)
//   - dispatches (delivered quantities)
//   - orders (with their items[] — user can one-click import order items
//     into the bill, so they don't have to retype line items)
//   - payments (customer's payment history — both manual payments and
//     auto-synced payments from previous bills. Useful for setting the
//     paidAmount field on a new bill.)
//   - previous bills (so user can see what's already been billed)
//   - aggregated product totals across all production records — gives the
//     user a one-click "Add all unbilled production to bill" UX
export async function GET(
  _request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const session = await requireSession()
    if (session instanceof NextResponse) return session

    await connectDB()
    const { id } = await params

    const customer = await Customer.findById(id).lean()
    if (!customer) {
      return NextResponse.json({ error: 'Customer not found' }, { status: 404 })
    }

    // Match production by customerId only. (customerName/address fields
    // have been removed from Production — only customerId links them now.)
    const [productions, dispatches, bills, orders, payments] = await Promise.all([
      Production.find({ customerId: id }).sort({ date: -1 }).lean(),
      Dispatch.find({ customerId: id }).sort({ date: -1 }).lean(),
      Bill.find({ customerId: id }).sort({ createdAt: -1 }).lean(),
      Order.find({ customerId: id }).sort({ createdAt: -1 }).lean(),
      Payment.find({ customerId: id }).sort({ date: -1 }).lean(),
    ])

    // Aggregate product totals across all production rows — gives a
    // quick "bill everything produced" summary.
    const productTotals: Record<string, number> = {}
    for (const p of productions) {
      for (const { key } of PRODUCT_FIELDS) {
        const qty = Number((p as any)[key]) || 0
        if (qty > 0) {
          productTotals[key] = (productTotals[key] || 0) + qty
        }
      }
    }

    // Same aggregation for dispatches (delivered quantity)
    const dispatchedTotals: Record<string, number> = {}
    // Dispatches store quantities under `quantity` + `brickType` (legacy
    // schema) — they're not product-wise like production. We just sum the
    // total dispatched quantity for display.
    const totalDispatchedQty = dispatches.reduce(
```

```
      (s, d) => s + (Number(d.quantity) || 0),
      0
    )

    // Sum previously-billed amounts so the user can see outstanding balance
    const totalPreviouslyBilled = bills.reduce(
      (s, b) => s + (Number(b.grandTotal) || 0),
      0
    )
    const totalPreviouslyPaid = bills.reduce(
      (s, b) => s + (Number(b.paidAmount) || 0),
      0
    )

    // Sum all customer payments (manual + auto-synced from bills) — gives
    // the user a quick view of how much the customer has already paid
    // across all channels.
    const totalPaymentsReceived = payments.reduce(
      (s, p) => s + (Number(p.amount) || 0),
      0
    )

    const res = NextResponse.json({
      customer: toObject(customer),
      productions: productions.map(toObject),
      dispatches: dispatches.map(toObject),
      bills: bills.map(toObject),
      orders: orders.map(toObject),
      payments: payments.map(toObject),
      productFields: PRODUCT_FIELDS,
      summary: {
        productionCount: productions.length,
        dispatchCount: dispatches.length,
        billCount: bills.length,
        orderCount: orders.length,
        paymentCount: payments.length,
        totalDispatchedQty,
        totalPreviouslyBilled,
        totalPreviouslyPaid,
        totalPaymentsReceived,
        outstanding: totalPreviouslyBilled - totalPreviouslyPaid,
        productTotals,
        dispatchedTotals,
      },
    })

    res.headers.set('Cache-Control', 'no-store, no-cache, must-revalidate, max-age=0')
    res.headers.set('Pragma', 'no-cache')
    res.headers.set('Expires', '0')
    return res
  } catch (error) {
    console.error('Error fetching customer bill history:', error)
    return NextResponse.json(
      { error: 'Failed to fetch customer bill history' },
      { status: 500 }
    )
  }
}
```

/api/customers/[id]/history/route.ts 150 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB, toObject } from '@/lib/db'
import {
  Customer, Order, Dispatch, Payment, CustomerPayment,
} from '@/lib/models'
import { requireSession } from '@/lib/auth'

// Force dynamic — customer history changes after every mutation.
export const dynamic = 'force-dynamic'
export const revalidate = 0

// Customer history now only contains: Orders, Dispatches, Payments
// (legacy Payment + CustomerPayment). Production and DailySell have
// been removed per user request — they belong to factory ops, not to
// a per-customer ledger.
export async function GET(
  _request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const session = await requireSession()
    if (session instanceof NextResponse) return session

    await connectDB()
    const { id } = await params

    const customer = await Customer.findById(id).lean()
    if (!customer) {
      return NextResponse.json({ error: 'Customer not found' }, { status: 404 })
    }

    const customerName = String(customer.name || '').trim()

    // Build parallel queries — match by customerId (legacy Orders/Dispatch/Payments)
    // OR by customerName (CustomerPayment stores name, not ID).
    const [orders, dispatches, payments, customerPayments] =
      await Promise.all([
        Order.find({ customerId: id }).sort({ createdAt: -1 }).lean(),
        Dispatch.find({ customerId: id }).sort({ date: -1 }).lean(),
        Payment.find({ customerId: id }).sort({ date: -1 }).lean(),
        CustomerPayment.find(customerName ? { name: customer.name } : { _id: null })
          .sort({ date: -1 }).lean(),
      ])

    // ── Aggregations ──────────────────────────────────────────
    const totalOrderedAmount = orders.reduce((s, o) => s + (Number(o.amount) || 0), 0)
    const totalOrderedQty = orders.reduce((s, o) => s + (Number(o.quantity) || 0), 0)
    const totalDispatchedQty = dispatches.reduce((s, d) => s + (Number(d.quantity) || 0), 0)
    const totalCustomerPayments = customerPayments.reduce((s, p) => s + (Number(p.amount) || 0), 0)
    const totalLegacyPayments = payments.reduce((s, p) => s + (Number(p.amount) || 0), 0)
    const totalPaid = totalCustomerPayments + totalLegacyPayments

    const balance = totalOrderedAmount - totalPaid

    // Timeline — merge order/dispatch/payment events into one sorted list
    type TimelineEvent = {
      id: string
      date: string
      type: 'order' | 'dispatch' | 'payment' | 'customer_payment'
      description: string
      amount: number
      qty?: number
      reference?: string
      remarks?: string
    }
    const timeline: TimelineEvent[] = []
    for (const o of orders) {
      timeline.push({
        id: String(o._id),
        date: String(o.deliveryDate || o.createdAt || ''),
        type: 'order',
        description: `Order — ${o.brickType || ''}`,
        amount: Number(o.amount) || 0,
        qty: Number(o.quantity) || 0,
        reference: o.orderNumber,
        remarks: o.status,
      })
    }
    for (const d of dispatches) {
      timeline.push({
        id: String(d._id),
        date: String(d.date || ''),
        type: 'dispatch',
```

```
        description: `Dispatch — ${d.brickType || ''} (Truck: ${d.truckNumber || '—'})`,
        amount: 0,
        qty: Number(d.quantity) || 0,
        reference: d.dispatchNumber,
        remarks: d.driverName,
      })
    }
    for (const p of payments) {
      timeline.push({
        id: String(p._id),
        date: String(p.date || ''),
        type: 'payment',
        description: `Payment — ${p.paymentType || ''}`,
        amount: Number(p.amount) || 0,
        reference: '',
        remarks: p.remarks,
      })
    }
    for (const p of customerPayments) {
      timeline.push({
        id: String(p._id),
        date: String(p.date || ''),
        type: 'customer_payment',
        description: `Customer Payment`,
        amount: Number(p.amount) || 0,
        reference: '',
        remarks: p.remarks,
      })
    }
    timeline.sort((a, b) => {
      const da = new Date(a.date || 0).getTime()
      const db = new Date(b.date || 0).getTime()
      return db - da
    })

    const res = NextResponse.json({
      customer: toObject(customer),
      summary: {
        totalOrderedAmount,
        totalOrderedQty,
        totalDispatchedQty,
        totalCustomerPayments,
        totalLegacyPayments,
        totalPaid,
        billableAmount: totalOrderedAmount,
        balance,
        orderCount: orders.length,
        dispatchCount: dispatches.length,
        paymentCount: payments.length + customerPayments.length,
      },
      orders: orders.map(toObject),
      dispatches: dispatches.map(toObject),
      payments: payments.map(toObject),
      customerPayments: customerPayments.map(toObject),
      timeline,
    })

    res.headers.set('Cache-Control', 'no-store, no-cache, must-revalidate, max-age=0')
    res.headers.set('Pragma', 'no-cache')
    res.headers.set('Expires', '0')
    return res
  } catch (error) {
    console.error('Error fetching customer history:', error)
    return NextResponse.json({ error: 'Failed to fetch customer history' }, { status: 500 })
  }
}
```

/api/customers/[id]/route.ts 68 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB, toObject } from '@/lib/db'
import { Customer } from '@/lib/models'
import { requireSession, requireRole } from '@/lib/auth'

export async function GET(request: Request, { params }: { params: Promise<{ id: string }> }) {
  try {
    const session = await requireSession()
    if (session instanceof NextResponse) return session

    await connectDB()
    const { id } = await params
    const customer = await Customer.findById(id)
    if (!customer) {
      return NextResponse.json({ error: 'Customer not found' }, { status: 404 })
    }
    return NextResponse.json({ customer: toObject(customer) })
  } catch (error) {
    console.error('Error fetching customer:', error)
    return NextResponse.json({ error: 'Failed to fetch customer' }, { status: 500 })
  }
}

export async function PUT(request: Request, { params }: { params: Promise<{ id: string }> }) {
  try {
    const session = await requireRole(['admin', 'operator'])
    if (session instanceof NextResponse) return session

    await connectDB()
    const { id } = await params
    const body = await request.json()

    const updateData: Record<string, unknown> = {}
    const fields = ['name', 'mobile', 'gstNumber', 'address', 'creditLimit']
    for (const field of fields) {
      if (body[field] !== undefined) updateData[field] = body[field]
    }

    const customer = await Customer.findByIdAndUpdate(id, updateData, { new: true })
    if (!customer) {
      return NextResponse.json({ error: 'Customer not found' }, { status: 404 })
    }

    return NextResponse.json({ customer: toObject(customer) })
  } catch (error) {
    console.error('Error updating customer:', error)
    return NextResponse.json({ error: 'Failed to update customer' }, { status: 500 })
  }
}

export async function DELETE(request: Request, { params }: { params: Promise<{ id: string }> }) {
  try {
    const session = await requireRole(['admin', 'operator'])
    if (session instanceof NextResponse) return session

    await connectDB()
    const { id } = await params
    const customer = await Customer.findByIdAndDelete(id)
    if (!customer) {
      return NextResponse.json({ error: 'Customer not found' }, { status: 404 })
    }
    return NextResponse.json({ message: 'Customer deleted successfully' })
  } catch (error) {
    console.error('Error deleting customer:', error)
    return NextResponse.json({ error: 'Failed to delete customer' }, { status: 500 })
  }
}
```

/api/customers/bulk-delete/route.ts 93 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB } from '@/lib/db'
import { Customer, Order, Payment, CustomerPayment, Production, Dispatch } from '@/lib/models'
import { getSession } from '@/lib/auth'

// Force dynamic — never cache
export const dynamic = 'force-dynamic'

// POST /api/customers/bulk-delete
// Body: { ids: string[] }
// Deletes the customers whose `_id` is in `ids`.
// Also nulls-out (or disassociates) references in linked collections so
// historical orders / payments don't carry dangling customerId pointers —
// we set customerId = null instead of hard-deleting the financial records
// because the user may still want the audit trail of past transactions.
// Gated behind admin/operator session — accountants cannot bulk-delete.
export async function POST(request: Request) {
  try {
    await connectDB()

    // Auth gate
    const session = await getSession()
    if (!session) {
      return NextResponse.json(
        { error: 'Unauthorized — please log in' },
        { status: 401 }
      )
    }
    if (session.role === 'accountant') {
      return NextResponse.json(
        { error: 'Forbidden — accountants cannot delete customers' },
        { status: 403 }
      )
    }

    const body = await request.json()
    const { ids } = body as { ids?: string[] }

    if (!Array.isArray(ids) || ids.length === 0) {
      return NextResponse.json(
        { error: 'ids array is required and must be non-empty' },
        { status: 400 }
      )
    }

    // Verify the customers exist (so we can give an accurate error if
    // they were already deleted by another tab/session).
    const existing = await Customer.find({ _id: { $in: ids } })
      .select('_id name')
      .lean()

    if (existing.length === 0) {
      return NextResponse.json(
        { error: 'No matching customers found — they may have been already deleted' },
        { status: 404 }
      )
    }

    const validIds = existing.map((c) => String(c._id))

    // Delete the customer documents
    const result = await Customer.deleteMany({ _id: { $in: validIds } })

    // Disassociate linked financial records — keep the audit trail but
    // remove the dangling customerId pointer so orders/payments don't
    // show a phantom customer name in their UIs.
    try {
      await Promise.all([
        Order.updateMany({ customerId: { $in: validIds } }, { $set: { customerId: null } }),
        Payment.updateMany({ customerId: { $in: validIds } }, { $set: { customerId: null } }),
        CustomerPayment.updateMany({ customerId: { $in: validIds } }, { $set: { customerId: null } }),
        Production.updateMany({ customerId: { $in: validIds } }, { $set: { customerId: null } }),
        Dispatch.updateMany({ customerId: { $in: validIds } }, { $set: { customerId: null } }),
      ])
    } catch (err) {
      console.error('[POST /customers/bulk-delete] Linked record cleanup failed:', err)
      // Don't fail the request — customers were already deleted.
    }

    return NextResponse.json({
      message: `${result.deletedCount} customer${result.deletedCount === 1 ? '' : 's'} deleted`,
      deletedCount: result.deletedCount,
      requestedCount: ids.length,
```

```
    })
  } catch (error) {
    console.error('Error bulk-deleting customers:', error)
    return NextResponse.json(
      { error: 'Failed to bulk-delete customers' },
      { status: 500 }
    )
  }
}
```

/api/customers/route.ts 89 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB, toObject } from '@/lib/db'
import { Customer } from '@/lib/models'
import { requireSession, requireRole } from '@/lib/auth'

// Force dynamic rendering — never cache customer list responses.
// This ensures that after an Excel import the GET /api/customers
// returns the freshly inserted rows instead of a stale snapshot.
export const dynamic = 'force-dynamic'
export const revalidate = 0

export async function GET(request: Request) {
  try {
    const session = await requireSession()
    if (session instanceof NextResponse) return session

    await connectDB()
    const { searchParams } = new URL(request.url)
    const search = searchParams.get('search')
    // Pagination params — default page 1, 100 per page (was previously
    // returning ALL customers which got slow at 500+ records)
    const page = Math.max(1, parseInt(searchParams.get('page') || '1', 10))
    const limit = Math.min(500, Math.max(1, parseInt(searchParams.get('limit') || '100', 10)))
    const skip = (page - 1) * limit

    const filter: any = {}
    if (search) {
      // Escape regex metacharacters to prevent ReDoS / wildcard surprises
      const safe = search.replace(/[.*+?^${}()|[\]\\]/g, '\\$&')
      filter.$or = [
        { name: { $regex: safe, $options: 'i' } },
        { mobile: { $regex: safe, $options: 'i' } },
      ]
    }

    // Run count + find in parallel — single round-trip instead of two
    const [customers, total] = await Promise.all([
      Customer.find(filter)
        .sort({ createdAt: -1 })
        .skip(skip)
        .limit(limit)
        .lean(),
      Customer.countDocuments(filter),
    ])

    const res = NextResponse.json({
      customers: customers.map(toObject),
      total,
      page,
      limit,
      totalPages: Math.ceil(total / limit),
    })
    res.headers.set('Cache-Control', 'no-store, no-cache, must-revalidate, max-age=0')
    res.headers.set('Pragma', 'no-cache')
    res.headers.set('Expires', '0')
    return res
  } catch (error) {
    console.error('Error fetching customers:', error)
    return NextResponse.json({ error: 'Failed to fetch customers' }, { status: 500 })
  }
}

export async function POST(request: Request) {
  try {
    const session = await requireRole(['admin', 'operator'])
    if (session instanceof NextResponse) return session

    await connectDB()
    const body = await request.json()

    if (!body.name || !body.mobile) {
      return NextResponse.json({ error: 'Name and mobile are required' }, { status: 400 })
    }

    const customer = await Customer.create({
      name: body.name,
      mobile: body.mobile,
      gstNumber: body.gstNumber || '',
      address: body.address || '',
      creditLimit: Number(body.creditLimit) || 0,
    })

    return NextResponse.json({ customer: toObject(customer) }, { status: 201 })
```

```
  } catch (error) {
    console.error('Error creating customer:', error)
    return NextResponse.json({ error: 'Failed to create customer' }, { status: 500 })
  }
}
```

/api/daily-sell/[id]/route.ts 239 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB, toObject } from '@/lib/db'
import { DailySell } from '@/lib/models'
import { syncAllFromDailySell, cleanupDailySellLinks } from '@/lib/daily-sell-sync'
import { requireSession, requireRole } from '@/lib/auth'
import { normalizeDate } from '@/lib/date-utils'

// Force dynamic — never cache individual daily-sell responses
export const dynamic = 'force-dynamic'
export const revalidate = 0

// Whitelist of updatable fields. Must match DailySellSchema in src/lib/models.ts.
// Note: customerId / orderId / customerPaymentId / syncNotes are managed by the
// auto-sync engine — NOT user-editable through this route.
// pendingAmount is auto-derived (amount − receivedAmount) on save.
const DAILY_SELL_FIELDS = [
  'date',
  'customerName',
  'address',
  'contactNumber',
  'product',
  'quantity',
  'rate',
  'amount',
  'transporterName',
  'transporterFair',
  'receivedAmount',
  'remarks',
] as const

const NUMERIC_FIELDS = new Set(['amount', 'quantity', 'rate', 'transporterFair', 'receivedAmount'])

// GET /api/daily-sell/[id] — fetch a single daily sell entry
export async function GET(
  _request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const session = await requireSession()
    if (session instanceof NextResponse) return session

    await connectDB()
    const { id } = await params
    const record = await DailySell.findById(id).lean()
    if (!record) {
      return NextResponse.json({ error: 'Daily sell entry not found' }, { status: 404 })
    }
    return NextResponse.json({ dailySell: toObject(record) })
  } catch (error) {
    console.error('Error fetching daily sell entry:', error)
    return NextResponse.json({ error: 'Failed to fetch daily sell entry' }, { status: 500 })
  }
}

// PUT /api/daily-sell/[id] — update a single daily sell entry
// After updating the user-facing fields, we re-run the auto-sync engine:
//   1. Delete the previously-linked Order + CustomerPayment (cleanup)
//   2. Re-create them with the new data
//   3. Update the linked Customer's address if it changed
//   4. Refresh syncNotes with the new summary
export async function PUT(
  request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const session = await requireRole(['admin', 'operator', 'accountant'])
    if (session instanceof NextResponse) return session

    await connectDB()
    const { id } = await params
    const body = await request.json()
    // Normalize date to canonical YYYY-MM-DD
    // (handles dd-mm-yyyy, dd/mm/yyyy, Excel serials, Date objects, etc.)
    if (body.date) body.date = normalizeDate(body.date)

    const updateData: Record<string, unknown> = {}
    for (const field of DAILY_SELL_FIELDS) {
      if (body[field] !== undefined) {
        if (NUMERIC_FIELDS.has(field)) {
          updateData[field] = Number(body[field])
        } else {
          updateData[field] = String(body[field])
        }
```

```
    }
  }

  // ── Multi-product support ──────────────────────────────────
  // If the body includes a `products` array, store it AND recompute the
  // legacy summary fields (product/quantity/rate/amount) from it. If the
  // body does NOT include `products`, the existing array is preserved
  // (so editing only the customer/date/transporter/received fields on a
  // multi-product record doesn't wipe out the line items).
  type ProductEntry = { product: string; quantity: number; rate: number; amount: number }
  if (Array.isArray(body.products)) {
    const products: ProductEntry[] = body.products
      .filter((p: any) => p && typeof p === 'object')
      .map((p: any) => ({
        product: String(p.product || '').trim(),
        quantity: Number(p.quantity) || 0,
        rate: Number(p.rate) || 0,
        amount: Number(p.amount) || (Number(p.quantity) || 0) * (Number(p.rate) || 0),
      }))
    if (products.length > 0) {
      // Recompute summary fields from the products array.
      updateData.products = products
      updateData.product =
        products.length === 1
          ? products[0].product
          : `${products[0].product}, +${products.length - 1} more`
      updateData.quantity = products.reduce((s, p) => s + (Number(p.quantity) || 0), 0)
      updateData.amount = products.reduce((s, p) => s + (Number(p.amount) || 0), 0)
      updateData.rate = 0
    } else {
      // Empty array — clear multi-product storage.
      updateData.products = []
    }
  }

  const record = await DailySell.findByIdAndUpdate(id, updateData, { new: true })
  if (!record) {
    return NextResponse.json({ error: 'Daily sell entry not found' }, { status: 404 })
  }

  // ── Auto-compute pendingAmount = amount − receivedAmount ──────────
  const amt = Number(record.amount) || 0
  const rec = Number(record.receivedAmount) || 0
  record.pendingAmount = Math.max(0, amt - rec)

  // ── Re-run auto-sync with the new field values ────────────────────
  // Step 1: clean up the previously-linked Order + CustomerPayment +
  // Payment + TractorPayment so we don't leave stale mirrors when the
  // user edits the entry.
  try {
    await cleanupDailySellLinks({
      customerId: record.customerId?.toString(),
      orderId: record.orderId?.toString(),
      customerPaymentId: record.customerPaymentId?.toString(),
      paymentId: (record as any).paymentId?.toString(),
      tractorPaymentId: (record as any).tractorPaymentId?.toString(),
    })
  } catch (cleanupErr) {
    console.error('[daily-sell PUT] Cleanup error (non-blocking):', cleanupErr)
  }

  // Step 2: re-create the mirrors with the updated field values.
  // If the record has a `products` array, pass it through so syncOrder
  // creates the Order with one line item per product.
  try {
    const recordProducts = Array.isArray((record as any).products)
      ? (record as any).products.map((p: any) => ({
          product: String(p.product || ''),
          quantity: Number(p.quantity) || 0,
          rate: Number(p.rate) || 0,
          amount: Number(p.amount) || 0,
        }))
      : undefined
    const sync = await syncAllFromDailySell({
      dailySellId: String(record._id),
      date: String(record.date),
      customerName: String(record.customerName),
      address: String(record.address || ''),
      contactNumber: String(record.contactNumber || ''),
      product: String(record.product || ''),
      quantity: Number(record.quantity) || 0,
      rate: Number(record.rate) || 0,
      amount: amt,
      transporterName: String(record.transporterName || ''),
      transporterFair: Number(record.transporterFair) || 0,
      receivedAmount: rec,
```

```
        pendingAmount: Number(record.pendingAmount) || 0,
        remarks: String(record.remarks || ''),
        products: recordProducts && recordProducts.length > 0 ? recordProducts : undefined,
      })
      record.customerId = sync.customerId as any
      record.orderId = sync.orderId as any
      record.customerPaymentId = sync.customerPaymentId as any
      ;(record as any).paymentId = sync.paymentId as any
      ;(record as any).tractorPaymentId = sync.tractorPaymentId as any
      record.syncNotes = sync.syncNotes
      await record.save()
    } catch (syncErr) {
      console.error('[daily-sell PUT] Auto-sync failed (non-blocking):', syncErr)
      record.customerId = null
      record.orderId = null
      record.customerPaymentId = null
      ;(record as any).paymentId = null
      ;(record as any).tractorPaymentId = null
      record.syncNotes = 'Auto-sync failed on edit — entry saved but mirrors may be stale'
      await record.save()
    }

    return NextResponse.json({ dailySell: toObject(record) })
  } catch (error) {
    console.error('Error updating daily sell entry:', error)
    return NextResponse.json({ error: 'Failed to update daily sell entry' }, { status: 500 })
  }
}

// DELETE /api/daily-sell/[id] — delete a single daily sell entry
// Before deleting, clean up the linked Order + CustomerPayment mirrors.
// The Customer record is preserved (may have other transactions).
export async function DELETE(
  _request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const session = await requireRole(['admin', 'operator', 'accountant'])
    if (session instanceof NextResponse) return session

    await connectDB()
    const { id } = await params

    const record = await DailySell.findById(id).lean()
    if (!record) {
      return NextResponse.json({ error: 'Daily sell entry not found' }, { status: 404 })
    }

    // Clean up linked mirrors (best-effort)
    try {
      await cleanupDailySellLinks({
        customerId: (record as any).customerId?.toString(),
        orderId: (record as any).orderId?.toString(),
        customerPaymentId: (record as any).customerPaymentId?.toString(),
        paymentId: (record as any).paymentId?.toString(),
        tractorPaymentId: (record as any).tractorPaymentId?.toString(),
      })
    } catch (cleanupErr) {
      console.error('[daily-sell DELETE] Cleanup error (non-blocking):', cleanupErr)
    }

    await DailySell.findByIdAndDelete(id)

    return NextResponse.json({ message: 'Daily sell entry deleted successfully' })
  } catch (error) {
    console.error('Error deleting daily sell entry:', error)
    return NextResponse.json({ error: 'Failed to delete daily sell entry' }, { status: 500 })
  }
}
```

/api/daily-sell/route.ts 242 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB, toObject } from '@/lib/db'
import { DailySell } from '@/lib/models'
import { requireRole } from '@/lib/auth'
import { syncAllFromDailySell } from '@/lib/daily-sell-sync'
import { normalizeDate } from '@/lib/date-utils'

// Force dynamic — never cache list responses
export const dynamic = 'force-dynamic'
export const revalidate = 0

export async function GET() {
  try {
    const session = await requireRole(['admin', 'operator', 'accountant'])
    if (session instanceof NextResponse) return session

    await connectDB()
    const records = await DailySell.find({}).sort({ date: -1 }).lean()
    return NextResponse.json({ dailySells: records.map(toObject) })
  } catch (error) {
    console.error('Error fetching daily sells:', error)
    return NextResponse.json({ error: 'Failed to fetch daily sells' }, { status: 500 })
  }
}

export async function POST(request: Request) {
  try {
    const session = await requireRole(['admin', 'operator', 'accountant'])
    if (session instanceof NextResponse) return session

    await connectDB()
    const body = await request.json()
    // Normalize date to canonical YYYY-MM-DD
    // (handles dd-mm-yyyy, dd/mm/yyyy, Excel serials, Date objects, etc.)
    if (body.date) body.date = normalizeDate(body.date)

    // ── Bulk delete: POST /api/daily-sell with { ids: [...] } ──────────
    // Admin/operator only — destructive bulk op.
    if (body && Array.isArray(body.ids)) {
      // Only admins can bulk-delete via this branch — operators/accountants must use the dedicated bulk-delete route
      if (session && typeof session === 'object' && 'role' in session && session.role !== 'admin') {
        return NextResponse.json(
          { error: 'Forbidden — only admins can bulk-delete via POST' },
          { status: 403 }
        )
      }
      const ids = body.ids.filter((id: unknown) => typeof id === 'string' && id.length > 0)
      if (ids.length === 0) {
        return NextResponse.json({ error: 'No ids provided' }, { status: 400 })
      }
      // Before deleting, fetch the records so we can clean up their linked
      // Order + CustomerPayment mirrors. Best-effort — cleanup failures
      // don't block the parent delete.
      try {
        const records = await DailySell.find({ _id: { $in: ids } }).lean()
        const { cleanupDailySellLinks } = await import('@/lib/daily-sell-sync')
        await Promise.all(
          records.map((r: any) =>
            cleanupDailySellLinks({
              customerId: r.customerId?.toString(),
              orderId: r.orderId?.toString(),
              customerPaymentId: r.customerPaymentId?.toString(),
              paymentId: r.paymentId?.toString(),
              tractorPaymentId: r.tractorPaymentId?.toString(),
            }).catch(() => {})
          )
        )
      } catch (cleanupErr) {
        console.error('[daily-sell bulk-delete] Cleanup error (non-blocking):', cleanupErr)
      }
      const result = await DailySell.deleteMany({ _id: { $in: ids } })
      return NextResponse.json({
        message: 'Daily sell entries deleted successfully',
        deletedCount: result.deletedCount || 0,
      })
    }

    if (!body.date || !body.customerName || body.amount == null) {
      return NextResponse.json({ error: 'Date, customer name and amount are required' }, { status: 400 })
    }

    // ── Multi-product support ──────────────────────────────────────────
    // If the request includes a `products` array (>=1 item), we store it
```

```
// AND use it to derive the legacy single-product fields as a SUMMARY:
//   • product  = first product's name (with ", +N more" if multiple)
//   • quantity = sum of all line-item quantities
//   • amount   = sum of all line-item amounts
//   • rate     = 0 (varies per item — meaningless as a single value)
// If no `products` array is provided, fall back to the legacy single-
// product path (use body.product/quantity/rate/amount as-is).
type ProductEntry = { product: string; quantity: number; rate: number; amount: number }
let products: ProductEntry[] = []
if (Array.isArray(body.products) && body.products.length > 0) {
  products = body.products
    .filter((p: any) => p && typeof p === 'object')
    .map((p: any) => ({
      product: String(p.product || '').trim(),
      quantity: Number(p.quantity) || 0,
      rate: Number(p.rate) || 0,
      amount: Number(p.amount) || (Number(p.quantity) || 0) * (Number(p.rate) || 0),
    }))
}

const hasMulti = products.length > 0

// Derive the legacy summary fields.
let legacyProduct: string
let legacyQuantity: number
let legacyRate: number
let legacyAmount: number

if (hasMulti) {
  legacyProduct =
    products.length === 1
      ? products[0].product
      : `${products[0].product}, +${products.length - 1} more`
  legacyQuantity = products.reduce((s, p) => s + (Number(p.quantity) || 0), 0)
  legacyAmount = products.reduce((s, p) => s + (Number(p.amount) || 0), 0)
  legacyRate = 0
} else {
  legacyProduct = String(body.product || '')
  legacyQuantity = Number(body.quantity) || 0
  legacyRate = Number(body.rate) || 0
  legacyAmount = Number(body.amount)
}

const received = Number(body.receivedAmount) || 0
const pending = Math.max(0, legacyAmount - received)

const record = await DailySell.create({
  date: body.date,
  customerName: body.customerName,
  address: body.address || '',
  contactNumber: body.contactNumber || '',
  product: legacyProduct,
  quantity: legacyQuantity,
  rate: legacyRate,
  amount: legacyAmount,
  transporterName: body.transporterName || '',
  transporterFair: Number(body.transporterFair) || 0,
  receivedAmount: received,
  pendingAmount: pending,
  products: hasMulti ? products : [],
  remarks: body.remarks || '',
})

// ── AUTO-SYNC to Customer, Order, Customer Payment, Payment, Tractor Payment, Stock ──
// Best-effort: if any sub-sync fails, the DailySell record still
// exists; the failure is recorded in `syncNotes` so the UI can
// surface it to the user.
try {
  const sync = await syncAllFromDailySell({
    dailySellId: String(record._id),
    date: body.date,
    customerName: body.customerName,
    address: body.address || '',
    contactNumber: body.contactNumber || '',
    product: legacyProduct,
    quantity: legacyQuantity,
    rate: legacyRate,
    amount: legacyAmount,
    transporterName: body.transporterName || '',
    transporterFair: Number(body.transporterFair) || 0,
    receivedAmount: received,
    pendingAmount: pending,
    remarks: body.remarks || '',
    products: hasMulti ? products : undefined,
  })
  record.customerId = sync.customerId as any
```

```
      record.orderId = sync.orderId as any
      record.customerPaymentId = sync.customerPaymentId as any
      record.paymentId = sync.paymentId as any
      record.tractorPaymentId = sync.tractorPaymentId as any
      record.syncNotes = sync.syncNotes
      await record.save()
    } catch (syncErr) {
      console.error('[daily-sell POST] Auto-sync failed (non-blocking):', syncErr)
      record.syncNotes = 'Auto-sync failed — entry saved but not mirrored to other modules'
      await record.save()
    }

    return NextResponse.json({ dailySell: toObject(record) }, { status: 201 })
  } catch (error) {
    console.error('Error creating daily sell:', error)
    return NextResponse.json({ error: 'Failed to create daily sell' }, { status: 500 })
  }
}

// DELETE /api/daily-sell?all=true — delete every daily sell entry
// (Delete All button). Mirrors the production delete-all API so the same
// client-side pattern works. Gated behind admin session — only admins can
// perform bulk destructive operations.
export async function DELETE(request: Request) {
  try {
    const session = await requireRole(['admin'])
    if (session instanceof NextResponse) return session

    await connectDB()

    const { searchParams } = new URL(request.url)
    const all = searchParams.get('all')

    if (all === 'true' || all === '1') {
      // Before deleting everything, clean up linked Order + CustomerPayment
      // mirrors for each record. Best-effort — cleanup failures don't block
      // the parent delete.
      try {
        const records = await DailySell.find({}).lean()
        const { cleanupDailySellLinks } = await import('@/lib/daily-sell-sync')
        await Promise.all(
          records.map((r: any) =>
            cleanupDailySellLinks({
              customerId: r.customerId?.toString(),
              orderId: r.orderId?.toString(),
              customerPaymentId: r.customerPaymentId?.toString(),
              paymentId: r.paymentId?.toString(),
              tractorPaymentId: r.tractorPaymentId?.toString(),
            }).catch(() => {})
          )
        )
      } catch (cleanupErr) {
        console.error('[daily-sell delete-all] Cleanup error (non-blocking):', cleanupErr)
      }
      const result = await DailySell.deleteMany({})
      return NextResponse.json({
        message: 'All daily sell entries deleted successfully',
        deletedCount: result.deletedCount || 0,
      })
    }

    // Without ?all=true this route is not used for single deletes —
    // those go through /api/daily-sell/[id]. Return a clear error.
    return NextResponse.json(
      { error: 'Use DELETE /api/daily-sell/[id] for single deletes, or ?all=true to delete every entry.' },
      { status: 400 }
    )
  } catch (error) {
    console.error('Error deleting daily sell entries:', error)
    return NextResponse.json({ error: 'Failed to delete daily sell entries' }, { status: 500 })
  }
}
```

/api/dashboard/route.ts 135 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB, toObject, extractCustomer } from '@/lib/db'
import { Production, Stock, Dispatch, Order, Payment, Expense, Customer } from '@/lib/models'
import { requireSession } from '@/lib/auth'

export async function GET() {
  try {
    const session = await requireSession()
    if (session instanceof NextResponse) return session

    await connectDB()

    const today = new Date().toISOString().split('T')[0]
    const currentMonth = today.substring(0, 7)
    // Pre-compute the date range for "this month" queries. Using $gte/$lt
    // lets Mongo use the `date` index efficiently — much faster than the
    // old `{ date: { $regex: '^YYYY-MM' } }` which scanned every doc.
    const monthStart = `${currentMonth}-01`
    const monthEnd = `${currentMonth}-31` // inclusive upper bound (YYYY-MM-31)

    // ── Run all independent queries in parallel ──────────────────────
    // Previously these ran as 12 sequential awaits — each adding 1 RTT of
    // latency. Batching them cuts total latency from ~12×RTT to ~1×RTT.
    const [
      todayProductions,
      stocks,
      todayDispatches,
      pendingOrders,
      // Outstanding payments — replaced two full-collection scans with two
      // $group aggregations that return one row per customer.
      ordersAgg,
      paymentsAgg,
      monthlyDispatches,
      monthlyExpenses,
      monthlyProductions,
      recentProductions,
      recentDispatchDocs,
    ] = await Promise.all([
      Production.find({ date: today }),
      Stock.find({}),
      Dispatch.find({ date: today }),
      Order.countDocuments({ status: 'Pending' }),
      Order.aggregate([
        { $group: { _id: '$customerId', total: { $sum: '$amount' } } },
      ]),
      Payment.aggregate([
        { $group: { _id: '$customerId', total: { $sum: '$amount' } } },
      ]),
      Dispatch.find({ date: { $gte: monthStart, $lte: monthEnd } }),
      Expense.find({ date: { $gte: monthStart, $lte: monthEnd } }),
      Production.find({ date: { $gte: monthStart, $lte: monthEnd } }).sort({ date: 1 }),
      Production.find({}).sort({ createdAt: -1 }).limit(5),
      Dispatch.find({}).sort({ createdAt: -1 }).limit(5).populate('customerId'),
    ])

    // ── Reduce: today's production, total stock, today's dispatch ────
    const todayProduction = todayProductions.reduce((sum, p) => sum + p.quantityProduced, 0)
    const totalStock = stocks.reduce((sum, s) => sum + s.currentStock, 0)
    const todayDispatch = todayDispatches.reduce((sum, d) => sum + d.quantity, 0)

    // ── Outstanding payments — built from the two $group results ─────
    const paymentsByCustomer = new Map<string, number>()
    for (const row of paymentsAgg) {
      paymentsByCustomer.set(String(row._id), row.total)
    }
    let outstandingPayments = 0
    for (const row of ordersAgg) {
      const paidAmount = paymentsByCustomer.get(String(row._id)) || 0
      const outstanding = row.total - paidAmount
      if (outstanding > 0) outstandingPayments += outstanding
    }

    // ── Monthly sales — need rates from referenced orders ────────────
    const orderIds = monthlyDispatches.map((d) => d.orderId).filter(Boolean)
    const ordersForRates = orderIds.length > 0 ? await Order.find({ _id: { $in: orderIds } }) : []
    const orderRateMap = new Map(ordersForRates.map((o) => [o._id.toString(), o.rate]))
    let monthlySales = 0
    for (const dispatch of monthlyDispatches) {
      if (dispatch.orderId) {
        const rate = orderRateMap.get(dispatch.orderId.toString()) || 0
        monthlySales += dispatch.quantity * rate
      }
    }
```

```
    // ── Monthly expenses / profit ──────────────────────────────
    const totalMonthlyExpenses = monthlyExpenses.reduce((sum, e) => sum + e.amount, 0)
    const monthlyProfit = monthlySales - totalMonthlyExpenses

    // ── Recent items for dashboard lists ──────────────────────
    const recentProductionsObj = recentProductions.map(toObject)
    const recentDispatches = recentDispatchDocs.map((d: any) => {
      const obj = toObject(d)
      const { customer, customerId } = extractCustomer(d)
      obj.customer = customer
      obj.customerId = customerId
      obj.customerName = customer?.name || ''
      return obj
    })

    // ── Monthly production chart data — single pass ───────────
    const monthlyProductionDataMap: Record<string, number> = {}
    for (const p of monthlyProductions) {
      monthlyProductionDataMap[p.date] = (monthlyProductionDataMap[p.date] || 0) + p.quantityProduced
    }
    const monthlyProductionChartData = Object.entries(monthlyProductionDataMap).map(([date, quantity]) => ({ date, quantity }))

    // ── Monthly expense chart data — single pass over already-fetched expenses ──
    const monthlyExpenseDataMap: Record<string, number> = {}
    for (const e of monthlyExpenses) {
      monthlyExpenseDataMap[e.category] = (monthlyExpenseDataMap[e.category] || 0) + e.amount
    }
    const monthlyExpenseChartData = Object.entries(monthlyExpenseDataMap).map(([category, amount]) => ({ category, amount }))

    return NextResponse.json({
      todayProduction,
      totalStock,
      todayDispatch,
      pendingOrders,
      outstandingPayments,
      monthlySales,
      monthlyProfit,
      recentProductions: recentProductionsObj,
      recentDispatches,
      monthlyProductionData: monthlyProductionChartData,
      monthlyExpenseData: monthlyExpenseChartData,
    })
  } catch (error) {
    console.error('Error fetching dashboard stats:', error)
    return NextResponse.json(
      { error: 'Failed to fetch dashboard stats' },
      { status: 500 }
    )
  }
}
```

/api/dashboard/stats/route.ts 192 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB } from '@/lib/db'
import {
  Production, Stock, DailySell, LabourPayment, CustomerPayment,
  TractorPayment, DustPurchase, CementPurchase, Hardner,
  Electricity, FactoryStuff,
} from '@/lib/models'
import { requireSession } from '@/lib/auth'

// Force dynamic — stats change after every mutation.
export const dynamic = 'force-dynamic'
export const revalidate = 0

// Single endpoint that returns ALL dashboard KPIs in one round-trip.
// Replaces the old approach where the dashboard made 11 separate API
// calls (each fetching ALL records) and then filtered client-side.
// Now we use MongoDB aggregation pipelines + indexes so each query
// only returns the numbers we actually need.
export async function GET() {
  try {
    const session = await requireSession()
    if (session instanceof NextResponse) return session

    await connectDB()

    const today = new Date().toISOString().split('T')[0]

    // ── Run all aggregations in parallel ──────────────────────
    // Each aggregation uses $match on the indexed `date` field, then
    // $group to compute the sum server-side. This means MongoDB only
    // streams a single number back over the wire, not entire documents.
    const [
      todayProductionAgg,
      latestStockAgg,
      todayDailySellAgg,
      todayLabourAgg,
      todayCustomerPaymentAgg,
      tractorRemainingAgg,
      todayDustAgg,
      todayCementAgg,
      todayHardnerAgg,
      todayElectricityAgg,
      todayFactoryStuffAgg,
    ] = await Promise.all([
      // Today's production — sum of all product piece columns
      Production.aggregate([
        { $match: { date: today } },
        { $group: {
          _id: null,
          pieces: {
            $sum: {
              $add: [
                { $ifNull: ['$zigZagGrey80', 0] },
                { $ifNull: ['$zigZagRed80', 0] },
                { $ifNull: ['$zigZagYellow80', 0] },
                { $ifNull: ['$zigZagGrey60', 0] },
                { $ifNull: ['$zigZagRed60', 0] },
                { $ifNull: ['$zigZagYellow60', 0] },
                { $ifNull: ['$curveStone', 0] },
                { $ifNull: ['$chequreTile', 0] },
                { $ifNull: ['$dumbleGrey80', 0] },
                { $ifNull: ['$dumbleRed80', 0] },
                { $ifNull: ['$dumbleYellow80', 0] },
              ],
            },
          },
        }},
      ]),

      // Latest stock entry — get the most recent stock row
      Stock.find({}).sort({ date: -1 }).limit(1).lean(),

      // Today's daily sell — sum of amount
      DailySell.aggregate([
        { $match: { date: today } },
        { $group: { _id: null, total: { $sum: { $ifNull: ['$amount', 0] } } } },
      ]),

      // Today's labour payments — sum of amount
      LabourPayment.aggregate([
        { $match: { date: today } },
        { $group: { _id: null, total: { $sum: { $ifNull: ['$amount', 0] } } } },
      ]),
```

```
    // Today's customer payments — sum of amount
    CustomerPayment.aggregate([
      { $match: { date: today } },
      { $group: { _id: null, total: { $sum: { $ifNull: ['$amount', 0] } } } },
    ]),

    // Tractor remaining — sum across ALL tractor payments (not just today)
    TractorPayment.aggregate([
      { $group: { _id: null, total: { $sum: { $ifNull: ['$remainingAmount', 0] } } } },
    ]),

    // Today's dust purchase — sum of totalAmount
    DustPurchase.aggregate([
      { $match: { date: today } },
      { $group: { _id: null, total: { $sum: { $ifNull: ['$totalAmount', 0] } } } },
    ]),

    // Today's cement purchase — sum of totalAmount
    CementPurchase.aggregate([
      { $match: { date: today } },
      { $group: { _id: null, total: { $sum: { $ifNull: ['$totalAmount', 0] } } } },
    ]),

    // Today's hardner — sum of amount
    Hardner.aggregate([
      { $match: { date: today } },
      { $group: { _id: null, total: { $sum: { $ifNull: ['$amount', 0] } } } },
    ]),

    // Today's electricity — sum of amount
    Electricity.aggregate([
      { $match: { date: today } },
      { $group: { _id: null, total: { $sum: { $ifNull: ['$amount', 0] } } } },
    ]),

    // Today's factory stuff — sum of amount
    FactoryStuff.aggregate([
      { $match: { date: today } },
      { $group: { _id: null, total: { $sum: { $ifNull: ['$amount', 0] } } } },
    ]),
  ])

  // Extract scalar values from aggregation results
  const pick = (agg: { _id: null; total?: number; pieces?: number }[], key: 'total' | 'pieces') =>
    agg && agg.length > 0 ? Number(agg[0][key] || 0) : 0

  const todayProduction = pick(todayProductionAgg as any, 'pieces')
  const todaySales = pick(todayDailySellAgg as any, 'total')
  const todayLabourPayments = pick(todayLabourAgg as any, 'total')
  const todayCustomerPayments = pick(todayCustomerPaymentAgg as any, 'total')
  const totalTractorRemaining = pick(tractorRemainingAgg as any, 'total')
  const todayDustPurchase = pick(todayDustAgg as any, 'total')
  const todayCementPurchase = pick(todayCementAgg as any, 'total')
  const todayHardner = pick(todayHardnerAgg as any, 'total')
  const todayElectricity = pick(todayElectricityAgg as any, 'total')
  const todayFactoryStuff = pick(todayFactoryStuffAgg as any, 'total')

  // Latest stock totals
  const latest = (latestStockAgg && latestStockAgg.length > 0) ? latestStockAgg[0] : null
  const totalStockPieces = latest ? (
    Number(latest.zigZagGrey80 || 0) +
    Number(latest.zigZagRed80 || 0) +
    Number(latest.zigZagYellow80 || 0) +
    Number(latest.zigZagGrey60 || 0) +
    Number(latest.zigZagRed60 || 0) +
    Number(latest.zigZagYellow60 || 0) +
    Number(latest.chequreTile || 0) +
    Number(latest.curveStone || 0) +
    Number(latest.dumbleGrey80 || 0) +
    Number(latest.dumbleRed80 || 0) +
    Number(latest.dumbleYellow80 || 0)
  ) : 0
  const totalStockCement = latest ? Number(latest.cement || 0) : 0

  const totalExpensesToday =
    todayLabourPayments + todayDustPurchase + todayCementPurchase +
    todayHardner + todayElectricity + todayFactoryStuff

  const res = NextResponse.json({
    todayProduction,
    todaySales,
    todayLabourPayments,
    todayCustomerPayments,
    totalTractorRemaining,
    todayDustPurchase,
```

```
      todayCementPurchase,
      todayHardner,
      todayElectricity,
      todayFactoryStuff,
      totalStock: totalStockPieces,
      totalStockCement,
      totalExpensesToday,
      netCashFlow: todaySales + todayCustomerPayments - totalExpensesToday,
    })

    res.headers.set('Cache-Control', 'no-store, no-cache, must-revalidate, max-age=0')
    res.headers.set('Pragma', 'no-cache')
    res.headers.set('Expires', '0')
    return res
  } catch (error) {
    console.error('Error fetching dashboard stats:', error)
    return NextResponse.json(
      { error: 'Failed to fetch dashboard stats' },
      { status: 500 }
    )
  }
}
```

/api/database/clear-section/route.ts 147 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB } from '@/lib/db'
import { getSession } from '@/lib/auth'
import {
  Company,
  User,
  Customer,
  Production,
  Stock,
  DailySell,
  CustomerPayment,
  LabourPayment,
  TractorPayment,
  DustPurchase,
  CementPurchase,
  Hardner,
  Electricity,
  FactoryStuff,
  Order,
  Dispatch,
  Payment,
  Expense,
  Bill,
} from '@/lib/models'

export const dynamic = 'force-dynamic'

type ModelLike = {
  countDocuments: (filter?: any) => Promise<number>
  deleteMany: (filter?: any) => Promise<{ deletedCount: number }>
}

// Map of clearable collections — `companies` and `users` are intentionally
// excluded so the admin can never wipe their own login or company profile
// via this endpoint (use the dedicated Company / Users tabs for those).
const CLEARABLE: { key: string; label: string; model: ModelLike }[] = [
  { key: 'customers',        label: 'Customers',           model: Customer as unknown as ModelLike },
  { key: 'productions',      label: 'Production Records',  model: Production as unknown as ModelLike },
  { key: 'stocks',           label: 'Stock Entries',       model: Stock as unknown as ModelLike },
  { key: 'dailySells',       label: 'Daily Sales',         model: DailySell as unknown as ModelLike },
  { key: 'customerPayments', label: 'Customer Payments',   model: CustomerPayment as unknown as ModelLike },
  { key: 'labourPayments',   label: 'Labour Payments',     model: LabourPayment as unknown as ModelLike },
  { key: 'tractorPayments',  label: 'Tractor Payments',    model: TractorPayment as unknown as ModelLike },
  { key: 'dustPurchases',    label: 'Dust Purchases',      model: DustPurchase as unknown as ModelLike },
  { key: 'cementPurchases',  label: 'Cement Purchases',    model: CementPurchase as unknown as ModelLike },
  { key: 'hardners',         label: 'Hardner Records',     model: Hardner as unknown as ModelLike },
  { key: 'electricities',    label: 'Electricity Records', model: Electricity as unknown as ModelLike },
  { key: 'factoryStuffs',    label: 'Factory Stuff',       model: FactoryStuff as unknown as ModelLike },
  { key: 'orders',           label: 'Orders',              model: Order as unknown as ModelLike },
  { key: 'dispatches',       label: 'Dispatches',          model: Dispatch as unknown as ModelLike },
  { key: 'payments',         label: 'Payments',            model: Payment as unknown as ModelLike },
  { key: 'expenses',         label: 'Expenses',            model: Expense as unknown as ModelLike },
  { key: 'bills',            label: 'Bills',               model: Bill as unknown as ModelLike },
]

// GET /api/database/clear-section
// Returns the list of clearable collections + current count for each — used
// to populate the dropdown in the Database tab UI.
export async function GET() {
  try {
    await connectDB()

    const session = await getSession()
    if (!session) {
      return NextResponse.json({ error: 'Unauthorized' }, { status: 401 })
    }
    if (session.role !== 'admin') {
      return NextResponse.json(
        { error: 'Forbidden — only admins can list clearable sections' },
        { status: 403 }
      )
    }

    const counts = await Promise.all(
      CLEARABLE.map(({ model }) => model.countDocuments({}))
    )

    const sections = CLEARABLE.map((c, i) => ({
      key: c.key,
      label: c.label,
      count: counts[i],
    }))
```

```
    return NextResponse.json({ sections })
  } catch (error) {
    console.error('Error listing clearable sections:', error)
    return NextResponse.json(
      { error: 'Failed to list clearable sections' },
      { status: 500 }
    )
  }
}

// POST /api/database/clear-section
// Body: { collection: 'customers' | 'orders' | ... }
// Deletes ALL documents from the requested collection. Admin-only.
// Returns the deletedCount so the UI can show "N records deleted".
export async function POST(request: Request) {
  try {
    await connectDB()

    const session = await getSession()
    if (!session) {
      return NextResponse.json({ error: 'Unauthorized' }, { status: 401 })
    }
    if (session.role !== 'admin') {
      return NextResponse.json(
        { error: 'Forbidden — only admins can clear sections' },
        { status: 403 }
      )
    }

    const body = await request.json()
    const { collection } = body as { collection?: string }

    if (!collection) {
      return NextResponse.json(
        { error: 'collection is required' },
        { status: 400 }
      )
    }

    const target = CLEARABLE.find((c) => c.key === collection)
    if (!target) {
      return NextResponse.json(
        { error: `Unknown collection "${collection}"` },
        { status: 400 }
      )
    }

    const result = await target.model.deleteMany({})

    return NextResponse.json({
      message: `Cleared ${result.deletedCount} record${result.deletedCount === 1 ? '' : 's'} from ${target.label}`,
      collection: target.key,
      label: target.label,
      deletedCount: result.deletedCount,
    })
  } catch (error) {
    console.error('Error clearing section:', error)
    return NextResponse.json(
      { error: 'Failed to clear section: ' + (error instanceof Error ? error.message : 'unknown error') },
      { status: 500 }
    )
  }
}
```

/api/database/route.ts 408 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB, toObject } from '@/lib/db'
import { requireAdmin } from '@/lib/auth'
import {
  Company,
  User,
  Customer,
  Production,
  Stock,
  DailySell,
  CustomerPayment,
  LabourPayment,
  TractorPayment,
  DustPurchase,
  CementPurchase,
  Hardner,
  Electricity,
  FactoryStuff,
  Order,
  Dispatch,
  Payment,
  Expense,
  Bill,
} from '@/lib/models'

// Force dynamic — never cache database backup/restore responses
export const dynamic = 'force-dynamic'
export const revalidate = 0

// ─────────────────────────────────────────────────────────────────────
// All 19 collections we back up / restore / clear.
// Listed in dependency order so that restore() can insert parents before
// children (Customers before Orders, Orders before Dispatches, etc.) — even
// though Mongoose doesn't enforce FK on insert, this keeps the data
// referentially consistent at the end of the restore.
// ─────────────────────────────────────────────────────────────────────
type ModelLike = {
  find: (...args: any[]) => any
  deleteMany: (...args: any[]) => any
  insertMany: (docs: any[], opts?: any) => Promise<any>
  countDocuments: (filter?: any) => Promise<number>
}

const COLLECTIONS: { key: string; model: ModelLike; preservedOnClear?: boolean }[] = [
  { key: 'companies',         model: Company as unknown as ModelLike, preservedOnClear: true },
  { key: 'users',             model: User as unknown as ModelLike,     preservedOnClear: true },
  { key: 'customers',         model: Customer as unknown as ModelLike },
  { key: 'productions',       model: Production as unknown as ModelLike },
  { key: 'stocks',            model: Stock as unknown as ModelLike },
  { key: 'dailySells',        model: DailySell as unknown as ModelLike },
  { key: 'customerPayments',  model: CustomerPayment as unknown as ModelLike },
  { key: 'labourPayments',    model: LabourPayment as unknown as ModelLike },
  { key: 'tractorPayments',   model: TractorPayment as unknown as ModelLike },
  { key: 'dustPurchases',     model: DustPurchase as unknown as ModelLike },
  { key: 'cementPurchases',   model: CementPurchase as unknown as ModelLike },
  { key: 'hardners',          model: Hardner as unknown as ModelLike },
  { key: 'electricities',     model: Electricity as unknown as ModelLike },
  { key: 'factoryStuffs',     model: FactoryStuff as unknown as ModelLike },
  { key: 'orders',            model: Order as unknown as ModelLike },
  { key: 'dispatches',        model: Dispatch as unknown as ModelLike },
  { key: 'payments',          model: Payment as unknown as ModelLike },
  { key: 'expenses',          model: Expense as unknown as ModelLike },
  { key: 'bills',             model: Bill as unknown as ModelLike },
]

// Convert a Mongoose doc to a backup-safe plain object.
// Differs from `toObject` in that we KEEP `_id` (as a string) so that
// references between collections (e.g., Order.customerId → Customer._id)
// survive a backup → clear → restore cycle. We also strip `__v`.
function toBackupObject(doc: any): any {
  if (!doc) return doc
  if (Array.isArray(doc)) return doc.map(toBackupObject)
  const obj = doc.toObject ? doc.toObject() : { ...doc }
  // Preserve _id as a string — when restoring, we set it back so Mongoose
  // uses our value instead of generating a new ObjectId.
  if (obj._id) {
    obj._id = obj._id.toString()
  }
  if (obj.__v !== undefined) delete obj.__v
  // Convert any nested ObjectId instances to strings (customerId, orderId, etc.)
  for (const key of Object.keys(obj)) {
    const v = obj[key]
    if (v && typeof v === 'object' && v._bsontype === 'ObjectID') {
```

```
      obj[key] = v.toString()
    } else if (Array.isArray(v)) {
      obj[key] = v.map((item: any) =>
        item && typeof item === 'object' && item._bsontype === 'ObjectID'
          ? item.toString()
          : item
      )
    }
  }
  return obj
}

// ─────────────────────────────────────────────────────────────────────
// GET — Export a complete backup as JSON.
// Returns { version, exportedAt, data: { ...19 collections... }, counts: { ... } }
// ─────────────────────────────────────────────────────────────────────
export async function GET(request: Request) {
  try {
    const auth = await requireAdmin()
    if (auth instanceof NextResponse) return auth

    await connectDB()

    // Fetch all collections in parallel
    const findResults = await Promise.all(
      COLLECTIONS.map(({ model }) => model.find({}).lean())
    )

    const data: Record<string, unknown[]> = {}
    const counts: Record<string, number> = {}
    COLLECTIONS.forEach(({ key }, i) => {
      const rows = findResults[i]
      data[key] = rows.map(toBackupObject)
      counts[key] = rows.length
    })

    // Strip passwords from users before exporting — they cannot be restored
    // anyway because they're hashed with a salt that lives server-side.
    if (Array.isArray(data.users)) {
      data.users = data.users.map((u: any) => {
        const { password, ...safe } = u
        return safe
      })
    }

    const payload = {
      version: 2,
      exportedAt: new Date().toISOString(),
      data,
      counts,
    }

    const res = NextResponse.json(payload)
    res.headers.set('Cache-Control', 'no-store, no-cache, must-revalidate, max-age=0')
    res.headers.set('Pragma', 'no-cache')
    res.headers.set('Expires', '0')
    return res
  } catch (error) {
    console.error('Error exporting backup:', error)
    return NextResponse.json(
      { error: 'Failed to export backup: ' + (error instanceof Error ? error.message : 'unknown error') },
      { status: 500 }
    )
  }
}

// ─────────────────────────────────────────────────────────────────────
// PUT — Restore a backup (MERGE mode).
//
// IMPORTANT — MERGE SEMANTICS (not REPLACE):
//   • Docs in the backup file with the same `_id` as an existing doc
//     → REPLACE the existing doc (backup version wins).
//   • Docs in the backup file with a new `_id`
//     → INSERTED as new documents.
//   • Docs in the current DB whose `_id` is NOT in the backup file
//     → LEFT UNTOUCHED (current data survives).
//
// This is the safe behaviour the user expects: if they took a backup,
// entered some new data, then restored the backup, the new data is NOT lost.
// Only data whose `_id` matches a backup row gets overwritten.
//
// Implementation: per collection, we issue a `bulkWrite` of `replaceOne`
// operations with `upsert: true`. replaceOne (vs updateOne with $set) makes
// the entire backup doc replace the existing doc — so removed fields in the
// backup don't linger. upsert:true makes new _ids insert as new docs.
//
```

```
// Payload shapes accepted (for backwards compatibility):
//   Shape A (v2 backup file): { data: { customers: [...], ... }, counts, version }
//     → frontend calls api.restoreBackup(parsedFile) which wraps once more,
//       so the server actually sees { data: { data: {...}, counts, ... } }
//   Shape B (already-unwrapped): { data: { customers: [...], ... } }
//   Shape C (raw): { customers: [...], ... } — sent directly without wrapping
// We normalise all three to a single `data` object before processing.
// ──────────────────────────────────────────────────────────────────────
export async function PUT(request: Request) {
  try {
    const auth = await requireAdmin()
    if (auth instanceof NextResponse) return auth

    await connectDB()

    const body = await request.json()
    console.log('[restore] received body top-level keys:', Object.keys(body || {}))

    // Normalise: extract the actual collection map from any of the supported shapes
    let data: Record<string, unknown[]> = {}
    if (body?.data?.data && typeof body.data.data === 'object') {
      // Shape A — frontend wrapped a v2 backup file
      data = body.data.data
      console.log('[restore] detected Shape A (double-wrapped v2 file)')
    } else if (body?.data && (Array.isArray((body.data as any).customers) || (body.data as any)?.customers !== undefined)) {
      // Shape B — frontend passed the inner data object directly
      data = body.data as Record<string, unknown[]>
      console.log('[restore] detected Shape B (unwrapped v2 file)')
    } else if (body?.customers !== undefined || body?.data) {
      // Shape C — raw collections, or body.data is the collections map
      data = (body.data || body) as Record<string, unknown[]>
      console.log('[restore] detected Shape C (raw collections)')
    }

    // Log every collection's row count for debugging
    for (const k of Object.keys(data)) {
      const v = data[k]
      console.log(`[restore] payload collection "${k}":`, Array.isArray(v) ? `${v.length} rows` : typeof v)
    }

    // Safety: if we still don't have any recognizable collection, abort early
    // with a clear error so the user knows the file is corrupt.
    const recognizedKeys = COLLECTIONS.map((c) => c.key)
    const hasAnyCollection = recognizedKeys.some((k) => Array.isArray(data[k]))
    if (!hasAnyCollection) {
      return NextResponse.json(
        {
          error:
            'Backup file format not recognised. Expected a JSON export from the Admin → Database → Export Backup button.',
        },
        { status: 400 }
      )
    }

    // MERGE restore — NO deleteMany. For each collection, build a bulkWrite
    // of replaceOne + upsert operations keyed by _id. This preserves docs
    // whose _id is not in the backup file (the user's "current data").
    //
    // IMPORTANT: We SKIP the `users` collection during restore. The export
    // route strips User.password for security, so the backup file has no
    // usable passwords. Restoring users would either (a) replace existing
    // users with password-less docs (validation error) or (b) inject random
    // placeholder passwords that lock the user out of their own ERP.
    // Skipping users entirely is the safe choice — authentication state is
    // never restored from a backup file.
    const SKIP_ON_RESTORE = new Set(['users'])

    const counts: Record<string, number> = { inserted: 0, replaced: 0 }
    const perCollection: Record<string, { inserted: number; replaced: number; skipped: number }> = {}
    const errors: Record<string, string> = {}

    for (const { key, model } of COLLECTIONS) {
      if (SKIP_ON_RESTORE.has(key)) {
        perCollection[key] = { inserted: 0, replaced: 0, skipped: -1 } // -1 = intentionally skipped
        console.log(`[restore] ${key}: intentionally skipped (auth state never restored from backup)`)
        continue
      }
      const rows = data[key]
      if (!Array.isArray(rows) || rows.length === 0) {
        perCollection[key] = { inserted: 0, replaced: 0, skipped: 0 }
        continue
      }
      try {
        // Sanitize each row: ensure _id is a valid ObjectId string, convert
        // date strings back to Date objects for timestamp fields, etc.
        const sanitized = rows
```

```
          .map((row: any) => sanitizeRow(row, key))
          // Drop rows without a usable _id — without one we can't upsert and
          // Mongoose would generate a fresh ObjectId, which would create
          // duplicates on subsequent restores. Better to skip than corrupt.
          .filter((r: any) => r && r._id)

        if (sanitized.length === 0) {
          perCollection[key] = { inserted: 0, replaced: 0, skipped: rows.length }
          continue
        }

        const ops = sanitized.map((doc: any) => ({
          replaceOne: {
            filter: { _id: doc._id },
            replacement: doc,
            upsert: true,
          },
        }))

        const result: any = await (model as any).bulkWrite(ops, { ordered: false })
        const inserted = result?.upsertedCount ?? 0
        const replaced = result?.modifiedCount ?? 0
        perCollection[key] = {
          inserted,
          replaced,
          skipped: rows.length - sanitized.length,
        }
        counts.inserted += inserted
        counts.replaced += replaced
        console.log(`[restore] ${key}: inserted=${inserted} replaced=${replaced} skipped=${rows.length - sanitized.length}`)
      } catch (err: any) {
        // bulkWrite with ordered:false can throw a partial-error. Treat as
        // best-effort: surface what we know.
        const inserted = err?.result?.upsertedCount ?? 0
        const replaced = err?.result?.modifiedCount ?? 0
        perCollection[key] = { inserted, replaced, skipped: 0 }
        counts.inserted += inserted
        counts.replaced += replaced
        errors[key] = err?.message || String(err)
        console.error(`[restore] partial failure for ${key}:`, err?.message)
      }
    }

    const totalAffected = counts.inserted + counts.replaced
    const res = NextResponse.json({
      message: `Restore complete (MERGE mode) — ${counts.inserted} new + ${counts.replaced} replaced = ${totalAffected} docs affected. Current data not
      mode: 'merge',
      counts,
      perCollection,
      errors: Object.keys(errors).length ? errors : undefined,
    })
    res.headers.set('Cache-Control', 'no-store, no-cache, must-revalidate, max-age=0')
    return res
  } catch (error) {
    console.error('Error restoring backup:', error)
    return NextResponse.json(
      { error: 'Failed to restore backup: ' + (error instanceof Error ? error.message : 'unknown error') },
      { status: 500 }
    )
  }
}

// Sanitize a row about to be inserted via insertMany:
//   1. Keep _id (so cross-collection references survive)
//   2. Strip the `id` field — Mongoose treats it as a strict-schema field
//      only if defined; otherwise it gets stored as a useless extra key
//   3. Convert ISO date strings back to Date objects for createdAt/updatedAt
//   4. Per-collection quirks: e.g. User.password is required by schema but
//      the export route strips passwords for security. During restore we
//      substitute a placeholder so Mongoose validation doesn't reject the
//      entire users collection. The admin can reset passwords later via
//      the User Management screen.
function sanitizeRow(row: any, collectionKey: string): any {
  if (!row || typeof row !== 'object') return row
  const out: any = { ...row }

  // Keep _id as a string — Mongoose will cast it back to ObjectId on insert
  // (works for any field declared as ObjectId in the schema, including _id).
  if (out._id !== undefined && out._id !== null) {
    out._id = String(out._id)
  } else {
    delete out._id // let Mongoose generate a fresh one
  }

  // Drop the `id` alias — only `_id` is meaningful for restore.
  delete out.id
```

```
  // Drop __v if present (will be regenerated)
  delete out.__v

  // Convert date strings back to Date for the standard timestamp fields.
  // Mongoose's auto-cast usually handles this, but being explicit avoids
  // edge cases where a serialized Date came through as a string.
  for (const k of ['createdAt', 'updatedAt']) {
    if (out[k] && typeof out[k] === 'string') {
      const d = new Date(out[k])
      if (!isNaN(d.getTime())) out[k] = d
    }
  }

  // Per-collection quirks
  if (collectionKey === 'users') {
    // User schema requires `password`. Export strips it for security.
    // Restore with a placeholder so the row passes validation — admin
    // can reset passwords later. We use a recognisable placeholder so
    // it's obvious in the DB which accounts need a password reset.
    if (!out.password || typeof out.password !== 'string' || out.password.length === 0) {
      out.password = 'veda-reset-' + Math.random().toString(36).slice(2, 10)
    }
    // Also ensure `active` defaults to true if missing
    if (out.active === undefined) out.active = true
  }

  return out
}

// ─────────────────────────────────────────────────────────────────────
// DELETE — Clear all transactional data.
// Preserves Company and User collections (so the user can log back in and
// the company profile / branding stays intact). All other 17 collections are
// wiped.
// ─────────────────────────────────────────────────────────────────────
export async function DELETE(request: Request) {
  try {
    const auth = await requireAdmin()
    if (auth instanceof NextResponse) return auth

    await connectDB()

    const cleared: Record<string, number> = {}
    const toClear = COLLECTIONS.filter((c) => !c.preservedOnClear)
    const results = await Promise.all(
      toClear.map(({ model }) => model.deleteMany({}))
    )
    toClear.forEach(({ key }, i) => {
      cleared[key] = (results[i] as any)?.deletedCount ?? 0
    })

    const totalCleared = Object.values(cleared).reduce((s, n) => s + n, 0)
    const res = NextResponse.json({
      message: `Cleared ${totalCleared} documents across ${toClear.length} collections. Users and company preserved.`,
      cleared,
    })
    res.headers.set('Cache-Control', 'no-store, no-cache, must-revalidate, max-age=0')
    return res
  } catch (error) {
    console.error('Error clearing data:', error)
    return NextResponse.json(
      { error: 'Failed to clear data: ' + (error instanceof Error ? error.message : 'unknown error') },
      { status: 500 }
    )
  }
}
```

/api/debug/payment-sync/backfill/route.ts 93 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB } from '@/lib/db'
import { Payment, CustomerPayment, Customer } from '@/lib/models'
import { requireAdmin } from '@/lib/auth'

// Force dynamic — never cache
export const dynamic = 'force-dynamic'
export const revalidate = 0

// POST /api/debug/payment-sync/backfill
//
// One-time backfill: for every Payment that does NOT have a customerPaymentId,
// create the missing CustomerPayment mirror and link it back. Safe to run
// repeatedly — skips Payments that already have a mirror.
//
// This handles the case where Payments were created BEFORE the cross-module
// sync feature was deployed, so they never got their mirror record.
//
// Admin-only — mutates data.
export async function POST() {
  try {
    const auth = await requireAdmin()
    if (auth instanceof NextResponse) return auth

    await connectDB()

    const unlinked = await Payment.find({
      $or: [
        { customerPaymentId: null },
        { customerPaymentId: { $exists: false } },
      ],
    }).lean()

    const results: Array<{ paymentId: string; ok: boolean; error?: string }> = []

    for (const p of unlinked as any[]) {
      try {
        // Resolve customer name + address
        const customer = await Customer.findById(p.customerId).lean()
        if (!customer) {
          results.push({
            paymentId: String(p._id),
            ok: false,
            error: 'customer not found',
          })
          continue
        }

        const mirror = await CustomerPayment.create({
          date: p.date,
          name: String((customer as any).name || ''),
          address: String((customer as any).address || ''),
          amount: Number(p.amount) || 0,
          remarks: [
            p.paymentType,
            p.billNumber ? `Bill ${p.billNumber}` : '',
            p.remarks || '',
            '[synced from Payments]',
          ]
            .filter(Boolean)
            .join(' • '),
        })

        await Payment.findByIdAndUpdate(p._id, {
          customerPaymentId: mirror._id,
        })

        results.push({ paymentId: String(p._id), ok: true })
      } catch (err) {
        results.push({
          paymentId: String(p._id),
          ok: false,
          error: err instanceof Error ? err.message : String(err),
        })
      }
    }

    return NextResponse.json({
      totalScanned: unlinked.length,
      successCount: results.filter((r) => r.ok).length,
      failureCount: results.filter((r) => !r.ok).length,
      results,
    })
```

```
  } catch (err) {
    return NextResponse.json(
      {
        error: err instanceof Error ? err.message : String(err),
      },
      { status: 500 }
    )
  }
}
```

/api/debug/payment-sync/route.ts 78 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB } from '@/lib/db'
import { Payment, CustomerPayment } from '@/lib/models'
import { requireAdmin } from '@/lib/auth'

// Force dynamic — never cache
export const dynamic = 'force-dynamic'
export const revalidate = 0

// GET /api/debug/payment-sync
// Returns a diagnostic snapshot showing the link state between Payment and
// CustomerPayment records. Useful to verify the cross-module sync is working.
// Admin-only — exposes financial PII.
export async function GET() {
  try {
    const auth = await requireAdmin()
    if (auth instanceof NextResponse) return auth

    await connectDB()

    const payments = await Payment.find({})
      .sort({ createdAt: -1 })
      .limit(20)
      .populate('customerId')
      .lean()

    const customerPayments = await CustomerPayment.find({})
      .sort({ createdAt: -1 })
      .limit(20)
      .lean()

    const linked = payments.filter((p: any) => p.customerPaymentId)
    const unlinked = payments.filter((p: any) => !p.customerPaymentId)

    return NextResponse.json({
      summary: {
        totalPayments: payments.length,
        linkedToCustomerPayment: linked.length,
        unlinked: unlinked.length,
        totalCustomerPayments: customerPayments.length,
      },
      unlinkedPayments: unlinked.map((p: any) => ({
        id: String(p._id),
        amount: p.amount,
        date: p.date,
        paymentType: p.paymentType,
        customerId: p.customerId?._id ?? p.customerId,
        customerName: p.customerId?.name ?? null,
        customerPaymentId: p.customerPaymentId ?? null,
        createdAt: p.createdAt,
      })),
      recentPayments: payments.map((p: any) => ({
        id: String(p._id),
        amount: p.amount,
        date: p.date,
        paymentType: p.paymentType,
        customerName: p.customerId?.name ?? null,
        customerPaymentId: p.customerPaymentId ? String(p.customerPaymentId) : null,
      })),
      recentCustomerPayments: customerPayments.map((cp: any) => ({
        id: String(cp._id),
        date: cp.date,
        name: cp.name,
        amount: cp.amount,
        remarks: cp.remarks,
        createdAt: cp.createdAt,
      })),
    })
  } catch (err) {
    return NextResponse.json(
      {
        error: err instanceof Error ? err.message : String(err),
      },
      { status: 500 }
    )
  }
}
```

/api/debug/sync/route.ts 97 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB } from '@/lib/db'
import { Production, Stock } from '@/lib/models'
import { syncStockForDate } from '@/lib/sync-stock'
import { requireAdmin } from '@/lib/auth'

// Force dynamic — never cache
export const dynamic = 'force-dynamic'
export const revalidate = 0

// GET /api/debug/sync?date=YYYY-MM-DD
// Runs the stock sync for the given date and returns detailed debug info.
// Admin-only — mutates data (syncStockForDate writes Stock documents).
export async function GET(request: Request) {
  const { searchParams } = new URL(request.url)
  const date = searchParams.get('date')

  if (!date) {
    return NextResponse.json(
      { error: 'Missing ?date=YYYY-MM-DD parameter' },
      { status: 400 }
    )
  }

  // Validate date format to prevent injection / weird Mongo queries
  if (!/^\d{4}-\d{2}-\d{2}$/.test(date)) {
    return NextResponse.json(
      { error: 'Invalid date format — expected YYYY-MM-DD' },
      { status: 400 }
    )
  }

  const steps: Array<Record<string, unknown>> = []
  const debug: Record<string, unknown> = {
    input_date: date,
    steps,
  }

  try {
    const auth = await requireAdmin()
    if (auth instanceof NextResponse) return auth

    await connectDB()
    steps.push({ step: 'connectDB', ok: true })

    // 1. Find productions for this date
    const productions = await Production.find({ date }).lean()
    steps.push({
      step: 'findProductions',
      ok: true,
      count: productions.length,
      sample: productions[0] ?? null,
    })

    // 2. Find existing stock for this date
    const existingStock = await Stock.findOne({ date }).lean()
    steps.push({
      step: 'findExistingStock',
      ok: true,
      found: !!existingStock,
      stock: existingStock,
    })

    // 3. Run the sync
    const syncResult = await syncStockForDate(date)
    steps.push({
      step: 'syncStockForDate',
      ok: true,
      result: syncResult
        ? {
            id: (syncResult as any)._id ?? (syncResult as any).id,
            date: (syncResult as any).date,
            cement: (syncResult as any).cement,
            zigZagGrey80: (syncResult as any).zigZagGrey80,
            dumbleRed80: (syncResult as any).dumbleRed80,
          }
        : null,
    })

    // 4. Re-fetch stock to confirm
    const afterStock = await Stock.findOne({ date }).lean()
    steps.push({
      step: 'verifyAfterSync',
```

```
      ok: true,
      stock: afterStock,
    })

    return NextResponse.json(debug)
  } catch (err) {
    debug.error = err instanceof Error ? {
      message: err.message,
      stack: err.stack?.split('\n').slice(0, 10),
    } : String(err)
    return NextResponse.json(debug, { status: 500 })
  }
}
```

/api/dispatch/[id]/route.ts 90 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB, toObject, extractCustomer } from '@/lib/db'
import { Dispatch, Stock } from '@/lib/models'
import { requireSession, requireRole } from '@/lib/auth'
import { normalizeDate } from '@/lib/date-utils'

export async function GET(request: Request, { params }: { params: Promise<{ id: string }> }) {
  try {
    const session = await requireSession()
    if (session instanceof NextResponse) return session

    await connectDB()
    const { id } = await params
    const dispatch = await Dispatch.findById(id).populate('customerId').populate('orderId')
    if (!dispatch) {
      return NextResponse.json({ error: 'Dispatch not found' }, { status: 404 })
    }
    const obj = toObject(dispatch)
    const { customer, customerId } = extractCustomer(dispatch)
    obj.customer = customer
    obj.customerId = customerId
    return NextResponse.json({ dispatch: obj })
  } catch (error) {
    console.error('Error fetching dispatch:', error)
    return NextResponse.json({ error: 'Failed to fetch dispatch' }, { status: 500 })
  }
}

export async function PUT(request: Request, { params }: { params: Promise<{ id: string }> }) {
  try {
    const session = await requireRole(['admin', 'operator'])
    if (session instanceof NextResponse) return session

    await connectDB()
    const { id } = await params
    const body = await request.json()
    // Normalize date to canonical YYYY-MM-DD
    // (handles dd-mm-yyyy, dd/mm/yyyy, Excel serials, Date objects, etc.)
    if (body.date) body.date = normalizeDate(body.date)

    const updateData: Record<string, unknown> = {}
    const fields = ['customerId', 'orderId', 'truckNumber', 'driverName', 'quantity', 'brickType', 'date']
    for (const field of fields) {
      if (body[field] !== undefined) updateData[field] = body[field]
    }

    const dispatch = await Dispatch.findByIdAndUpdate(id, updateData, { new: true }).populate('customerId')
    if (!dispatch) {
      return NextResponse.json({ error: 'Dispatch not found' }, { status: 404 })
    }

    const obj = toObject(dispatch)
    const { customer, customerId } = extractCustomer(dispatch)
    obj.customer = customer
    obj.customerId = customerId
    return NextResponse.json({ dispatch: obj })
  } catch (error) {
    console.error('Error updating dispatch:', error)
    return NextResponse.json({ error: 'Failed to update dispatch' }, { status: 500 })
  }
}

export async function DELETE(request: Request, { params }: { params: Promise<{ id: string }> }) {
  try {
    const session = await requireRole(['admin', 'operator'])
    if (session instanceof NextResponse) return session

    await connectDB()
    const { id } = await params

    const dispatch = await Dispatch.findById(id)
    if (!dispatch) {
      return NextResponse.json({ error: 'Dispatch not found' }, { status: 404 })
    }

    // Restore stock on delete
    const stock = await Stock.findOne({ brickType: dispatch.brickType })
    if (stock) {
      stock.currentStock += dispatch.quantity
      await stock.save()
    }

    await Dispatch.findByIdAndDelete(id)
```

```
    return NextResponse.json({ message: 'Dispatch deleted successfully' })
  } catch (error) {
    console.error('Error deleting dispatch:', error)
    return NextResponse.json({ error: 'Failed to delete dispatch' }, { status: 500 })
  }
}
```

/api/dispatch/bulk-delete/route.ts 61 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB } from '@/lib/db'
import { Dispatch } from '@/lib/models'
import { getSession } from '@/lib/auth'

export const dynamic = 'force-dynamic'

// POST /api/dispatch/bulk-delete
// Body: { ids: string[] }
// Gated behind admin/operator session — accountants are read-only.
export async function POST(request: Request) {
  try {
    await connectDB()

    const session = await getSession()
    if (!session) {
      return NextResponse.json(
        { error: 'Unauthorized — please log in' },
        { status: 401 }
      )
    }
    if (session.role === 'accountant') {
      return NextResponse.json(
        { error: 'Forbidden — accountants cannot delete dispatch entries' },
        { status: 403 }
      )
    }

    const body = await request.json()
    const { ids } = body as { ids?: string[] }

    if (!Array.isArray(ids) || ids.length === 0) {
      return NextResponse.json(
        { error: 'ids array is required and must be non-empty' },
        { status: 400 }
      )
    }

    const result = await Dispatch.deleteMany({ _id: { $in: ids } })

    if (result.deletedCount === 0) {
      return NextResponse.json(
        { error: 'No matching dispatch entries found — they may have been already deleted' },
        { status: 404 }
      )
    }

    return NextResponse.json({
      message: `${result.deletedCount} dispatch entr${result.deletedCount === 1 ? 'y' : 'ies'} deleted`,
      deletedCount: result.deletedCount,
      requestedCount: ids.length,
    })
  } catch (error) {
    console.error('Error bulk-deleting dispatches:', error)
    return NextResponse.json(
      { error: 'Failed to bulk-delete dispatch entries' },
      { status: 500 }
    )
  }
}
```

/api/dispatch/route.ts 93 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB, toObject, extractCustomer, extractOrder } from '@/lib/db'
import { Dispatch, Stock } from '@/lib/models'
import { requireSession, requireRole } from '@/lib/auth'
import { normalizeDate } from '@/lib/date-utils'

// Force dynamic — never cache list responses
export const dynamic = 'force-dynamic'
export const revalidate = 0

export async function GET() {
  try {
    const session = await requireSession()
    if (session instanceof NextResponse) return session

    await connectDB()
    const dispatches = await Dispatch.find({}).populate('customerId').populate('orderId').sort({ createdAt: -1 })

    const result = dispatches.map((d: any) => {
      const obj = toObject(d)
      const { customer, customerId } = extractCustomer(d)
      const { order, orderId } = extractOrder(d)
      obj.customer = customer
      obj.customerId = customerId
      obj.order = order
      obj.orderId = orderId
      obj.customerName = customer?.name || ''
      return obj
    })

    return NextResponse.json({ dispatches: result })
  } catch (error) {
    console.error('Error fetching dispatches:', error)
    return NextResponse.json({ error: 'Failed to fetch dispatches' }, { status: 500 })
  }
}

export async function POST(request: Request) {
  try {
    const session = await requireRole(['admin', 'operator'])
    if (session instanceof NextResponse) return session

    await connectDB()
    const body = await request.json()
    // Normalize date to canonical YYYY-MM-DD
    // (handles dd-mm-yyyy, dd/mm/yyyy, Excel serials, Date objects, etc.)
    if (body.date) body.date = normalizeDate(body.date)

    if (!body.customerId || !body.truckNumber || !body.quantity || !body.brickType || !body.date) {
      return NextResponse.json({ error: 'Missing required fields' }, { status: 400 })
    }

    const count = await Dispatch.countDocuments({})
    const { Company } = await import('@/lib/models')
    const company = await Company.findOne({})
    const prefix = company?.dispatchPrefix || 'DSP'
    const dispatchNumber = `${prefix}-${String(count + 1).padStart(4, '0')}`

    const dispatch = await Dispatch.create({
      dispatchNumber,
      customerId: body.customerId,
      orderId: body.orderId || null,
      truckNumber: body.truckNumber,
      driverName: body.driverName || '',
      quantity: Number(body.quantity),
      brickType: body.brickType,
      date: body.date,
    })

    // Auto-update stock - reduce on dispatch
    const stock = await Stock.findOne({ brickType: body.brickType })
    if (stock) {
      stock.currentStock = Math.max(0, stock.currentStock - Number(body.quantity))
      await stock.save()
    }

    const populated = await Dispatch.findById(dispatch._id).populate('customerId').populate('orderId')
    const obj = toObject(populated)
    const { customer, customerId } = extractCustomer(populated)
    const { order, orderId } = extractOrder(populated)
    obj.customer = customer
    obj.customerId = customerId
    obj.order = order
```

```
    obj.orderId = orderId
    obj.customerName = customer?.name || ''

    return NextResponse.json({ dispatch: obj }, { status: 201 })
  } catch (error) {
    console.error('Error creating dispatch:', error)
    return NextResponse.json({ error: 'Failed to create dispatch' }, { status: 500 })
  }
}
```

/api/dust-purchase/[id]/route.ts 105 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB, toObject } from '@/lib/db'
import { DustPurchase } from '@/lib/models'
import { requireSession, requireRole } from '@/lib/auth'
import { normalizeDate } from '@/lib/date-utils'

// Force dynamic — never cache individual responses
export const dynamic = 'force-dynamic'
export const revalidate = 0

// Whitelist of updatable fields. Must match DustPurchaseSchema in src/lib/models.ts.
const FIELDS = [
  'date',
  'vendorName',
  'cementName',
  'quantity',
  'rate',
  'totalAmount',
  'paidAmount',
  'transportationCharge',
  'gst',
  'remarks',
] as const

// GET /api/dust-purchase/[id] — fetch a single dust-purchase entry
export async function GET(
  _request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const session = await requireSession()
    if (session instanceof NextResponse) return session

    await connectDB()
    const { id } = await params
    const record = await DustPurchase.findById(id).lean()
    if (!record) {
      return NextResponse.json({ error: 'dustPurchase entry not found' }, { status: 404 })
    }
    return NextResponse.json({ dustPurchase: toObject(record) })
  } catch (error) {
    console.error('Error fetching dust-purchase entry:', error)
    return NextResponse.json({ error: 'Failed to fetch dust-purchase entry' }, { status: 500 })
  }
}

// PUT /api/dust-purchase/[id] — update a single dust-purchase entry
export async function PUT(
  request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const session = await requireRole(['admin', 'operator', 'accountant'])
    if (session instanceof NextResponse) return session

    await connectDB()
    const { id } = await params
    const body = await request.json()
    // Normalize date to canonical YYYY-MM-DD
    // (handles dd-mm-yyyy, dd/mm/yyyy, Excel serials, Date objects, etc.)
    if (body.date) body.date = normalizeDate(body.date)

    const updateData: Record<string, unknown> = {}
    for (const field of FIELDS) {
      if (body[field] !== undefined) {
        updateData[field] = body[field]
      }
    }

    const record = await DustPurchase.findByIdAndUpdate(id, updateData, { new: true })
    if (!record) {
      return NextResponse.json({ error: 'dustPurchase entry not found' }, { status: 404 })
    }

    return NextResponse.json({ dustPurchase: toObject(record) })
  } catch (error) {
    console.error('Error updating dust-purchase entry:', error)
    return NextResponse.json({ error: 'Failed to update dust-purchase entry' }, { status: 500 })
  }
}

// DELETE /api/dust-purchase/[id] — delete a single dust-purchase entry
export async function DELETE(
```

```
  _request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const session = await requireRole(['admin', 'operator', 'accountant'])
    if (session instanceof NextResponse) return session

    await connectDB()
    const { id } = await params

    const record = await DustPurchase.findByIdAndDelete(id)
    if (!record) {
      return NextResponse.json({ error: 'dustPurchase entry not found' }, { status: 404 })
    }

    return NextResponse.json({ message: 'dustPurchase entry deleted successfully' })
  } catch (error) {
    console.error('Error deleting dust-purchase entry:', error)
    return NextResponse.json({ error: 'Failed to delete dust-purchase entry' }, { status: 500 })
  }
}
```

/api/dust-purchase/bulk-delete/route.ts 61 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB } from '@/lib/db'
import { DustPurchase } from '@/lib/models'
import { getSession } from '@/lib/auth'

export const dynamic = 'force-dynamic'

// POST /api/dust-purchase/bulk-delete
// Body: { ids: string[] }
// Gated behind admin/operator session — accountants are read-only.
export async function POST(request: Request) {
  try {
    await connectDB()

    const session = await getSession()
    if (!session) {
      return NextResponse.json(
        { error: 'Unauthorized — please log in' },
        { status: 401 }
      )
    }
    if (session.role === 'accountant') {
      return NextResponse.json(
        { error: 'Forbidden — accountants cannot delete dust purchase entries' },
        { status: 403 }
      )
    }

    const body = await request.json()
    const { ids } = body as { ids?: string[] }

    if (!Array.isArray(ids) || ids.length === 0) {
      return NextResponse.json(
        { error: 'ids array is required and must be non-empty' },
        { status: 400 }
      )
    }

    const result = await DustPurchase.deleteMany({ _id: { $in: ids } })

    if (result.deletedCount === 0) {
      return NextResponse.json(
        { error: 'No matching dust purchase entries found — they may have been already deleted' },
        { status: 404 }
      )
    }

    return NextResponse.json({
      message: `${result.deletedCount} dust purchase entr${result.deletedCount === 1 ? 'y' : 'ies'} deleted`,
      deletedCount: result.deletedCount,
      requestedCount: ids.length,
    })
  } catch (error) {
    console.error('Error bulk-deleting dust purchases:', error)
    return NextResponse.json(
      { error: 'Failed to bulk-delete dust purchase entries' },
      { status: 500 }
    )
  }
}
```

/api/dust-purchase/route.ts 57 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB, toObject } from '@/lib/db'
import { DustPurchase } from '@/lib/models'
import { requireSession, requireRole } from '@/lib/auth'
import { normalizeDate } from '@/lib/date-utils'

// Force dynamic — never cache list responses
export const dynamic = 'force-dynamic'
export const revalidate = 0

export async function GET() {
  try {
    const session = await requireSession()
    if (session instanceof NextResponse) return session

    await connectDB()
    const records = await DustPurchase.find({}).sort({ date: -1 }).lean()
    return NextResponse.json({ dustPurchases: records.map(toObject) })
  } catch (error) {
    console.error('Error fetching dust purchases:', error)
    return NextResponse.json({ error: 'Failed to fetch dust purchases' }, { status: 500 })
  }
}

export async function POST(request: Request) {
  try {
    const session = await requireRole(['admin', 'operator', 'accountant'])
    if (session instanceof NextResponse) return session

    await connectDB()
    const body = await request.json()
    // Normalize date to canonical YYYY-MM-DD
    // (handles dd-mm-yyyy, dd/mm/yyyy, Excel serials, Date objects, etc.)
    if (body.date) body.date = normalizeDate(body.date)
    if (!body.date || !body.vendorName || !body.quantity || !body.rate) {
      return NextResponse.json({ error: 'Date, vendor name, quantity and rate are required' }, { status: 400 })
    }
    const totalAmount = Number(body.quantity) * Number(body.rate)
    const record = await DustPurchase.create({
      date: body.date,
      vendorName: body.vendorName,
      cementName: body.cementName || '',
      quantity: Number(body.quantity),
      rate: Number(body.rate),
      totalAmount,
      paidAmount: Number(body.paidAmount) || 0,
      transportationCharge: Number(body.transportationCharge) || 0,
      gst: Number(body.gst) || 0,
      remarks: body.remarks || '',
    })
    return NextResponse.json({ dustPurchase: toObject(record) }, { status: 201 })
  } catch (error) {
    console.error('Error creating dust purchase:', error)
    return NextResponse.json({ error: 'Failed to create dust purchase' }, { status: 500 })
  }
}
```

/api/electricity/[id]/route.ts 100 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB, toObject } from '@/lib/db'
import { Electricity } from '@/lib/models'
import { requireSession, requireRole } from '@/lib/auth'
import { normalizeDate } from '@/lib/date-utils'

// Force dynamic — never cache individual responses
export const dynamic = 'force-dynamic'
export const revalidate = 0

// Whitelist of updatable fields. Must match ElectricitySchema in src/lib/models.ts.
const FIELDS = [
  'date',
  'name',
  'work',
  'amount',
  'remarks',
] as const

// GET /api/electricity/[id] — fetch a single electricity entry
export async function GET(
  _request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const session = await requireSession()
    if (session instanceof NextResponse) return session

    await connectDB()
    const { id } = await params
    const record = await Electricity.findById(id).lean()
    if (!record) {
      return NextResponse.json({ error: 'electricity entry not found' }, { status: 404 })
    }
    return NextResponse.json({ electricity: toObject(record) })
  } catch (error) {
    console.error('Error fetching electricity entry:', error)
    return NextResponse.json({ error: 'Failed to fetch electricity entry' }, { status: 500 })
  }
}

// PUT /api/electricity/[id] — update a single electricity entry
export async function PUT(
  request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const session = await requireRole(['admin', 'operator', 'accountant'])
    if (session instanceof NextResponse) return session

    await connectDB()
    const { id } = await params
    const body = await request.json()
    // Normalize date to canonical YYYY-MM-DD
    // (handles dd-mm-yyyy, dd/mm/yyyy, Excel serials, Date objects, etc.)
    if (body.date) body.date = normalizeDate(body.date)

    const updateData: Record<string, unknown> = {}
    for (const field of FIELDS) {
      if (body[field] !== undefined) {
        updateData[field] = body[field]
      }
    }

    const record = await Electricity.findByIdAndUpdate(id, updateData, { new: true })
    if (!record) {
      return NextResponse.json({ error: 'electricity entry not found' }, { status: 404 })
    }

    return NextResponse.json({ electricity: toObject(record) })
  } catch (error) {
    console.error('Error updating electricity entry:', error)
    return NextResponse.json({ error: 'Failed to update electricity entry' }, { status: 500 })
  }
}

// DELETE /api/electricity/[id] — delete a single electricity entry
export async function DELETE(
  _request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const session = await requireRole(['admin', 'operator', 'accountant'])
```

```
    if (session instanceof NextResponse) return session

    await connectDB()
    const { id } = await params

    const record = await Electricity.findByIdAndDelete(id)
    if (!record) {
      return NextResponse.json({ error: 'electricity entry not found' }, { status: 404 })
    }

    return NextResponse.json({ message: 'electricity entry deleted successfully' })
  } catch (error) {
    console.error('Error deleting electricity entry:', error)
    return NextResponse.json({ error: 'Failed to delete electricity entry' }, { status: 500 })
  }
}
```

/api/electricity/bulk-delete/route.ts 61 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB } from '@/lib/db'
import { Electricity } from '@/lib/models'
import { getSession } from '@/lib/auth'

export const dynamic = 'force-dynamic'

// POST /api/electricity/bulk-delete
// Body: { ids: string[] }
// Gated behind admin/operator session — accountants are read-only.
export async function POST(request: Request) {
  try {
    await connectDB()

    const session = await getSession()
    if (!session) {
      return NextResponse.json(
        { error: 'Unauthorized — please log in' },
        { status: 401 }
      )
    }
    if (session.role === 'accountant') {
      return NextResponse.json(
        { error: 'Forbidden — accountants cannot delete electricity entries' },
        { status: 403 }
      )
    }

    const body = await request.json()
    const { ids } = body as { ids?: string[] }

    if (!Array.isArray(ids) || ids.length === 0) {
      return NextResponse.json(
        { error: 'ids array is required and must be non-empty' },
        { status: 400 }
      )
    }

    const result = await Electricity.deleteMany({ _id: { $in: ids } })

    if (result.deletedCount === 0) {
      return NextResponse.json(
        { error: 'No matching electricity entries found — they may have been already deleted' },
        { status: 404 }
      )
    }

    return NextResponse.json({
      message: `${result.deletedCount} electricity entr${result.deletedCount === 1 ? 'y' : 'ies'} deleted`,
      deletedCount: result.deletedCount,
      requestedCount: ids.length,
    })
  } catch (error) {
    console.error('Error bulk-deleting electricity entries:', error)
    return NextResponse.json(
      { error: 'Failed to bulk-delete electricity entries' },
      { status: 500 }
    )
  }
}
```

/api/electricity/route.ts 51 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB, toObject } from '@/lib/db'
import { Electricity } from '@/lib/models'
import { requireSession, requireRole } from '@/lib/auth'
import { normalizeDate } from '@/lib/date-utils'

// Force dynamic — never cache list responses
export const dynamic = 'force-dynamic'
export const revalidate = 0

export async function GET() {
  try {
    const session = await requireSession()
    if (session instanceof NextResponse) return session

    await connectDB()
    const records = await Electricity.find({}).sort({ date: -1 }).lean()
    return NextResponse.json({ electricitys: records.map(toObject) })
  } catch (error) {
    console.error('Error fetching electricity:', error)
    return NextResponse.json({ error: 'Failed to fetch electricity' }, { status: 500 })
  }
}

export async function POST(request: Request) {
  try {
    const session = await requireRole(['admin', 'operator', 'accountant'])
    if (session instanceof NextResponse) return session

    await connectDB()
    const body = await request.json()
    // Normalize date to canonical YYYY-MM-DD
    // (handles dd-mm-yyyy, dd/mm/yyyy, Excel serials, Date objects, etc.)
    if (body.date) body.date = normalizeDate(body.date)
    if (!body.date || !body.amount) {
      return NextResponse.json({ error: 'Date and amount are required' }, { status: 400 })
    }
    const record = await Electricity.create({
      date: body.date,
      name: body.name || '',
      work: body.work || '',
      amount: Number(body.amount),
      remarks: body.remarks || '',
    })
    return NextResponse.json({ electricity: toObject(record) }, { status: 201 })
  } catch (error) {
    console.error('Error creating electricity:', error)
    return NextResponse.json({ error: 'Failed to create electricity' }, { status: 500 })
  }
}
```

/api/expenses/[id]/route.ts 71 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB, toObject } from '@/lib/db'
import { Expense } from '@/lib/models'
import { requireSession, requireRole } from '@/lib/auth'
import { normalizeDate } from '@/lib/date-utils'

export async function GET(request: Request, { params }: { params: Promise<{ id: string }> }) {
  try {
    const session = await requireSession()
    if (session instanceof NextResponse) return session

    await connectDB()
    const { id } = await params
    const expense = await Expense.findById(id)
    if (!expense) {
      return NextResponse.json({ error: 'Expense not found' }, { status: 404 })
    }
    return NextResponse.json({ expense: toObject(expense) })
  } catch (error) {
    console.error('Error fetching expense:', error)
    return NextResponse.json({ error: 'Failed to fetch expense' }, { status: 500 })
  }
}

export async function PUT(request: Request, { params }: { params: Promise<{ id: string }> }) {
  try {
    const session = await requireRole(['admin', 'accountant'])
    if (session instanceof NextResponse) return session

    await connectDB()
    const { id } = await params
    const body = await request.json()
    // Normalize date to canonical YYYY-MM-DD
    // (handles dd-mm-yyyy, dd/mm/yyyy, Excel serials, Date objects, etc.)
    if (body.date) body.date = normalizeDate(body.date)

    const updateData: Record<string, unknown> = {}
    const fields = ['category', 'amount', 'date', 'description']
    for (const field of fields) {
      if (body[field] !== undefined) updateData[field] = body[field]
    }

    const expense = await Expense.findByIdAndUpdate(id, updateData, { new: true })
    if (!expense) {
      return NextResponse.json({ error: 'Expense not found' }, { status: 404 })
    }
    return NextResponse.json({ expense: toObject(expense) })
  } catch (error) {
    console.error('Error updating expense:', error)
    return NextResponse.json({ error: 'Failed to update expense' }, { status: 500 })
  }
}

export async function DELETE(request: Request, { params }: { params: Promise<{ id: string }> }) {
  try {
    const session = await requireRole(['admin', 'accountant'])
    if (session instanceof NextResponse) return session

    await connectDB()
    const { id } = await params
    const expense = await Expense.findByIdAndDelete(id)
    if (!expense) {
      return NextResponse.json({ error: 'Expense not found' }, { status: 404 })
    }
    return NextResponse.json({ message: 'Expense deleted successfully' })
  } catch (error) {
    console.error('Error deleting expense:', error)
    return NextResponse.json({ error: 'Failed to delete expense' }, { status: 500 })
  }
}
```

/api/expenses/bulk-delete/route.ts 61 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB } from '@/lib/db'
import { Expense } from '@/lib/models'
import { getSession } from '@/lib/auth'

export const dynamic = 'force-dynamic'

// POST /api/expenses/bulk-delete
// Body: { ids: string[] }
// Gated behind admin/operator session — accountants are read-only.
export async function POST(request: Request) {
  try {
    await connectDB()

    const session = await getSession()
    if (!session) {
      return NextResponse.json(
        { error: 'Unauthorized — please log in' },
        { status: 401 }
      )
    }
    if (session.role === 'accountant') {
      return NextResponse.json(
        { error: 'Forbidden — accountants cannot delete expenses' },
        { status: 403 }
      )
    }

    const body = await request.json()
    const { ids } = body as { ids?: string[] }

    if (!Array.isArray(ids) || ids.length === 0) {
      return NextResponse.json(
        { error: 'ids array is required and must be non-empty' },
        { status: 400 }
      )
    }

    const result = await Expense.deleteMany({ _id: { $in: ids } })

    if (result.deletedCount === 0) {
      return NextResponse.json(
        { error: 'No matching expenses found — they may have been already deleted' },
        { status: 404 }
      )
    }

    return NextResponse.json({
      message: `${result.deletedCount} expense${result.deletedCount === 1 ? '' : 's'} deleted`,
      deletedCount: result.deletedCount,
      requestedCount: ids.length,
    })
  } catch (error) {
    console.error('Error bulk-deleting expenses:', error)
    return NextResponse.json(
      { error: 'Failed to bulk-delete expenses' },
      { status: 500 }
    )
  }
}
```

/api/expenses/route.ts 69 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB, toObject } from '@/lib/db'
import { Expense } from '@/lib/models'
import { requireSession, requireRole } from '@/lib/auth'
import { normalizeDate } from '@/lib/date-utils'

// Force dynamic — never cache list responses
export const dynamic = 'force-dynamic'
export const revalidate = 0

export async function GET(request: Request) {
  try {
    const session = await requireSession()
    if (session instanceof NextResponse) return session

    await connectDB()
    const { searchParams } = new URL(request.url)
    const category = searchParams.get('category')
    const dateRange = searchParams.get('date')

    const filter: any = {}
    if (category) filter.category = category

    if (dateRange) {
      const parts = dateRange.split(',')
      if (parts.length === 2) {
        filter.date = { $gte: parts[0], $lte: parts[1] }
      } else {
        filter.date = dateRange
      }
    }

    const expenses = await Expense.find(filter).sort({ date: -1 }).lean()
    return NextResponse.json({ expenses: expenses.map(toObject) })
  } catch (error) {
    console.error('Error fetching expenses:', error)
    return NextResponse.json({ error: 'Failed to fetch expenses' }, { status: 500 })
  }
}

export async function POST(request: Request) {
  try {
    const session = await requireRole(['admin', 'accountant'])
    if (session instanceof NextResponse) return session

    await connectDB()
    const body = await request.json()
    // Normalize date to canonical YYYY-MM-DD
    // (handles dd-mm-yyyy, dd/mm/yyyy, Excel serials, Date objects, etc.)
    if (body.date) body.date = normalizeDate(body.date)

    if (!body.category || !body.amount || !body.date) {
      return NextResponse.json({ error: 'Missing required fields' }, { status: 400 })
    }

    const expense = await Expense.create({
      category: body.category,
      amount: Number(body.amount),
      date: body.date,
      description: body.description || '',
    })

    return NextResponse.json({ expense: toObject(expense) }, { status: 201 })
  } catch (error) {
    console.error('Error creating expense:', error)
    return NextResponse.json({ error: 'Failed to create expense' }, { status: 500 })
  }
}
```

/api/factory-stuff/[id]/route.ts 100 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB, toObject } from '@/lib/db'
import { FactoryStuff } from '@/lib/models'
import { requireSession, requireRole } from '@/lib/auth'
import { normalizeDate } from '@/lib/date-utils'

// Force dynamic — never cache individual responses
export const dynamic = 'force-dynamic'
export const revalidate = 0

// Whitelist of updatable fields. Must match FactoryStuffSchema in src/lib/models.ts.
const FIELDS = [
  'date',
  'itemName',
  'quantity',
  'amount',
  'remarks',
] as const

// GET /api/factory-stuff/[id] — fetch a single factory-stuff entry
export async function GET(
  _request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const session = await requireSession()
    if (session instanceof NextResponse) return session

    await connectDB()
    const { id } = await params
    const record = await FactoryStuff.findById(id).lean()
    if (!record) {
      return NextResponse.json({ error: 'factoryStuff entry not found' }, { status: 404 })
    }
    return NextResponse.json({ factoryStuff: toObject(record) })
  } catch (error) {
    console.error('Error fetching factory-stuff entry:', error)
    return NextResponse.json({ error: 'Failed to fetch factory-stuff entry' }, { status: 500 })
  }
}

// PUT /api/factory-stuff/[id] — update a single factory-stuff entry
export async function PUT(
  request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const session = await requireRole(['admin', 'operator', 'accountant'])
    if (session instanceof NextResponse) return session

    await connectDB()
    const { id } = await params
    const body = await request.json()
    // Normalize date to canonical YYYY-MM-DD
    // (handles dd-mm-yyyy, dd/mm/yyyy, Excel serials, Date objects, etc.)
    if (body.date) body.date = normalizeDate(body.date)

    const updateData: Record<string, unknown> = {}
    for (const field of FIELDS) {
      if (body[field] !== undefined) {
        updateData[field] = body[field]
      }
    }

    const record = await FactoryStuff.findByIdAndUpdate(id, updateData, { new: true })
    if (!record) {
      return NextResponse.json({ error: 'factoryStuff entry not found' }, { status: 404 })
    }

    return NextResponse.json({ factoryStuff: toObject(record) })
  } catch (error) {
    console.error('Error updating factory-stuff entry:', error)
    return NextResponse.json({ error: 'Failed to update factory-stuff entry' }, { status: 500 })
  }
}

// DELETE /api/factory-stuff/[id] — delete a single factory-stuff entry
export async function DELETE(
  _request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const session = await requireRole(['admin', 'operator', 'accountant'])
```

```
    if (session instanceof NextResponse) return session

    await connectDB()
    const { id } = await params

    const record = await FactoryStuff.findByIdAndDelete(id)
    if (!record) {
      return NextResponse.json({ error: 'factoryStuff entry not found' }, { status: 404 })
    }

    return NextResponse.json({ message: 'factoryStuff entry deleted successfully' })
  } catch (error) {
    console.error('Error deleting factory-stuff entry:', error)
    return NextResponse.json({ error: 'Failed to delete factory-stuff entry' }, { status: 500 })
  }
}
```

/api/factory-stuff/bulk-delete/route.ts 61 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB } from '@/lib/db'
import { FactoryStuff } from '@/lib/models'
import { getSession } from '@/lib/auth'

export const dynamic = 'force-dynamic'

// POST /api/factory-stuff/bulk-delete
// Body: { ids: string[] }
// Gated behind admin/operator session — accountants are read-only.
export async function POST(request: Request) {
  try {
    await connectDB()

    const session = await getSession()
    if (!session) {
      return NextResponse.json(
        { error: 'Unauthorized — please log in' },
        { status: 401 }
      )
    }
    if (session.role === 'accountant') {
      return NextResponse.json(
        { error: 'Forbidden — accountants cannot delete factory stuff entries' },
        { status: 403 }
      )
    }

    const body = await request.json()
    const { ids } = body as { ids?: string[] }

    if (!Array.isArray(ids) || ids.length === 0) {
      return NextResponse.json(
        { error: 'ids array is required and must be non-empty' },
        { status: 400 }
      )
    }

    const result = await FactoryStuff.deleteMany({ _id: { $in: ids } })

    if (result.deletedCount === 0) {
      return NextResponse.json(
        { error: 'No matching factory stuff entries found — they may have been already deleted' },
        { status: 404 }
      )
    }

    return NextResponse.json({
      message: `${result.deletedCount} factory stuff entr${result.deletedCount === 1 ? 'y' : 'ies'} deleted`,
      deletedCount: result.deletedCount,
      requestedCount: ids.length,
    })
  } catch (error) {
    console.error('Error bulk-deleting factory stuff entries:', error)
    return NextResponse.json(
      { error: 'Failed to bulk-delete factory stuff entries' },
      { status: 500 }
    )
  }
}
```

/api/factory-stuff/route.ts 51 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB, toObject } from '@/lib/db'
import { FactoryStuff } from '@/lib/models'
import { requireSession, requireRole } from '@/lib/auth'
import { normalizeDate } from '@/lib/date-utils'

// Force dynamic — never cache list responses
export const dynamic = 'force-dynamic'
export const revalidate = 0

export async function GET() {
  try {
    const session = await requireSession()
    if (session instanceof NextResponse) return session

    await connectDB()
    const records = await FactoryStuff.find({}).sort({ date: -1 }).lean()
    return NextResponse.json({ factoryStuffs: records.map(toObject) })
  } catch (error) {
    console.error('Error fetching factory stuff:', error)
    return NextResponse.json({ error: 'Failed to fetch factory stuff' }, { status: 500 })
  }
}

export async function POST(request: Request) {
  try {
    const session = await requireRole(['admin', 'operator', 'accountant'])
    if (session instanceof NextResponse) return session

    await connectDB()
    const body = await request.json()
    // Normalize date to canonical YYYY-MM-DD
    // (handles dd-mm-yyyy, dd/mm/yyyy, Excel serials, Date objects, etc.)
    if (body.date) body.date = normalizeDate(body.date)
    if (!body.date || !body.itemName || !body.amount) {
      return NextResponse.json({ error: 'Date, item name and amount are required' }, { status: 400 })
    }
    const record = await FactoryStuff.create({
      date: body.date,
      itemName: body.itemName,
      quantity: Number(body.quantity) || 0,
      amount: Number(body.amount),
      remarks: body.remarks || '',
    })
    return NextResponse.json({ factoryStuff: toObject(record) }, { status: 201 })
  } catch (error) {
    console.error('Error creating factory stuff:', error)
    return NextResponse.json({ error: 'Failed to create factory stuff' }, { status: 500 })
  }
}
```

/api/hardner/[id]/route.ts 97 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB, toObject } from '@/lib/db'
import { Hardner } from '@/lib/models'
import { requireSession, requireRole } from '@/lib/auth'
import { normalizeDate } from '@/lib/date-utils'

// Force dynamic — never cache individual responses
export const dynamic = 'force-dynamic'
export const revalidate = 0

// Whitelist of updatable fields. Must match HardnerSchema in src/lib/models.ts.
const FIELDS = [
  'date',
  'amount',
] as const

// GET /api/hardner/[id] — fetch a single hardner entry
export async function GET(
  _request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const session = await requireSession()
    if (session instanceof NextResponse) return session

    await connectDB()
    const { id } = await params
    const record = await Hardner.findById(id).lean()
    if (!record) {
      return NextResponse.json({ error: 'hardner entry not found' }, { status: 404 })
    }
    return NextResponse.json({ hardner: toObject(record) })
  } catch (error) {
    console.error('Error fetching hardner entry:', error)
    return NextResponse.json({ error: 'Failed to fetch hardner entry' }, { status: 500 })
  }
}

// PUT /api/hardner/[id] — update a single hardner entry
export async function PUT(
  request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const session = await requireRole(['admin', 'operator', 'accountant'])
    if (session instanceof NextResponse) return session

    await connectDB()
    const { id } = await params
    const body = await request.json()
    // Normalize date to canonical YYYY-MM-DD
    // (handles dd-mm-yyyy, dd/mm/yyyy, Excel serials, Date objects, etc.)
    if (body.date) body.date = normalizeDate(body.date)

    const updateData: Record<string, unknown> = {}
    for (const field of FIELDS) {
      if (body[field] !== undefined) {
        updateData[field] = body[field]
      }
    }

    const record = await Hardner.findByIdAndUpdate(id, updateData, { new: true })
    if (!record) {
      return NextResponse.json({ error: 'hardner entry not found' }, { status: 404 })
    }

    return NextResponse.json({ hardner: toObject(record) })
  } catch (error) {
    console.error('Error updating hardner entry:', error)
    return NextResponse.json({ error: 'Failed to update hardner entry' }, { status: 500 })
  }
}

// DELETE /api/hardner/[id] — delete a single hardner entry
export async function DELETE(
  _request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const session = await requireRole(['admin', 'operator', 'accountant'])
    if (session instanceof NextResponse) return session

    await connectDB()
```

```
    const { id } = await params

    const record = await Hardner.findByIdAndDelete(id)
    if (!record) {
      return NextResponse.json({ error: 'hardner entry not found' }, { status: 404 })
    }

    return NextResponse.json({ message: 'hardner entry deleted successfully' })
  } catch (error) {
    console.error('Error deleting hardner entry:', error)
    return NextResponse.json({ error: 'Failed to delete hardner entry' }, { status: 500 })
  }
}
```

/api/hardner/bulk-delete/route.ts 61 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB } from '@/lib/db'
import { Hardner } from '@/lib/models'
import { getSession } from '@/lib/auth'

export const dynamic = 'force-dynamic'

// POST /api/hardner/bulk-delete
// Body: { ids: string[] }
// Gated behind admin/operator session — accountants are read-only.
export async function POST(request: Request) {
  try {
    await connectDB()

    const session = await getSession()
    if (!session) {
      return NextResponse.json(
        { error: 'Unauthorized — please log in' },
        { status: 401 }
      )
    }
    if (session.role === 'accountant') {
      return NextResponse.json(
        { error: 'Forbidden — accountants cannot delete hardner entries' },
        { status: 403 }
      )
    }

    const body = await request.json()
    const { ids } = body as { ids?: string[] }

    if (!Array.isArray(ids) || ids.length === 0) {
      return NextResponse.json(
        { error: 'ids array is required and must be non-empty' },
        { status: 400 }
      )
    }

    const result = await Hardner.deleteMany({ _id: { $in: ids } })

    if (result.deletedCount === 0) {
      return NextResponse.json(
        { error: 'No matching hardner entries found — they may have been already deleted' },
        { status: 404 }
      )
    }

    return NextResponse.json({
      message: `${result.deletedCount} hardner entr${result.deletedCount === 1 ? 'y' : 'ies'} deleted`,
      deletedCount: result.deletedCount,
      requestedCount: ids.length,
    })
  } catch (error) {
    console.error('Error bulk-deleting hardners:', error)
    return NextResponse.json(
      { error: 'Failed to bulk-delete hardner entries' },
      { status: 500 }
    )
  }
}
```

/api/hardner/route.ts 48 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB, toObject } from '@/lib/db'
import { Hardner } from '@/lib/models'
import { requireSession, requireRole } from '@/lib/auth'
import { normalizeDate } from '@/lib/date-utils'

// Force dynamic — never cache list responses
export const dynamic = 'force-dynamic'
export const revalidate = 0

export async function GET() {
  try {
    const session = await requireSession()
    if (session instanceof NextResponse) return session

    await connectDB()
    const records = await Hardner.find({}).sort({ date: -1 }).lean()
    return NextResponse.json({ hardners: records.map(toObject) })
  } catch (error) {
    console.error('Error fetching hardners:', error)
    return NextResponse.json({ error: 'Failed to fetch hardners' }, { status: 500 })
  }
}

export async function POST(request: Request) {
  try {
    const session = await requireRole(['admin', 'operator', 'accountant'])
    if (session instanceof NextResponse) return session

    await connectDB()
    const body = await request.json()
    // Normalize date to canonical YYYY-MM-DD
    // (handles dd-mm-yyyy, dd/mm/yyyy, Excel serials, Date objects, etc.)
    if (body.date) body.date = normalizeDate(body.date)
    if (!body.date || !body.amount) {
      return NextResponse.json({ error: 'Date and amount are required' }, { status: 400 })
    }
    const record = await Hardner.create({
      date: body.date,
      amount: Number(body.amount),
    })
    return NextResponse.json({ hardner: toObject(record) }, { status: 201 })
  } catch (error) {
    console.error('Error creating hardner:', error)
    return NextResponse.json({ error: 'Failed to create hardner' }, { status: 500 })
  }
}
```

/api/import/route.ts 902 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB, toObject } from '@/lib/db'
import {
  Customer, Production, Stock, Order, Dispatch, Payment, Expense,
  DailySell, CustomerPayment, LabourPayment, TractorPayment,
  DustPurchase, CementPurchase, Hardner, Electricity, FactoryStuff,
} from '@/lib/models'
import { syncStockForDates } from '@/lib/sync-stock'
import { requireRole } from '@/lib/auth'
import { normalizeDate } from '@/lib/date-utils'

// Force dynamic — this route must never be cached/previewed as a static asset.
export const dynamic = 'force-dynamic'

// Max rows per import call — prevents abuse / OOM on huge files
const MAX_IMPORT_ROWS = 5000

// ─── Server-side date normalization ──────────────────────────────
//
// The client (excel-import.tsx) already normalizes dates to YYYY-MM-DD, but
// we re-normalize here as defense-in-depth so a direct API call (or a future
// caller that bypasses the Excel wizard) is still safe.
//
// The normalizeDate() function is imported from src/lib/date-utils.ts so the
// client and server use IDENTICAL parsing logic. Supported formats include:
//   • YYYY-MM-DD, YYYY/MM/DD, YYYY.MM.DD
//   • DD-MM-YYYY, DD/MM/YYYY, DD.MM.YYYY  (Indian default — day first)
//   • DD-MM-YY, DD/MM/YY  (short year → 20XX)
//   • MM-DD-YYYY  (US format — only when first number > 12 forces it)
//   • Datetime strings (time portion stripped)
//   • Excel serial numbers (e.g. 46178)
//   • JavaScript Date objects (uses LOCAL getters — no IST off-by-one)
//   • Fallback: today's date (so a row is still inserted; user can fix manually)
//
// IMPORTANT: We never use native new Date(string) as a fallback because JS
// interprets "05-06-2026" as MM-DD-YYYY (US format), silently swapping
// day/month.

// Normalize every date-like field on a row before validation/insert.
// This keeps the per-module switch statement below simple.
function normalizeRowDates(row: Record<string, unknown>): void {
  if ('date' in row) row.date = normalizeDate(row.date)
  if ('deliveryDate' in row) row.deliveryDate = normalizeDate(row.deliveryDate)
}

// ─── Duplicate detection ─────────────────────────────────────────
//
// Per user request: "jo duplicate ho use skip kre or uska mess de but jo
// duplicate nhi h usko import kre record me" — when an Excel row matches an
// existing record in the DB (by a natural key), SKIP it with a clear message
// but still IMPORT every non-duplicate row.
//
// To keep this fast, for each module we run ONE batched `find()` that
// returns every existing row whose date falls within the dates seen in
// the import payload, then build a Set of natural-key strings. Per-row
// lookup then becomes an O(1) Set.has() check instead of a findOne().
//
// APPEND-ONLY semantics are preserved — we never wipe existing rows.

// Build a stable natural-key string for a row, per module.
function rowKey(module: string, row: Record<string, unknown>): string {
  const s = (v: unknown) => String(v ?? '').trim().toLowerCase()
  switch (module) {
    case 'customers':
      return s(row.mobile)
    case 'production':
      // One production entry per date — same date = duplicate (matches stock semantics)
      return s(row.date)
    case 'stock':
      // One stock entry per date — same date = duplicate
      return s(row.date)
    case 'dailySell':
      return `${s(row.date)}|${s(row.customerName)}|${s(row.amount)}`
    case 'customerPayment':
      return `${s(row.date)}|${s(row.name)}|${s(row.amount)}`
    case 'labourPayment':
      return `${s(row.date)}|${s(row.name)}|${s(row.amount)}`
    case 'tractorPayment':
      return `${s(row.date)}|${s(row.vendorName)}|${s(row.quantityTon)}|${s(row.rate)}`
    case 'dustPurchase':
      return `${s(row.date)}|${s(row.vendorName)}|${s(row.quantity)}|${s(row.rate)}`
    case 'cementPurchase':
      return `${s(row.date)}|${s(row.vendorName)}|${s(row.itemName)}|${s(row.quantity)}|${s(row.rate)}`
```

```
    case 'hardner':
      // Only date+amount available — same date AND same amount = duplicate
      return `${s(row.date)}|${s(row.amount)}`
    case 'electricity':
      return `${s(row.date)}|${s(row.name)}|${s(row.work)}|${s(row.amount)}`
    case 'factoryStuff':
      return `${s(row.date)}|${s(row.itemName)}|${s(row.amount)}`
    case 'orders':
      return `${s(row.customerId)}|${s(row.brickType)}|${s(row.quantity)}|${s(row.rate)}|${s(row.deliveryDate || row.date)}`
    case 'dispatch':
      return `${s(row.customerId)}|${s(row.truckNumber)}|${s(row.brickType)}|${s(row.quantity)}|${s(row.date)}`
    case 'payments':
      return `${s(row.customerId)}|${s(row.paymentType)}|${s(row.amount)}|${s(row.date)}`
    case 'expenses':
      return `${s(row.category)}|${s(row.amount)}|${s(row.date)}|${s(row.description)}`
    default:
      return ''
  }
}

// Map a DB record back into the same natural-key shape as a row.
// IMPORTANT: this MUST mirror rowKey() exactly — if rowKey uses just `date`
// for production, dbKey must also use just `date` for production. A mismatch
// here means existingKeys will contain keys like "2026-06-21||" while the
// incoming row key is "2026-06-21", so the duplicate check NEVER matches and
// duplicates get imported. This was the root cause of duplicate production
// entries being imported.
function dbKey(module: string, doc: Record<string, unknown>): string {
  const s = (v: unknown) => String(v ?? '').trim().toLowerCase()
  switch (module) {
    case 'customers':
      return s(doc.mobile)
    case 'production':
      // One production entry per date — matches rowKey('production')
      return s(doc.date)
    case 'stock':
      return s(doc.date)
    case 'dailySell':
      return `${s(doc.date)}|${s(doc.customerName)}|${s(doc.amount)}`
    case 'customerPayment':
      return `${s(doc.date)}|${s(doc.name)}|${s(doc.amount)}`
    case 'labourPayment':
      return `${s(doc.date)}|${s(doc.name)}|${s(doc.amount)}`
    case 'tractorPayment':
      return `${s(doc.date)}|${s(doc.vendorName)}|${s(doc.quantityTon)}|${s(doc.rate)}`
    case 'dustPurchase':
      return `${s(doc.date)}|${s(doc.vendorName)}|${s(doc.quantity)}|${s(doc.rate)}`
    case 'cementPurchase':
      return `${s(doc.date)}|${s(doc.vendorName)}|${s(doc.itemName)}|${s(doc.quantity)}|${s(doc.rate)}`
    case 'hardner':
      return `${s(doc.date)}|${s(doc.amount)}`
    case 'electricity':
      return `${s(doc.date)}|${s(doc.name)}|${s(doc.work)}|${s(doc.amount)}`
    case 'factoryStuff':
      return `${s(doc.date)}|${s(doc.itemName)}|${s(doc.amount)}`
    case 'orders':
      return `${s(doc.customerId)}|${s(doc.brickType)}|${s(doc.quantity)}|${s(doc.rate)}|${s(doc.deliveryDate)}`
    case 'dispatch':
      return `${s(doc.customerId)}|${s(doc.truckNumber)}|${s(doc.brickType)}|${s(doc.quantity)}|${s(doc.date)}`
    case 'payments':
      return `${s(doc.customerId)}|${s(doc.paymentType)}|${s(doc.amount)}|${s(doc.date)}`
    case 'expenses':
      return `${s(doc.category)}|${s(doc.amount)}|${s(doc.date)}|${s(doc.description)}`
    default:
      return ''
  }
}

// Pre-fetch existing DB records that could collide with any row in `data`.
// Returns a Set of natural-key strings.
async function buildExistingKeys(module: string, data: Record<string, unknown>[]): Promise<Set<string>> {
  const keys = new Set<string>()
  if (data.length === 0) return keys

  // Collect unique dates and customerIds from the import payload so we can
  // narrow the DB query. This keeps the result set small.
  const dates = new Set<string>()
  const mobiles = new Set<string>()
  const customerIds = new Set<string>()
  for (const row of data) {
    const d = String(row.date ?? '').split('T')[0]
    if (d) dates.add(d)
    const dd = String(row.deliveryDate ?? '').split('T')[0]
    if (dd) dates.add(dd)
    const m = String(row.mobile ?? '').trim()
    if (m) mobiles.add(m)
```

```
    const cid = String(row.customerId ?? '').trim()
    if (cid) customerIds.add(cid)
  }

  let docs: Record<string, unknown>[] = []

  switch (module) {
    case 'customers':
      docs = mobiles.size > 0
        ? await Customer.find({ mobile: { $in: Array.from(mobiles) } }).lean() as any
        : []
      break
    case 'production':
      docs = dates.size > 0
        ? await Production.find({ date: { $in: Array.from(dates) } }).lean() as any
        : []
      break
    case 'stock':
      docs = dates.size > 0
        ? await Stock.find({ date: { $in: Array.from(dates) } }).lean() as any
        : []
      break
    case 'dailySell':
      docs = dates.size > 0
        ? await DailySell.find({ date: { $in: Array.from(dates) } }).lean() as any
        : []
      break
    case 'customerPayment':
      docs = dates.size > 0
        ? await CustomerPayment.find({ date: { $in: Array.from(dates) } }).lean() as any
        : []
      break
    case 'labourPayment':
      docs = dates.size > 0
        ? await LabourPayment.find({ date: { $in: Array.from(dates) } }).lean() as any
        : []
      break
    case 'tractorPayment':
      docs = dates.size > 0
        ? await TractorPayment.find({ date: { $in: Array.from(dates) } }).lean() as any
        : []
      break
    case 'dustPurchase':
      docs = dates.size > 0
        ? await DustPurchase.find({ date: { $in: Array.from(dates) } }).lean() as any
        : []
      break
    case 'cementPurchase':
      docs = dates.size > 0
        ? await CementPurchase.find({ date: { $in: Array.from(dates) } }).lean() as any
        : []
      break
    case 'hardner':
      docs = dates.size > 0
        ? await Hardner.find({ date: { $in: Array.from(dates) } }).lean() as any
        : []
      break
    case 'electricity':
      docs = dates.size > 0
        ? await Electricity.find({ date: { $in: Array.from(dates) } }).lean() as any
        : []
      break
    case 'factoryStuff':
      docs = dates.size > 0
        ? await FactoryStuff.find({ date: { $in: Array.from(dates) } }).lean() as any
        : []
      break
    case 'orders':
      docs = customerIds.size > 0
        ? await Order.find({ customerId: { $in: Array.from(customerIds) } }).lean() as any
        : []
      break
    case 'dispatch':
      docs = customerIds.size > 0
        ? await Dispatch.find({ customerId: { $in: Array.from(customerIds) } }).lean() as any
        : []
      break
    case 'payments':
      docs = customerIds.size > 0
        ? await Payment.find({ customerId: { $in: Array.from(customerIds) } }).lean() as any
        : []
      break
    case 'expenses':
      // Expenses dedupe by category+amount+date — narrow by date
      docs = dates.size > 0
        ? await Expense.find({ date: { $in: Array.from(dates) } }).lean() as any
```

```
        : []
      break
  }

  for (const d of docs) {
    const k = dbKey(module, d as Record<string, unknown>)
    if (k) keys.add(k)
  }
  return keys
}

export async function POST(request: Request) {
  try {
    // Admin/operator only — bulk import is a privileged write operation
    const session = await requireRole(['admin', 'operator'])
    if (session instanceof NextResponse) return session

    await connectDB()
    const body = await request.json()
    const { module, data } = body

    if (!module || !data || !Array.isArray(data)) {
      return NextResponse.json({ error: 'Module and data array are required' }, { status: 400 })
    }

    // Cap import size to prevent abuse / OOM
    if (data.length > MAX_IMPORT_ROWS) {
      return NextResponse.json(
        { error: `Too many rows (${data.length}). Maximum ${MAX_IMPORT_ROWS} rows per import. Split your file and try again.` },
        { status: 413 }
      )
    }

    // ── Pre-fetch existing keys (1 DB query for the whole batch) ──────────
    const existingKeys = await buildExistingKeys(module, data as Record<string, unknown>[])
    // Track keys we've already seen WITHIN this batch so a duplicate Excel
    // row doesn't get inserted twice in the same import run.
    const seenInBatch = new Set<string>()

    let imported = 0
    let skipped = 0
    let duplicatesSkipped = 0
    const errors: string[] = []
    const skippedReasons: string[] = []

    // ── Bulk insert optimization ──────────────────────────────────────────
    // Instead of calling `await Model.create(doc)` for each row (N DB round
    // trips), we collect ALL valid documents into a `toInsert` array and do a
    // single `Model.insertMany(toInsert)` call after the loop. This makes the
    // import dramatically faster — 48 rows that previously took 8-10 seconds
    // (timing out on Vercel Hobby's 10s limit) now take under 1 second.
    //
    // `rowIndexByDoc` maps each document back to its original Excel row index
    // so we can report per-row errors if insertMany throws.
    const toInsert: any[] = []
    const rowIndexByDoc: number[] = []

    for (let i = 0; i < data.length; i++) {
      try {
        const row = data[i]

        // Normalize date fields BEFORE duplicate check / validation / insert
        // so DD-MM-YYYY, DD/MM/YYYY, DD.MM.YYYY, datetime strings, and Excel
        // serial numbers are all converted to canonical YYYY-MM-DD.
        normalizeRowDates(row as Record<string, unknown>)

        // ── Duplicate check (against DB AND within this batch) ──────────
        // Duplicates are pushed to the `errors` array (not just skippedReasons)
        // so they appear in the RED error popup with a clear "Duplicate data
        // found" message, exactly as the user requested. The row is still
        // counted in duplicatesSkipped for the summary card.
        const key = rowKey(module, row as Record<string, unknown>)
        if (key) {
          if (existingKeys.has(key)) {
            skipped++
            duplicatesSkipped++
            const label = duplicateRowLabel(module, row as Record<string, unknown>)
            errors.push(`Row ${i + 1}: Duplicate data found — ${label} already exists in records`)
            continue
          }
          if (seenInBatch.has(key)) {
            skipped++
            duplicatesSkipped++
            const label = duplicateRowLabel(module, row as Record<string, unknown>)
            errors.push(`Row ${i + 1}: Duplicate data found — ${label} appears more than once in this Excel file`)
            continue
```

```
      }
      seenInBatch.add(key)
    }

    switch (module) {
      // ── Customers ──────────────────────────────────────────
      case 'customers': {
        if (!row.name || !row.mobile) {
          errors.push(`Row ${i + 1}: Name and mobile are required`)
          skipped++
          continue
        }
        toInsert.push({
          name: String(row.name).trim(),
          mobile: String(row.mobile).trim(),
          gstNumber: row.gstNumber || '',
          address: row.address || '',
          creditLimit: Number(row.creditLimit) || 0,
        })
        rowIndexByDoc.push(i)
        break
      }

      // ── Production ─────────────────────────────────────────
      case 'production': {
        if (!row.date) {
          errors.push(`Row ${i + 1}: Date is required`)
          skipped++
          continue
        }
        toInsert.push({
          date: String(row.date),
          customerId: row.customerId || null,
          cement: Number(row.cement) || 0,
          zigZagGrey80: Number(row.zigZagGrey80) || 0,
          zigZagRed80: Number(row.zigZagRed80) || 0,
          zigZagYellow80: Number(row.zigZagYellow80) || 0,
          zigZagGrey60: Number(row.zigZagGrey60) || 0,
          zigZagRed60: Number(row.zigZagRed60) || 0,
          zigZagYellow60: Number(row.zigZagYellow60) || 0,
          curveStone: Number(row.curveStone) || 0,
          chequreTile: Number(row.chequreTile) || 0,
          dumbleGrey80: Number(row.dumbleGrey80) || 0,
          dumbleRed80: Number(row.dumbleRed80) || 0,
          dumbleYellow80: Number(row.dumbleYellow80) || 0,
          transportationCharge: Number(row.transportationCharge) || 0,
          remarks: String(row.remarks || ''),
        })
        rowIndexByDoc.push(i)
        break
      }

      // ── Stock ──────────────────────────────────────────────
      case 'stock': {
        if (!row.date) {
          errors.push(`Row ${i + 1}: Date is required`)
          skipped++
          continue
        }
        toInsert.push({
          date: String(row.date),
          cement: Number(row.cement) || 0,
          zigZagGrey80: Number(row.zigZagGrey80) || 0,
          zigZagRed80: Number(row.zigZagRed80) || 0,
          zigZagYellow80: Number(row.zigZagYellow80) || 0,
          zigZagGrey60: Number(row.zigZagGrey60) || 0,
          zigZagRed60: Number(row.zigZagRed60) || 0,
          zigZagYellow60: Number(row.zigZagYellow60) || 0,
          chequreTile: Number(row.chequreTile) || 0,
          curveStone: Number(row.curveStone) || 0,
          dumbleGrey80: Number(row.dumbleGrey80) || 0,
          dumbleRed80: Number(row.dumbleRed80) || 0,
          dumbleYellow80: Number(row.dumbleYellow80) || 0,
        })
        rowIndexByDoc.push(i)
        break
      }

      // ── Daily Sell ─────────────────────────────────────────
      case 'dailySell': {
        // Mandatory: Date, Customer Name, Address, Contact.
        // Everything else (Product, Qty, Rate, Amount, Transporter,
        // T. Fair, Received, Pending, Remarks) is optional.
        if (!row.date || !row.customerName || !row.address || !row.contactNumber) {
          errors.push(`Row ${i + 1}: Date, customer name, address, and contact are required`)
          skipped++
```

```
            continue
          }
          const dsAmount = Number(row.amount) || 0
          const dsReceived = Number(row.receivedAmount) || 0
          const dsPending = row.pendingAmount != null && row.pendingAmount !== ''
            ? Number(row.pendingAmount)
            : Math.max(0, dsAmount - dsReceived)
          toInsert.push({
            date: String(row.date),
            customerName: String(row.customerName),
            address: String(row.address),
            contactNumber: String(row.contactNumber),
            product: String(row.product || ''),
            quantity: Number(row.quantity) || 0,
            rate: Number(row.rate) || 0,
            amount: dsAmount,
            transporterName: String(row.transporterName || ''),
            transporterFair: Number(row.transporterFair) || 0,
            receivedAmount: dsReceived,
            pendingAmount: dsPending,
            remarks: String(row.remarks || ''),
          })
          rowIndexByDoc.push(i)
          break
        }

        // ── Customer Payment ──────────────────────────────
        case 'customerPayment': {
          if (!row.date || !row.name || !row.amount) {
            errors.push(`Row ${i + 1}: Date, name, and amount are required`)
            skipped++
            continue
          }
          toInsert.push({
            date: String(row.date),
            name: String(row.name),
            address: String(row.address || ''),
            amount: Number(row.amount),
            remarks: String(row.remarks || ''),
          })
          rowIndexByDoc.push(i)
          break
        }

        // ── Labour Payment ────────────────────────────────
        case 'labourPayment': {
          if (!row.date || !row.name || !row.amount) {
            errors.push(`Row ${i + 1}: Date, name, and amount are required`)
            skipped++
            continue
          }
          toInsert.push({
            date: String(row.date),
            name: String(row.name),
            address: String(row.address || ''),
            amount: Number(row.amount),
            remarks: String(row.remarks || ''),
          })
          rowIndexByDoc.push(i)
          break
        }

        // ── Tractor Payment ───────────────────────────────
        case 'tractorPayment': {
          if (!row.date || !row.vendorName || !row.quantityTon || !row.rate) {
            errors.push(`Row ${i + 1}: Date, vendor, quantity, and rate are required`)
            skipped++
            continue
          }
          const qty = Number(row.quantityTon)
          const rate = Number(row.rate)
          const totalAmount = Number(row.totalAmount) || qty * rate
          const paidAmount = Number(row.paidAmount) || 0
          toInsert.push({
            date: String(row.date),
            vendorName: String(row.vendorName),
            quantityTon: qty,
            rate,
            totalAmount,
            paidAmount,
            remainingAmount: Number(row.remainingAmount) || (totalAmount - paidAmount),
            remarks: String(row.remarks || ''),
          })
          rowIndexByDoc.push(i)
          break
        }
```

```
      // ── Dust Purchase ──────────────────────────────────────
      case 'dustPurchase': {
        if (!row.date || !row.vendorName || !row.quantity || !row.rate) {
          errors.push(`Row ${i + 1}: Date, vendor, quantity, and rate are required`)
          skipped++
          continue
        }
        const qty = Number(row.quantity)
        const rate = Number(row.rate)
        const totalAmount = Number(row.totalAmount) || qty * rate
        const paidAmount = Number(row.paidAmount) || 0
        toInsert.push({
          date: String(row.date),
          vendorName: String(row.vendorName),
          cementName: String(row.cementName || ''),
          quantity: qty,
          rate,
          totalAmount,
          paidAmount,
          transportationCharge: Number(row.transportationCharge) || 0,
          gst: Number(row.gst) || 0,
          remarks: String(row.remarks || ''),
        })
        rowIndexByDoc.push(i)
        break
      }

      // ── Cement Purchase ────────────────────────────────────
      case 'cementPurchase': {
        if (!row.date || !row.vendorName || !row.quantity || !row.rate) {
          errors.push(`Row ${i + 1}: Date, vendor, quantity, and rate are required`)
          skipped++
          continue
        }
        const qty = Number(row.quantity)
        const rate = Number(row.rate)
        const totalAmount = Number(row.totalAmount) || qty * rate
        const paidAmount = Number(row.paidAmount) || 0
        toInsert.push({
          date: String(row.date),
          vendorName: String(row.vendorName),
          itemName: String(row.itemName || ''),
          quantity: qty,
          rate,
          totalAmount,
          paidAmount,
          transportationCharge: Number(row.transportationCharge) || 0,
          gst: Number(row.gst) || 0,
          remarks: String(row.remarks || ''),
        })
        rowIndexByDoc.push(i)
        break
      }

      // ── Hardner ────────────────────────────────────────────
      case 'hardner': {
        if (!row.date || !row.amount) {
          errors.push(`Row ${i + 1}: Date and amount are required`)
          skipped++
          continue
        }
        toInsert.push({
          date: String(row.date),
          amount: Number(row.amount),
        })
        rowIndexByDoc.push(i)
        break
      }

      // ── Electricity ────────────────────────────────────────
      case 'electricity': {
        if (!row.date || !row.amount) {
          errors.push(`Row ${i + 1}: Date and amount are required`)
          skipped++
          continue
        }
        toInsert.push({
          date: String(row.date),
          name: String(row.name || ''),
          work: String(row.work || ''),
          amount: Number(row.amount),
          remarks: String(row.remarks || ''),
        })
        rowIndexByDoc.push(i)
        break
```

```
        }

        // ── Factory Stuff ──────────────────────────────────────
        case 'factoryStuff': {
          if (!row.date || !row.itemName || !row.amount) {
            errors.push(`Row ${i + 1}: Date, item name, and amount are required`)
            skipped++
            continue
          }
          toInsert.push({
            date: String(row.date),
            itemName: String(row.itemName),
            quantity: Number(row.quantity) || 0,
            amount: Number(row.amount),
            remarks: String(row.remarks || ''),
          })
          rowIndexByDoc.push(i)
          break
        }

        // ── Legacy modules (Orders/Dispatch/Payments/Expenses) ──
        case 'orders': {
          if (!row.customerId || !row.brickType || !row.quantity || !row.rate) {
            errors.push(`Row ${i + 1}: Customer, brick type, quantity, and rate are required`)
            skipped++
            continue
          }
          // Defer orderNumber generation to the bulk phase below — we just
          // collect the row here. The bulk phase runs ONE countDocuments
          // and ONE Company.findOne for the whole batch, then issues a
          // single insertMany with sequential orderNumbers computed in
          // memory. This eliminates the previous N × 3 query pattern.
          toInsert.push({
            customerId: row.customerId,
            brickType: row.brickType,
            quantity: Number(row.quantity),
            rate: Number(row.rate),
            amount: Number(row.amount) || Number(row.quantity) * Number(row.rate),
            deliveryDate: row.deliveryDate || row.date || new Date().toISOString().split('T')[0],
            status: row.status || 'Pending',
          })
          rowIndexByDoc.push(i)
          break
        }

        case 'dispatch': {
          if (!row.customerId || !row.truckNumber || !row.quantity || !row.brickType || !row.date) {
            errors.push(`Row ${i + 1}: Customer, truck, quantity, brick type, and date are required`)
            skipped++
            continue
          }
          // Same deferral as orders — orderNumber generation happens in
          // the bulk phase below.
          toInsert.push({
            customerId: row.customerId,
            orderId: row.orderId || null,
            truckNumber: row.truckNumber,
            driverName: row.driverName || '',
            quantity: Number(row.quantity),
            brickType: row.brickType,
            date: row.date,
          })
          rowIndexByDoc.push(i)
          break
        }

        case 'payments': {
          if (!row.customerId || !row.paymentType || !row.amount || !row.date) {
            errors.push(`Row ${i + 1}: Customer, payment type, amount, and date are required`)
            skipped++
            continue
          }
          toInsert.push({
            customerId: row.customerId,
            paymentType: row.paymentType,
            amount: Number(row.amount),
            date: row.date,
            remarks: row.remarks || '',
          })
          rowIndexByDoc.push(i)
          break
        }

        case 'expenses': {
          if (!row.category || !row.amount || !row.date) {
            errors.push(`Row ${i + 1}: Category, amount, and date are required`)
```

```
            skipped++
            continue
          }
          toInsert.push({
            category: row.category,
            amount: Number(row.amount),
            date: row.date,
            description: row.description || '',
          })
          rowIndexByDoc.push(i)
          break
        }

        default:
          errors.push(`Unknown module: ${module}`)
      }
    } catch (err) {
      errors.push(`Row ${i + 1}: ${err instanceof Error ? err.message : 'Import failed'}`)
      skipped++
    }
  }

  // ── Bulk insert all valid documents in ONE DB call ──────────────
  // This is the key optimization: instead of N separate `Model.create()`
  // calls (each a DB round trip), we do a single `Model.insertMany()`.
  //
  // For `orders` and `dispatch`, we need sequential orderNumber/dispatchNumber
  // values. We do that efficiently here: ONE countDocuments + ONE
  // Company.findOne for the whole batch, then compute sequential numbers
  // in memory and insertMany in one call. Previously this was done
  // per-row inside the loop (N × 3 queries).
  if (toInsert.length > 0) {
    try {
      const Model = getModelForModule(module)

      // ── Sequential numbering for orders & dispatch ──────────────
      // Pre-compute orderNumber/dispatchNumber for each doc in memory
      // using a single count + single Company lookup.
      if (module === 'orders' || module === 'dispatch') {
        const { Company } = await import('@/lib/models')
        const [existingCount, company] = await Promise.all([
          module === 'orders' ? Order.countDocuments({}) : Dispatch.countDocuments({}),
          Company.findOne({}),
        ])
        const prefix =
          module === 'orders'
            ? (company?.orderPrefix || 'ORD')
            : (company?.dispatchPrefix || 'DSP')
        const numField = module === 'orders' ? 'orderNumber' : 'dispatchNumber'
        toInsert.forEach((doc, idx) => {
          doc[numField] = `${prefix}-${String(existingCount + idx + 1).padStart(4, '0')}`
        })
      }

      if (Model) {
        // `ordered: false` lets MongoDB insert all valid docs even if some
        // fail validation — we collect per-doc errors from the result.
        const result = await Model.insertMany(toInsert, { ordered: false, rawResult: false })
        imported += Array.isArray(result) ? result.length : 0
      }
    } catch (err: any) {
      // insertMany with `ordered: false` throws a BulkWriteError containing
      // `insertErrors` and `result` with details about which docs failed.
      if (err?.name === 'BulkWriteError' || err?.code === 11000) {
        // Some docs may have been inserted before the error — count them.
        const insertedCount = err?.insertedDocs?.length ?? err?.result?.nInserted ?? 0
        imported += insertedCount
        // Walk the writeErrors to map each failure back to its row index.
        const writeErrors = err?.result?.writeErrors || err?.writeErrors || []
        for (const we of writeErrors) {
          const docIdx = we.index
          const rowIdx = rowIndexByDoc[docIdx]
          if (rowIdx !== undefined) {
            const isDup = we.code === 11000
            const label = duplicateRowLabel(module, data[rowIdx] as Record<string, unknown>)
            if (isDup) {
              errors.push(`Row ${rowIdx + 1}: Duplicate data found — ${label}`)
              duplicatesSkipped++
              skipped++
            } else {
              errors.push(`Row ${rowIdx + 1}: ${we.errmsg || we.message || 'Insert failed'}`)
              skipped++
            }
          }
        }
      } else {
```

```
            // Generic error — record it once with the count.
            errors.push(`Bulk insert failed: ${err instanceof Error ? err.message : 'Unknown error'}. ${toInsert.length} row(s) were not imported.`)
            skipped += toInsert.length
          }
        }
      }

      // Merge errors and skippedReasons so the UI can show every reason a row
      // was not imported (validation error OR duplicate OR anything else).
      const allReasons = [...errors, ...skippedReasons]

      // Auto-sync Stock Overview from Production — when production rows are
      // imported, the Stock module must reflect the latest daily totals
      // automatically. We collect every date touched by THIS import and
      // re-aggregate the matching Stock snapshot for each.
      if (module === 'production' && imported > 0) {
        const touchedDates = data
          .map((row) => String(row.date || '').split('T')[0])
          .filter(Boolean)
        if (touchedDates.length > 0) {
          try {
            await syncStockForDates(touchedDates)
          } catch (err) {
            console.error('[import] Stock sync failed:', err)
            // Don't fail the import — production rows were already saved.
            // Just append a soft warning to the error list.
            allReasons.push('Warning: production rows imported, but Stock Overview auto-sync failed. Check server logs.')
          }
        }
      }

      return NextResponse.json({
        success: true,
        imported,
        skipped,
        duplicatesSkipped,
        total: data.length,
        errors: allReasons.length > 0 ? allReasons.slice(0, 50) : undefined,
      })
    } catch (error) {
      console.error('Error importing data:', error)
      return NextResponse.json({ error: 'Failed to import data' }, { status: 500 })
    }
}

// Map module name to its Mongoose model. Used by the bulk-insert path so we
// can do a single `Model.insertMany(toInsert)` call instead of N creates.
function getModelForModule(module: string) {
  switch (module) {
    case 'customers': return Customer
    case 'production': return Production
    case 'stock': return Stock
    case 'dailySell': return DailySell
    case 'customerPayment': return CustomerPayment
    case 'labourPayment': return LabourPayment
    case 'tractorPayment': return TractorPayment
    case 'dustPurchase': return DustPurchase
    case 'cementPurchase': return CementPurchase
    case 'hardner': return Hardner
    case 'electricity': return Electricity
    case 'factoryStuff': return FactoryStuff
    case 'payments': return Payment
    case 'expenses': return Expense
    // Orders and dispatch now use the bulk-insert path too — their
    // sequential orderNumber/dispatchNumber is pre-computed in memory
    // before insertMany is called. See the bulk phase in POST handler.
    case 'orders': return Order
    case 'dispatch': return Dispatch
    default: return null
  }
}

// Human-readable label for a duplicate row, used in the skip reason.
function duplicateRowLabel(module: string, row: Record<string, unknown>): string {
  switch (module) {
    case 'customers':
      return `customer "${row.name}" (mobile ${row.mobile})`
    case 'production':
      return `production entry on ${row.date}`
    case 'stock':
      return `stock entry for ${row.date}`
    case 'dailySell':
      return `daily sell for "${row.customerName}" on ${row.date} (₹${row.amount})`
    case 'customerPayment':
      return `payment by "${row.name}" on ${row.date} (₹${row.amount})`
    case 'labourPayment':
```

```
      return `payment to "${row.name}" on ${row.date} (₹${row.amount})`
    case 'tractorPayment':
      return `tractor payment to "${row.vendorName}" on ${row.date}`
    case 'dustPurchase':
      return `dust purchase from "${row.vendorName}" on ${row.date}`
    case 'cementPurchase':
      return `cement purchase from "${row.vendorName}" on ${row.date}`
    case 'hardner':
      return `hardner entry on ${row.date} (₹${row.amount})`
    case 'electricity':
      return `electricity entry on ${row.date} (₹${row.amount})`
    case 'factoryStuff':
      return `factory stuff "${row.itemName}" on ${row.date} (₹${row.amount})`
    case 'orders':
      return `order for customer ${row.customerId} on ${row.deliveryDate || row.date}`
    case 'dispatch':
      return `dispatch to customer ${row.customerId} on ${row.date}`
    case 'payments':
      return `payment for customer ${row.customerId} on ${row.date}`
    case 'expenses':
      return `expense "${row.category}" on ${row.date} (₹${row.amount})`
    default:
      return 'record'
  }
}

// GET endpoint — return all available modules + their required fields for import template
export async function GET() {
  const modules = [
    { id: 'customers', label: 'Customers', fields: ['name', 'mobile', 'address', 'gstNumber', 'creditLimit'] },
    { id: 'production', label: 'Production', fields: ['date', 'cement', 'zigZagGrey80', 'zigZagRed80', 'zigZagYellow80', 'zigZagGrey60', 'zigZagRed60',
    { id: 'stock', label: 'Stock', fields: ['date', 'cement', 'zigZagGrey80', 'zigZagRed80', 'zigZagYellow80', 'zigZagGrey60', 'zigZagRed60', 'zigZagYel
    { id: 'dailySell', label: 'Daily Sell', fields: ['date', 'customerName', 'address', 'contactNumber', 'product', 'quantity', 'rate', 'amount', 'transp
    { id: 'customerPayment', label: 'Customer Payment', fields: ['date', 'name', 'address', 'amount', 'remarks'] },
    { id: 'labourPayment', label: 'Labour Payment', fields: ['date', 'name', 'address', 'amount', 'remarks'] },
    { id: 'tractorPayment', label: 'Tractor Payment', fields: ['date', 'vendorName', 'quantityTon', 'rate', 'totalAmount', 'paidAmount', 'remainingAmount
    { id: 'dustPurchase', label: 'Dust Purchase', fields: ['date', 'vendorName', 'cementName', 'quantity', 'rate', 'totalAmount', 'paidAmount', 'transpo
    { id: 'cementPurchase', label: 'Cement Purchase', fields: ['date', 'vendorName', 'itemName', 'quantity', 'rate', 'totalAmount', 'paidAmount', 'transp
    { id: 'hardner', label: 'Hardner', fields: ['date', 'amount'] },
    { id: 'electricity', label: 'Electricity', fields: ['date', 'name', 'work', 'amount', 'remarks'] },
    { id: 'factoryStuff', label: 'Factory Stuff', fields: ['date', 'itemName', 'quantity', 'amount', 'remarks'] },
  ]
  return NextResponse.json({ modules })
}
```

/api/labour-payment/[id]/route.ts 100 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB, toObject } from '@/lib/db'
import { LabourPayment } from '@/lib/models'
import { requireSession, requireRole } from '@/lib/auth'
import { normalizeDate } from '@/lib/date-utils'

// Force dynamic — never cache individual responses
export const dynamic = 'force-dynamic'
export const revalidate = 0

// Whitelist of updatable fields. Must match LabourPaymentSchema in src/lib/models.ts.
const FIELDS = [
  'date',
  'name',
  'address',
  'amount',
  'remarks',
] as const

// GET /api/labour-payment/[id] — fetch a single labour-payment entry
export async function GET(
  _request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const session = await requireSession()
    if (session instanceof NextResponse) return session

    await connectDB()
    const { id } = await params
    const record = await LabourPayment.findById(id).lean()
    if (!record) {
      return NextResponse.json({ error: 'labourPayment entry not found' }, { status: 404 })
    }
    return NextResponse.json({ labourPayment: toObject(record) })
  } catch (error) {
    console.error('Error fetching labour-payment entry:', error)
    return NextResponse.json({ error: 'Failed to fetch labour-payment entry' }, { status: 500 })
  }
}

// PUT /api/labour-payment/[id] — update a single labour-payment entry
export async function PUT(
  request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const session = await requireRole(['admin', 'operator', 'accountant'])
    if (session instanceof NextResponse) return session

    await connectDB()
    const { id } = await params
    const body = await request.json()
    // Normalize date to canonical YYYY-MM-DD
    // (handles dd-mm-yyyy, dd/mm/yyyy, Excel serials, Date objects, etc.)
    if (body.date) body.date = normalizeDate(body.date)

    const updateData: Record<string, unknown> = {}
    for (const field of FIELDS) {
      if (body[field] !== undefined) {
        updateData[field] = body[field]
      }
    }

    const record = await LabourPayment.findByIdAndUpdate(id, updateData, { new: true })
    if (!record) {
      return NextResponse.json({ error: 'labourPayment entry not found' }, { status: 404 })
    }

    return NextResponse.json({ labourPayment: toObject(record) })
  } catch (error) {
    console.error('Error updating labour-payment entry:', error)
    return NextResponse.json({ error: 'Failed to update labour-payment entry' }, { status: 500 })
  }
}

// DELETE /api/labour-payment/[id] — delete a single labour-payment entry
export async function DELETE(
  _request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const session = await requireRole(['admin', 'operator', 'accountant'])
```

```
    if (session instanceof NextResponse) return session

    await connectDB()
    const { id } = await params

    const record = await LabourPayment.findByIdAndDelete(id)
    if (!record) {
      return NextResponse.json({ error: 'labourPayment entry not found' }, { status: 404 })
    }

    return NextResponse.json({ message: 'labourPayment entry deleted successfully' })
  } catch (error) {
    console.error('Error deleting labour-payment entry:', error)
    return NextResponse.json({ error: 'Failed to delete labour-payment entry' }, { status: 500 })
  }
}
```

/api/labour-payment/bulk-delete/route.ts 63 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB } from '@/lib/db'
import { LabourPayment } from '@/lib/models'
import { getSession } from '@/lib/auth'

// Force dynamic — never cache
export const dynamic = 'force-dynamic'

// POST /api/labour-payment/bulk-delete
// Body: { ids: string[] }
// Deletes labour payment entries whose `_id` is in `ids`.
// Gated behind admin/operator session — accountants are read-only.
export async function POST(request: Request) {
  try {
    await connectDB()

    const session = await getSession()
    if (!session) {
      return NextResponse.json(
        { error: 'Unauthorized — please log in' },
        { status: 401 }
      )
    }
    if (session.role === 'accountant') {
      return NextResponse.json(
        { error: 'Forbidden — accountants cannot delete labour payment entries' },
        { status: 403 }
      )
    }

    const body = await request.json()
    const { ids } = body as { ids?: string[] }

    if (!Array.isArray(ids) || ids.length === 0) {
      return NextResponse.json(
        { error: 'ids array is required and must be non-empty' },
        { status: 400 }
      )
    }

    const result = await LabourPayment.deleteMany({ _id: { $in: ids } })

    if (result.deletedCount === 0) {
      return NextResponse.json(
        { error: 'No matching labour payment entries found — they may have been already deleted' },
        { status: 404 }
      )
    }

    return NextResponse.json({
      message: `${result.deletedCount} labour payment entr${result.deletedCount === 1 ? 'y' : 'ies'} deleted`,
      deletedCount: result.deletedCount,
      requestedCount: ids.length,
    })
  } catch (error) {
    console.error('Error bulk-deleting labour payments:', error)
    return NextResponse.json(
      { error: 'Failed to bulk-delete labour payment entries' },
      { status: 500 }
    )
  }
}
```

/api/labour-payment/route.ts 51 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB, toObject } from '@/lib/db'
import { LabourPayment } from '@/lib/models'
import { requireSession, requireRole } from '@/lib/auth'
import { normalizeDate } from '@/lib/date-utils'

// Force dynamic — never cache list responses
export const dynamic = 'force-dynamic'
export const revalidate = 0

export async function GET() {
  try {
    const session = await requireSession()
    if (session instanceof NextResponse) return session

    await connectDB()
    const records = await LabourPayment.find({}).sort({ date: -1 }).lean()
    return NextResponse.json({ labourPayments: records.map(toObject) })
  } catch (error) {
    console.error('Error fetching labour payments:', error)
    return NextResponse.json({ error: 'Failed to fetch labour payments' }, { status: 500 })
  }
}

export async function POST(request: Request) {
  try {
    const session = await requireRole(['admin', 'operator', 'accountant'])
    if (session instanceof NextResponse) return session

    await connectDB()
    const body = await request.json()
    // Normalize date to canonical YYYY-MM-DD
    // (handles dd-mm-yyyy, dd/mm/yyyy, Excel serials, Date objects, etc.)
    if (body.date) body.date = normalizeDate(body.date)
    if (!body.date || !body.name || !body.amount) {
      return NextResponse.json({ error: 'Date, name and amount are required' }, { status: 400 })
    }
    const record = await LabourPayment.create({
      date: body.date,
      name: body.name,
      address: body.address || '',
      amount: Number(body.amount),
      remarks: body.remarks || '',
    })
    return NextResponse.json({ labourPayment: toObject(record) }, { status: 201 })
  } catch (error) {
    console.error('Error creating labour payment:', error)
    return NextResponse.json({ error: 'Failed to create labour payment' }, { status: 500 })
  }
}
```

/api/orders/[id]/route.ts 122 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB, toObject, extractCustomer } from '@/lib/db'
import { Order, Bill, Payment } from '@/lib/models'
import { requireSession, requireRole } from '@/lib/auth'
import { normalizeDate } from '@/lib/date-utils'

export async function GET(request: Request, { params }: { params: Promise<{ id: string }> }) {
  try {
    const session = await requireSession()
    if (session instanceof NextResponse) return session

    await connectDB()
    const { id } = await params
    const order = await Order.findById(id).populate('customerId')
    if (!order) {
      return NextResponse.json({ error: 'Order not found' }, { status: 404 })
    }
    const obj = toObject(order)
    const { customer, customerId } = extractCustomer(order)
    obj.customer = customer
    obj.customerId = customerId
    return NextResponse.json({ order: obj })
  } catch (error) {
    console.error('Error fetching order:', error)
    return NextResponse.json({ error: 'Failed to fetch order' }, { status: 500 })
  }
}

export async function PUT(request: Request, { params }: { params: Promise<{ id: string }> }) {
  try {
    const session = await requireRole(['admin', 'operator'])
    if (session instanceof NextResponse) return session

    await connectDB()
    const { id } = await params
    const body = await request.json()
    // Normalize date to canonical YYYY-MM-DD
    // (handles dd-mm-yyyy, dd/mm/yyyy, Excel serials, Date objects, etc.)
    if (body.deliveryDate) body.deliveryDate = normalizeDate(body.deliveryDate)

    const updateData: Record<string, unknown> = {}
    const fields = ['customerId', 'brickType', 'quantity', 'rate', 'amount', 'deliveryDate', 'status']
    for (const field of fields) {
      if (body[field] !== undefined) updateData[field] = body[field]
    }

    // Items array — when provided, normalize each item and recompute
    // the summary quantity/rate/amount fields so they stay in sync.
    if (Array.isArray(body.items)) {
      const items = body.items
        .map((it: any) => ({
          description: String(it.description || '').trim(),
          hsn: String(it.hsn || ''),
          quantity: Number(it.quantity) || 0,
          unit: String(it.unit || 'pcs'),
          rate: Number(it.rate) || 0,
          amount: Number(it.amount) || (Number(it.quantity) || 0) * (Number(it.rate) || 0),
        }))
        .filter((it: any) => it.description)
      updateData.items = items
      if (items.length > 0) {
        const totalQty = items.reduce((s: number, it: any) => s + it.quantity, 0)
        const totalAmt = items.reduce((s: number, it: any) => s + it.amount, 0)
        updateData.quantity = totalQty
        updateData.amount = totalAmt
        updateData.rate = totalQty > 0 ? totalAmt / totalQty : 0
        if (!updateData.brickType) updateData.brickType = items[0].description
      }
    }

    const order = await Order.findByIdAndUpdate(id, updateData, { new: true }).populate('customerId')
    if (!order) {
      return NextResponse.json({ error: 'Order not found' }, { status: 404 })
    }

    const obj = toObject(order)
    const { customer, customerId } = extractCustomer(order)
    obj.customer = customer
    obj.customerId = customerId
    return NextResponse.json({ order: obj })
  } catch (error) {
    console.error('Error updating order:', error)
    return NextResponse.json({ error: 'Failed to update order' }, { status: 500 })
```

```
  }
}

// ─────────────────────────────────────────────────────────────────────
// DELETE — remove an Order.
//
// CASCADE:
//   • Bills that reference this order via remarks/notes are NOT deleted
//     automatically — the user may have already printed and sent them.
//     Instead, we leave them as historical records.
//   • Dispatches linked to this order (orderId) are NOT deleted either —
//     they represent physical goods that left the factory and shouldn't be
//     erased just because the order was removed.
//
// What we DO clean up:
//   • Nothing automatically — the order is the source of truth, but its
//     deletion doesn't invalidate downstream financial records.
//
// This is the safe default. If you want aggressive cascade deletion,
// uncomment the cascade block below.
// ─────────────────────────────────────────────────────────────────────
export async function DELETE(request: Request, { params }: { params: Promise<{ id: string }> }) {
  try {
    const session = await requireRole(['admin', 'operator'])
    if (session instanceof NextResponse) return session

    await connectDB()
    const { id } = await params
    const order = await Order.findByIdAndDelete(id)
    if (!order) {
      return NextResponse.json({ error: 'Order not found' }, { status: 404 })
    }
    return NextResponse.json({ message: 'Order deleted successfully' })
  } catch (error) {
    console.error('Error deleting order:', error)
    return NextResponse.json({ error: 'Failed to delete order' }, { status: 500 })
  }
}
```

/api/orders/bulk-delete/route.ts 61 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB } from '@/lib/db'
import { Order } from '@/lib/models'
import { getSession } from '@/lib/auth'

export const dynamic = 'force-dynamic'

// POST /api/orders/bulk-delete
// Body: { ids: string[] }
// Gated behind admin/operator session — accountants are read-only.
export async function POST(request: Request) {
  try {
    await connectDB()

    const session = await getSession()
    if (!session) {
      return NextResponse.json(
        { error: 'Unauthorized — please log in' },
        { status: 401 }
      )
    }
    if (session.role === 'accountant') {
      return NextResponse.json(
        { error: 'Forbidden — accountants cannot delete orders' },
        { status: 403 }
      )
    }

    const body = await request.json()
    const { ids } = body as { ids?: string[] }

    if (!Array.isArray(ids) || ids.length === 0) {
      return NextResponse.json(
        { error: 'ids array is required and must be non-empty' },
        { status: 400 }
      )
    }

    const result = await Order.deleteMany({ _id: { $in: ids } })

    if (result.deletedCount === 0) {
      return NextResponse.json(
        { error: 'No matching orders found — they may have been already deleted' },
        { status: 404 }
      )
    }

    return NextResponse.json({
      message: `${result.deletedCount} order${result.deletedCount === 1 ? '' : 's'} deleted`,
      deletedCount: result.deletedCount,
      requestedCount: ids.length,
    })
  } catch (error) {
    console.error('Error bulk-deleting orders:', error)
    return NextResponse.json(
      { error: 'Failed to bulk-delete orders' },
      { status: 500 }
    )
  }
}
```

/api/orders/route.ts 115 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB, toObject, extractCustomer } from '@/lib/db'
import { Order } from '@/lib/models'
import { requireSession, requireRole } from '@/lib/auth'
import { normalizeDate } from '@/lib/date-utils'

// Force dynamic — never cache list responses
export const dynamic = 'force-dynamic'
export const revalidate = 0

export async function GET() {
  try {
    const session = await requireSession()
    if (session instanceof NextResponse) return session

    await connectDB()
    const orders = await Order.find({}).populate('customerId').sort({ createdAt: -1 })

    const result = orders.map((o: any) => {
      const obj = toObject(o)
      const { customer, customerId } = extractCustomer(o)
      obj.customer = customer
      obj.customerId = customerId
      return obj
    })

    return NextResponse.json({ orders: result })
  } catch (error) {
    console.error('Error fetching orders:', error)
    return NextResponse.json({ error: 'Failed to fetch orders' }, { status: 500 })
  }
}

export async function POST(request: Request) {
  try {
    const session = await requireRole(['admin', 'operator'])
    if (session instanceof NextResponse) return session

    await connectDB()
    const body = await request.json()
    // Normalize date to canonical YYYY-MM-DD
    // (handles dd-mm-yyyy, dd/mm/yyyy, Excel serials, Date objects, etc.)
    if (body.deliveryDate) body.deliveryDate = normalizeDate(body.deliveryDate)

    if (!body.customerId || !body.deliveryDate) {
      return NextResponse.json({ error: 'Customer and delivery date are required' }, { status: 400 })
    }
    // Either brickType+quantity+rate OR items[] must be provided
    const hasItems = Array.isArray(body.items) && body.items.length > 0
    const hasBrickType = body.brickType && body.quantity && body.rate
    if (!hasItems && !hasBrickType) {
      return NextResponse.json({ error: 'Provide either brick type + qty + rate, or at least one line item' }, { status: 400 })
    }

    const count = await Order.countDocuments({})
    const { Company } = await import('@/lib/models')
    const company = await Company.findOne({})
    const prefix = company?.orderPrefix || 'ORD'
    const orderNumber = `${prefix}-${String(count + 1).padStart(4, '0')}`

    // Normalize items array — each item gets its amount computed from
    // quantity * rate if not explicitly provided.
    const items = hasItems
      ? body.items.map((it: any) => ({
          description: String(it.description || '').trim(),
          hsn: String(it.hsn || ''),
          quantity: Number(it.quantity) || 0,
          unit: String(it.unit || 'pcs'),
          rate: Number(it.rate) || 0,
          amount: Number(it.amount) || (Number(it.quantity) || 0) * (Number(it.rate) || 0),
        })).filter((it: any) => it.description)
      : []

    // Compute summary fields from items (if present) — these populate the
    // legacy brickType/quantity/rate/amount columns for backward compat
    // with the dispatch module + reports that read those fields.
    let brickType = String(body.brickType || '')
    let quantity = Number(body.quantity) || 0
    let rate = Number(body.rate) || 0
    let amount = Number(body.amount) || (quantity * rate)

    if (items.length > 0) {
      // If brickType wasn't provided, use the first item's description
```

```
      if (!brickType) brickType = items[0].description
      quantity = items.reduce((s: number, it: any) => s + it.quantity, 0)
      amount = items.reduce((s: number, it: any) => s + it.amount, 0)
      // Weighted average rate
      rate = quantity > 0 ? amount / quantity : 0
    }

    const order = await Order.create({
      orderNumber,
      customerId: body.customerId,
      brickType,
      quantity,
      rate,
      amount,
      items,
      deliveryDate: body.deliveryDate,
      status: body.status || 'Pending',
    })

    const populated = await Order.findById(order._id).populate('customerId')
    const obj = toObject(populated)
    const { customer, customerId } = extractCustomer(populated)
    obj.customer = customer
    obj.customerId = customerId

    return NextResponse.json({ order: obj }, { status: 201 })
  } catch (error) {
    console.error('Error creating order:', error)
    return NextResponse.json({ error: 'Failed to create order' }, { status: 500 })
  }
}
```

/api/payments/[id]/route.ts 181 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB, toObject, extractCustomer } from '@/lib/db'
import { Payment } from '@/lib/models'
import { resyncBillPaidAmount } from '../route'
import { syncUpdateCustomerPayment, syncDeleteCustomerPayment } from '@/lib/payment-customer-sync'
import { requireSession, requireRole } from '@/lib/auth'
import { normalizeDate } from '@/lib/date-utils'

export async function GET(request: Request, { params }: { params: Promise<{ id: string }> }) {
  try {
    const session = await requireSession()
    if (session instanceof NextResponse) return session

    await connectDB()
    const { id } = await params
    const payment = await Payment.findById(id).populate('customerId')
    if (!payment) {
      return NextResponse.json({ error: 'Payment not found' }, { status: 404 })
    }
    const obj = toObject(payment)
    const { customer, customerId } = extractCustomer(payment)
    obj.customer = customer
    obj.customerId = customerId
    return NextResponse.json({ payment: obj })
  } catch (error) {
    console.error('Error fetching payment:', error)
    return NextResponse.json({ error: 'Failed to fetch payment' }, { status: 500 })
  }
}

// ─────────────────────────────────────────────────────────────────────
// PUT — update a Payment.
//
// AUTO-SYNC (reverse direction):
//   • If this Payment is linked to a Bill (billId), we re-sync the Bill's
//     paidAmount after the update so the Bill reflects the new payment amount.
//   • If the billId is being CHANGED (payment moved to a different bill):
//       1. Re-sync the OLD bill (its paidAmount drops by the old amount).
//       2. Re-sync the NEW bill (its paidAmount rises by the new amount).
//   • If billId is being CLEARED (unlinked): re-sync the old bill.
//   • If billId is being SET for the first time: re-sync the new bill.
//
// This makes the Bill module always reflect the truth of all Payments,
// regardless of which side was edited.
// ─────────────────────────────────────────────────────────────────────
export async function PUT(request: Request, { params }: { params: Promise<{ id: string }> }) {
  try {
    const session = await requireRole(['admin', 'accountant'])
    if (session instanceof NextResponse) return session

    await connectDB()
    const { id } = await params
    const body = await request.json()
    // Normalize date to canonical YYYY-MM-DD
    // (handles dd-mm-yyyy, dd/mm/yyyy, Excel serials, Date objects, etc.)
    if (body.date) body.date = normalizeDate(body.date)

    // Capture the old billId (if any) before update so we can re-sync it
    const existing = await Payment.findById(id).lean()
    if (!existing) {
      return NextResponse.json({ error: 'Payment not found' }, { status: 404 })
    }
    const oldBillId = existing.billId ? String(existing.billId) : null

    const updateData: Record<string, unknown> = {}
    const fields = ['customerId', 'paymentType', 'amount', 'date', 'remarks']
    for (const field of fields) {
      if (body[field] !== undefined) updateData[field] = body[field]
    }

    // Handle billId changes — explicit null means "unlink", undefined means "no change"
    let newBillId: string | null = oldBillId
    if (body.billId !== undefined) {
      if (body.billId) {
        // Linking to a new bill — also update billNumber for display
        const { Bill } = await import('@/lib/models')
        const linkedBill = await Bill.findById(body.billId).lean()
        if (!linkedBill) {
          return NextResponse.json({ error: 'Linked bill not found' }, { status: 400 })
        }
        updateData.billId = linkedBill._id
        updateData.billNumber = linkedBill.billNumber
        newBillId = String(linkedBill._id)
```

```
      } else {
        // Explicit unlink — clear billId + billNumber
        updateData.billId = null
        updateData.billNumber = ''
        newBillId = null
      }
    }

    const payment = await Payment.findByIdAndUpdate(id, updateData, { new: true }).populate('customerId')
    if (!payment) {
      return NextResponse.json({ error: 'Payment not found' }, { status: 404 })
    }

    // ── Reverse sync: recompute Bills' paidAmount ──────────────────
    // Re-sync OLD bill (if payment was previously linked and bill changed)
    if (oldBillId && oldBillId !== newBillId) {
      try { await resyncBillPaidAmount(oldBillId) } catch (e) { console.error('reverse-sync old bill failed:', e) }
    }
    // Re-sync NEW bill (if payment is now linked)
    if (newBillId) {
      try { await resyncBillPaidAmount(newBillId) } catch (e) { console.error('reverse-sync new bill failed:', e) }
    }

    // ── Mirror update into CustomerPayment module ──────────────────
    // Best-effort: any failure is logged but does not fail the Payment update.
    try {
      await syncUpdateCustomerPayment({
        paymentId: id,
        customerId: String(updateData.customerId || existing.customerId),
        amount: Number(updateData.amount ?? existing.amount) || 0,
        date: String(updateData.date || existing.date),
        paymentType: String(updateData.paymentType || existing.paymentType || ''),
        remarks: String((updateData.remarks ?? existing.remarks) || ''),
        billNumber: String((updateData.billNumber ?? existing.billNumber) || ''),
      })
    } catch (syncErr) {
      console.error('Payment → CustomerPayment mirror on update failed:', syncErr)
    }

    const obj = toObject(payment)
    const { customer, customerId } = extractCustomer(payment)
    obj.customer = customer
    obj.customerId = customerId
    return NextResponse.json({ payment: obj })
  } catch (error) {
    console.error('Error updating payment:', error)
    return NextResponse.json({ error: 'Failed to update payment' }, { status: 500 })
  }
}

// ─────────────────────────────────────────────────────────────────────
// DELETE — remove a Payment.
//
// AUTO-SYNC (reverse direction):
//   • If this Payment was linked to a Bill (billId), re-sync the Bill so its
//     paidAmount drops by this Payment's amount. The Bill's status is
//     recomputed (paid → partial → draft) automatically.
//   • The mirrored CustomerPayment record (if any) is also deleted so the
//     Customer Payment module stays in sync.
// ─────────────────────────────────────────────────────────────────────
export async function DELETE(request: Request, { params }: { params: Promise<{ id: string }> }) {
  try {
    const session = await requireRole(['admin', 'accountant'])
    if (session instanceof NextResponse) return session

    await connectDB()
    const { id } = await params

    // Capture the billId BEFORE deleting so we can re-sync afterwards
    const existing = await Payment.findById(id).lean()
    if (!existing) {
      return NextResponse.json({ error: 'Payment not found' }, { status: 404 })
    }
    const linkedBillId = existing.billId ? String(existing.billId) : null

    // ── Mirror delete into CustomerPayment module ──────────────────
    // Must run BEFORE the Payment itself is deleted, because the sync helper
    // reads `existing.customerPaymentId` to know which mirror to remove.
    try {
      await syncDeleteCustomerPayment(id)
    } catch (syncErr) {
      console.error('Payment → CustomerPayment mirror on delete failed:', syncErr)
    }

    await Payment.findByIdAndDelete(id)
```

```
    // Reverse sync: recompute the Bill's paidAmount (will drop by this payment's amount)
    if (linkedBillId) {
      try { await resyncBillPaidAmount(linkedBillId) } catch (e) { console.error('reverse-sync on delete failed:', e) }
    }

    return NextResponse.json({ message: 'Payment deleted successfully' })
  } catch (error) {
    console.error('Error deleting payment:', error)
    return NextResponse.json({ error: 'Failed to delete payment' }, { status: 500 })
  }
}
```

/api/payments/bulk-delete/route.ts 71 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB } from '@/lib/db'
import { Payment } from '@/lib/models'
import { getSession } from '@/lib/auth'
import { syncDeleteCustomerPayment } from '@/lib/payment-customer-sync'

export const dynamic = 'force-dynamic'

// POST /api/payments/bulk-delete
// Body: { ids: string[] }
// Gated behind admin/operator session — accountants are read-only.
//
// For each Payment being deleted, we also delete the linked CustomerPayment
// mirror (if any) so the Customer Payment module stays in sync.
export async function POST(request: Request) {
  try {
    await connectDB()

    const session = await getSession()
    if (!session) {
      return NextResponse.json(
        { error: 'Unauthorized — please log in' },
        { status: 401 }
      )
    }
    if (session.role === 'accountant') {
      return NextResponse.json(
        { error: 'Forbidden — accountants cannot delete payments' },
        { status: 403 }
      )
    }

    const body = await request.json()
    const { ids } = body as { ids?: string[] }

    if (!Array.isArray(ids) || ids.length === 0) {
      return NextResponse.json(
        { error: 'ids array is required and must be non-empty' },
        { status: 400 }
      )
    }

    // Best-effort: delete the CustomerPayment mirror for each Payment first
    // (so we can still read the `customerPaymentId` field before the Payment
    // itself is gone). Failures are logged inside the helper but never block
    // the parent delete.
    await Promise.all(ids.map((id) => syncDeleteCustomerPayment(id)))

    const result = await Payment.deleteMany({ _id: { $in: ids } })

    if (result.deletedCount === 0) {
      return NextResponse.json(
        { error: 'No matching payments found — they may have been already deleted' },
        { status: 404 }
      )
    }

    return NextResponse.json({
      message: `${result.deletedCount} payment${result.deletedCount === 1 ? '' : 's'} deleted`,
      deletedCount: result.deletedCount,
      requestedCount: ids.length,
    })
  } catch (error) {
    console.error('Error bulk-deleting payments:', error)
    return NextResponse.json(
      { error: 'Failed to bulk-delete payments' },
      { status: 500 }
    )
  }
}
```

/api/payments/route.ts 175 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB, toObject, extractCustomer } from '@/lib/db'
import { Payment, Bill } from '@/lib/models'
import {
  syncCreateCustomerPayment,
  syncUpdateCustomerPayment,
  syncDeleteCustomerPayment,
} from '@/lib/payment-customer-sync'
import { requireSession, requireRole } from '@/lib/auth'
import { normalizeDate } from '@/lib/date-utils'

// Force dynamic — never cache list responses
export const dynamic = 'force-dynamic'
export const revalidate = 0

export async function GET() {
  try {
    const session = await requireSession()
    if (session instanceof NextResponse) return session

    await connectDB()
    const payments = await Payment.find({}).populate('customerId').sort({ createdAt: -1 })

    const result = payments.map((p: any) => {
      const obj = toObject(p)
      const { customer, customerId } = extractCustomer(p)
      obj.customer = customer
      obj.customerId = customerId
      return obj
    })

    return NextResponse.json({ payments: result })
  } catch (error) {
    console.error('Error fetching payments:', error)
    return NextResponse.json({ error: 'Failed to fetch payments' }, { status: 500 })
  }
}

// ─────────────────────────────────────────────────────────────────
// POST — create a new Payment (manual entry from Payments module).
//
// AUTO-SYNC: This is the reverse direction of the Bill → Payment sync.
// When a user creates a Payment from the Payments module:
//   • If body.billId is supplied → link this Payment to that Bill and
//     increment the Bill's paidAmount by this Payment's amount. The Bill's
//     balanceAmount + status are recomputed.
//   • If body.billId is NOT supplied but the customer has an outstanding
//     Bill (status = 'partial' or 'draft' or 'sent'), we DO NOT auto-link.
//     The user must explicitly choose a bill from the dropdown in the UI.
//     We avoid auto-applying payments to arbitrary bills because that could
//     pay off the wrong invoice.
//
// After creating the Payment, if a billId was provided, we recompute the
// Bill's paidAmount as the SUM of all Payments linked to that bill
// (including this new one). This is more robust than incrementing by
// `amount` because it self-heals if any prior sync was missed.
// ─────────────────────────────────────────────────────────────────
export async function POST(request: Request) {
  try {
    const session = await requireRole(['admin', 'accountant'])
    if (session instanceof NextResponse) return session

    await connectDB()
    const body = await request.json()
    // Normalize date to canonical YYYY-MM-DD
    // (handles dd-mm-yyyy, dd/mm/yyyy, Excel serials, Date objects, etc.)
    if (body.date) body.date = normalizeDate(body.date)

    if (!body.customerId || !body.paymentType || !body.amount || !body.date) {
      return NextResponse.json({ error: 'Missing required fields' }, { status: 400 })
    }

    // Resolve billId — optional. If provided, must be a valid Bill _id.
    let billId: string | null = null
    let billNumber = ''
    if (body.billId) {
      const linkedBill = await Bill.findById(body.billId).lean()
      if (!linkedBill) {
        return NextResponse.json({ error: 'Linked bill not found' }, { status: 400 })
      }
      billId = String(linkedBill._id)
      billNumber = linkedBill.billNumber
    }
```

```
    const payment = await Payment.create({
      customerId: body.customerId,
      paymentType: body.paymentType,
      amount: Number(body.amount),
      date: body.date,
      remarks: body.remarks || '',
      billId,
      billNumber,
    })

    // ── Reverse sync: update the linked Bill's paidAmount ──────────────
    if (billId) {
      try {
        await resyncBillPaidAmount(billId)
      } catch (syncErr) {
        // Sync failure must NOT fail the payment creation.
        console.error('Payment → Bill reverse-sync failed on create:', syncErr)
      }
    }

    // ── Mirror into CustomerPayment module ─────────────────────────────
    // Best-effort: any failure is logged but does not fail the Payment create.
    try {
      await syncCreateCustomerPayment({
        paymentId: payment._id,
        customerId: body.customerId,
        amount: Number(body.amount) || 0,
        date: body.date,
        paymentType: body.paymentType,
        remarks: body.remarks || '',
        billNumber,
      })
    } catch (syncErr) {
      console.error('Payment → CustomerPayment mirror on create failed:', syncErr)
    }

    const populated = await Payment.findById(payment._id).populate('customerId')
    const obj = toObject(populated)
    const { customer, customerId } = extractCustomer(populated)
    obj.customer = customer
    obj.customerId = customerId
    return NextResponse.json({ payment: obj }, { status: 201 })
  } catch (error) {
    console.error('Error creating payment:', error)
    return NextResponse.json({ error: 'Failed to create payment' }, { status: 500 })
  }
}

// ─────────────────────────────────────────────────────────────────────────
// Helper: recompute a Bill's paidAmount + balanceAmount + status from the
// sum of all Payments linked to it via billId.
//
// Used by:
//   • /api/payments POST (new payment linked to bill)
//   • /api/payments/[id] PUT (payment updated — amount may have changed)
//   • /api/payments/[id] DELETE (payment removed — bill's paidAmount drops)
//
// This makes the Bill always reflect the actual sum of linked payments,
// regardless of which side was edited. Idempotent — safe to call repeatedly.
// ─────────────────────────────────────────────────────────────────────────
export async function resyncBillPaidAmount(billId: string): Promise<void> {
  const bill = await Bill.findById(billId)
  if (!bill) return

  // Sum all Payments linked to this bill
  const agg = await Payment.aggregate([
    { $match: { billId: bill._id } },
    { $group: { _id: null, total: { $sum: '$amount' } } },
  ])
  const totalPaid = agg.length > 0 ? Number(agg[0].total) || 0 : 0

  // Recompute balance + status
  const grandTotal = Number(bill.grandTotal) || 0
  const balanceAmount = grandTotal - totalPaid

  let status: string
  if (totalPaid >= grandTotal && grandTotal > 0) {
    status = 'paid'
  } else if (totalPaid > 0) {
    status = 'partial'
  } else {
    status = 'draft'
  }

  await Bill.findByIdAndUpdate(bill._id, {
```

```
    paidAmount: totalPaid,
    balanceAmount,
    status,
  })
}
```

/api/production/[id]/route.ts 118 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB, toObject } from '@/lib/db'
import { Production } from '@/lib/models'
import { syncStockForDate } from '@/lib/sync-stock'
import { requireSession, requireRole } from '@/lib/auth'
import { normalizeDate } from '@/lib/date-utils'

export async function GET(request: Request, { params }: { params: Promise<{ id: string }> }) {
  try {
    const session = await requireSession()
    if (session instanceof NextResponse) return session

    await connectDB()
    const { id } = await params
    const production = await Production.findById(id)
    if (!production) {
      return NextResponse.json({ error: 'Production entry not found' }, { status: 404 })
    }
    return NextResponse.json({ production: toObject(production) })
  } catch (error) {
    console.error('Error fetching production:', error)
    return NextResponse.json({ error: 'Failed to fetch production entry' }, { status: 500 })
  }
}

export async function PUT(request: Request, { params }: { params: Promise<{ id: string }> }) {
  try {
    const session = await requireRole(['admin', 'operator'])
    if (session instanceof NextResponse) return session

    await connectDB()
    const { id } = await params
    const body = await request.json()
    // Normalize date to canonical YYYY-MM-DD
    // (handles dd-mm-yyyy, dd/mm/yyyy, Excel serials, Date objects, etc.)
    if (body.date) body.date = normalizeDate(body.date)

    const updateData: Record<string, unknown> = {}
    const fields = [
      'date',
      'customerId',
      'cement',
      'zigZagGrey80',
      'zigZagRed80',
      'zigZagYellow80',
      'zigZagGrey60',
      'zigZagRed60',
      'zigZagYellow60',
      'curveStone',
      'chequreTile',
      'dumbleGrey80',
      'dumbleRed80',
      'dumbleYellow80',
      'transportationCharge',
      'remarks',
    ]
    for (const field of fields) {
      if (body[field] !== undefined) updateData[field] = body[field]
    }

    const production = await Production.findByIdAndUpdate(id, updateData, { new: true })
    if (!production) {
      return NextResponse.json({ error: 'Production entry not found' }, { status: 404 })
    }

    // Auto-sync Stock Overview for this production's date so manual edits
    // to a production row reflect in the stock snapshot.
    try {
      await syncStockForDate(String(production.date))
    } catch (err) {
      console.error('[PUT /production/[id]] Stock sync failed:', err)
    }

    return NextResponse.json({ production: toObject(production) })
  } catch (error) {
    console.error('Error updating production:', error)
    return NextResponse.json({ error: 'Failed to update production entry' }, { status: 500 })
  }
}

export async function DELETE(request: Request, { params }: { params: Promise<{ id: string }> }) {
  try {
    const session = await requireRole(['admin', 'operator'])
```

```
    if (session instanceof NextResponse) return session

    await connectDB()
    const { id } = await params

    const production = await Production.findById(id)
    if (!production) {
      return NextResponse.json({ error: 'Production entry not found' }, { status: 404 })
    }

    const touchedDate = String(production.date || '')

    // Note: Stock auto-update on production delete is intentionally omitted —
    // Production tracks daily output quantities per product, while Stock is a
    // separate daily snapshot. The two are reconciled via the Stock module UI.
    await Production.findByIdAndDelete(id)

    // Re-aggregate Stock snapshot for the deleted row's date — if no other
    // production rows exist for that date, the Stock entry will be replaced
    // with zeros (or could be deleted, but we keep a zero row for visibility).
    if (touchedDate) {
      try {
        await syncStockForDate(touchedDate)
      } catch (err) {
        console.error('[DELETE /production/[id]] Stock sync failed:', err)
      }
    }

    return NextResponse.json({ message: 'Production entry deleted successfully' })
  } catch (error) {
    console.error('Error deleting production:', error)
    return NextResponse.json({ error: 'Failed to delete production entry' }, { status: 500 })
  }
}
```

/api/production/bulk-delete/route.ts 95 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB } from '@/lib/db'
import { Production } from '@/lib/models'
import { syncStockForDates } from '@/lib/sync-stock'
import { getSession } from '@/lib/auth'

// Force dynamic — never cache
export const dynamic = 'force-dynamic'

// POST /api/production/bulk-delete
// Body: { ids: string[] }
// Deletes the production entries whose `_id` is in `ids`.
// Re-aggregates Stock snapshots for every touched date so the Stock module
// reflects the deletions automatically.
// Gated behind admin/operator session — bulk destructive operations should
// not be callable by unauthenticated users.
export async function POST(request: Request) {
  try {
    await connectDB()

    // Auth gate — any logged-in user with admin or operator role can bulk-delete.
    // Accountant role is read-only for production per the existing permission
    // matrix, so we deny here.
    const session = await getSession()
    if (!session) {
      return NextResponse.json(
        { error: 'Unauthorized — please log in' },
        { status: 401 }
      )
    }
    if (session.role === 'accountant') {
      return NextResponse.json(
        { error: 'Forbidden — accountants cannot delete production entries' },
        { status: 403 }
      )
    }

    const body = await request.json()
    const { ids } = body as { ids?: string[] }

    if (!Array.isArray(ids) || ids.length === 0) {
      return NextResponse.json(
        { error: 'ids array is required and must be non-empty' },
        { status: 400 }
      )
    }

    // Fetch the dates of the rows we're about to delete — we need them to
    // re-sync the Stock snapshot afterwards.
    const docs = await Production.find({ _id: { $in: ids } })
      .select('date')
      .lean() as { date?: string; _id: unknown }[]

    if (docs.length === 0) {
      return NextResponse.json(
        { error: 'No matching production entries found — they may have been already deleted' },
        { status: 404 }
      )
    }

    const result = await Production.deleteMany({ _id: { $in: ids } })

    // Re-aggregate Stock snapshots for every touched date so Stock Overview
    // is consistent. We deduplicate dates to avoid redundant re-syncs.
    const touchedDates = Array.from(
      new Set(
        docs
          .map((d) => String(d.date || '').split('T')[0])
          .filter(Boolean)
      )
    )
    if (touchedDates.length > 0) {
      try {
        await syncStockForDates(touchedDates)
      } catch (err) {
        console.error('[POST /production/bulk-delete] Stock sync failed:', err)
        // Don't fail the request — rows were already deleted.
      }
    }

    return NextResponse.json({
      message: `${result.deletedCount} production entr${result.deletedCount === 1 ? 'y' : 'ies'} deleted`,
      deletedCount: result.deletedCount,
```

```
      requestedCount: ids.length,
      stockResyncedDates: touchedDates.length,
    })
  } catch (error) {
    console.error('Error bulk-deleting productions:', error)
    return NextResponse.json(
      { error: 'Failed to bulk-delete production entries' },
      { status: 500 }
    )
  }
}
```

/api/production/route.ts 116 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB, toObject } from '@/lib/db'
import { Production, Stock } from '@/lib/models'
import { syncStockForDate } from '@/lib/sync-stock'
import { requireSession, requireRole, requireAdmin } from '@/lib/auth'
import { normalizeDate } from '@/lib/date-utils'

// Force dynamic — never cache list responses
export const dynamic = 'force-dynamic'
export const revalidate = 0

export async function GET(request: Request) {
  try {
    const session = await requireSession()
    if (session instanceof NextResponse) return session

    await connectDB()
    const { searchParams } = new URL(request.url)
    const date = searchParams.get('date')
    const filter: any = {}
    if (date) filter.date = date
    const productions = await Production.find(filter).sort({ date: -1 }).lean()
    return NextResponse.json({ productions: productions.map(toObject) })
  } catch (error) {
    console.error('Error fetching production:', error)
    return NextResponse.json({ error: 'Failed to fetch production entries' }, { status: 500 })
  }
}

export async function POST(request: Request) {
  try {
    const session = await requireRole(['admin', 'operator'])
    if (session instanceof NextResponse) return session

    await connectDB()
    const body = await request.json()
    // Normalize date to canonical YYYY-MM-DD
    // (handles dd-mm-yyyy, dd/mm/yyyy, Excel serials, Date objects, etc.)
    if (body.date) body.date = normalizeDate(body.date)
    if (!body.date) {
      return NextResponse.json({ error: 'Date is required' }, { status: 400 })
    }
    const production = await Production.create({
      date: body.date,
      customerId: body.customerId || null,
      cement: Number(body.cement) || 0,
      zigZagGrey80: Number(body.zigZagGrey80) || 0,
      zigZagRed80: Number(body.zigZagRed80) || 0,
      zigZagYellow80: Number(body.zigZagYellow80) || 0,
      zigZagGrey60: Number(body.zigZagGrey60) || 0,
      zigZagRed60: Number(body.zigZagRed60) || 0,
      zigZagYellow60: Number(body.zigZagYellow60) || 0,
      curveStone: Number(body.curveStone) || 0,
      chequreTile: Number(body.chequreTile) || 0,
      dumbleGrey80: Number(body.dumbleGrey80) || 0,
      dumbleRed80: Number(body.dumbleRed80) || 0,
      dumbleYellow80: Number(body.dumbleYellow80) || 0,
      transportationCharge: Number(body.transportationCharge) || 0,
      remarks: body.remarks || '',
    })

    // Auto-sync Stock Overview: re-aggregate the Stock snapshot for this date
    // so it reflects the newly-added production row.
    try {
      await syncStockForDate(String(body.date))
    } catch (err) {
      console.error('[POST /production] Stock sync failed:', err)
    }

    return NextResponse.json({ production: toObject(production) }, { status: 201 })
  } catch (error) {
    console.error('Error creating production:', error)
    return NextResponse.json({ error: 'Failed to create production entry' }, { status: 500 })
  }
}

// DELETE /api/production?all=true
// Wipes ALL production entries. Gated behind admin session — only admins
// can perform bulk destructive operations.
export async function DELETE(request: Request) {
  try {
    const session = await requireAdmin()
    if (session instanceof NextResponse) return session
```

```
    await connectDB()

    const { searchParams } = new URL(request.url)
    const all = searchParams.get('all')
    if (all !== 'true' && all !== '1') {
      return NextResponse.json(
        { error: 'Missing ?all=true — pass it to confirm bulk delete' },
        { status: 400 }
      )
    }

    const result = await Production.deleteMany({})

    // Also wipe all Stock entries since they're derived from Production.
    // (User specifically requested Stock Overview reflect Production data —
    // wiping production should also clear the synced stock snapshot.)
    try {
      await Stock.deleteMany({})
    } catch (err) {
      console.error('[DELETE /production?all] Stock wipe failed:', err)
    }

    return NextResponse.json({
      message: 'All production entries deleted',
      deletedCount: result.deletedCount,
    })
  } catch (error) {
    console.error('Error deleting all productions:', error)
    return NextResponse.json({ error: 'Failed to delete all production entries' }, { status: 500 })
  }
}
```

/api/reports/route.ts 365 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB } from '@/lib/db'
import { Production, DailySell, Expense, Customer, TractorPayment } from '@/lib/models'
import { requireSession } from '@/lib/auth'

// ── Product fields (single source of truth — must match /api/stock/summary) ─
const PRODUCT_FIELDS: { key: string; name: string }[] = [
  { key: 'cement',         name: 'Cement' },
  { key: 'zigZagGrey80',   name: 'Zig Zag Grey 80mm' },
  { key: 'zigZagRed80',    name: 'Zig Zag Red 80mm' },
  { key: 'zigZagYellow80', name: 'Zig Zag Yellow 80mm' },
  { key: 'zigZagGrey60',   name: 'Zig Zag Grey 60mm' },
  { key: 'zigZagRed60',    name: 'Zig Zag Red 60mm' },
  { key: 'zigZagYellow60', name: 'Zig Zag Yellow 60mm' },
  { key: 'chequreTile',    name: 'Chequre Tile' },
  { key: 'curveStone',     name: 'Curve Stone' },
  { key: 'dumbleGrey80',   name: 'Dumble Grey 80mm' },
  { key: 'dumbleRed80',    name: 'Dumble Red 80mm' },
  { key: 'dumbleYellow80', name: 'Dumble Yellow 80mm' },
]

export async function GET(request: Request) {
  try {
    const session = await requireSession()
    if (session instanceof NextResponse) return session

    await connectDB()

    const { searchParams } = new URL(request.url)
    const type = searchParams.get('type') || 'sales'
    // Optional month filter: YYYY-MM. If provided, sales/production/P&L are
    // narrowed to that month. Default = current month for P&L, all-time for others.
    const monthFilter = searchParams.get('month') // e.g. "2026-07"
    const fromFilter = searchParams.get('from')   // e.g. "2026-07-01"
    const toFilter = searchParams.get('to')       // e.g. "2026-07-31"

    // Helper: should this date string pass the filter?
    const dateMatches = (dateStr: string): boolean => {
      if (fromFilter && dateStr < fromFilter) return false
      if (toFilter && dateStr > toFilter) return false
      if (monthFilter && !dateStr.startsWith(monthFilter)) return false
      return true
    }

    switch (type) {
      // ──────────────────────────────────────────────────────────────
      // SALES REPORT — source: DailySell
      // Frontend expects: { data: SalesRow[], totalSales: number }
      // SalesRow = { id, date, customerId, customer: {id, name} | null,
      //              brickType, quantity, rate, totalAmount, orderId }
      // ──────────────────────────────────────────────────────────────
      case 'sales': {
        const dailySells = await DailySell.find({}).sort({ date: -1 }).lean()
        // ── Expand multi-product records ──────────────────────────
        // A DailySell entry can hold multiple line items in `products[]`.
        // For the sales report, we expand each multi-product record into
        // one row PER product so the user sees every product the customer
        // ordered (instead of the legacy "first product, +N more" string
        // stored on `d.product`). Single-product records (legacy or
        // products.length === 1) yield exactly one row, identical to before.
        const rows: any[] = []
        for (const d of dailySells) {
          if (!dateMatches(String(d.date || ''))) continue
          const base = {
            date: String(d.date || ''),
            customerId: d.customerId ? String(d.customerId) : '',
            customer: d.customerName
              ? { id: d.customerId ? String(d.customerId) : '', name: String(d.customerName) }
              : null,
            orderId: d.orderId ? String(d.orderId) : null,
          }
          const prods = Array.isArray((d as any).products) ? (d as any).products : []
          if (prods.length > 0) {
            prods.forEach((p: any, idx: number) => {
              const qty = Number(p.quantity) || 0
              const rate = Number(p.rate) || 0
              const amt = Number(p.amount) || qty * rate
              rows.push({
                id: `${d._id}-${idx}`,
                ...base,
                brickType: String(p.product || ''),
                quantity: qty,
                rate,
```

```
            totalAmount: amt,
          })
        })
      } else {
        // Legacy single-product record (no products[] field).
        rows.push({
          id: String(d._id),
          ...base,
          brickType: String((d as any).product || ''),
          quantity: Number((d as any).quantity) || 0,
          rate: Number((d as any).rate) || 0,
          totalAmount: Number((d as any).amount) || 0,
        })
      }
    }
    const totalSales = rows.reduce((sum, r) => sum + r.totalAmount, 0)
    return NextResponse.json({ data: rows, totalSales })
  }

  // ─────────────────────────────────────────────────────────────────
  // PRODUCTION REPORT — source: Production (12 product columns per row)
  // Frontend expects: { data: ProductionRow[], totalProduced, byBrickType }
  // ProductionRow = { id, date, brickType, quantityProduced, shift }
  // We flatten: each non-zero product field becomes a separate row.
  // ─────────────────────────────────────────────────────────────────
  case 'production': {
    const productions = await Production.find({}).sort({ date: -1 }).lean()
    const rows: any[] = []
    const byBrickType: Record<string, number> = {}
    let totalProduced = 0
    for (const p of productions) {
      const dateStr = String(p.date || '')
      if (!dateMatches(dateStr)) continue
      for (const f of PRODUCT_FIELDS) {
        const qty = Number((p as any)[f.key]) || 0
        if (qty <= 0) continue
        rows.push({
          id: `${p._id}-${f.key}`,
          date: dateStr,
          brickType: f.name,
          quantityProduced: qty,
          shift: '', // not tracked in this schema
        })
        byBrickType[f.name] = (byBrickType[f.name] || 0) + qty
        totalProduced += qty
      }
    }
    return NextResponse.json({ data: rows, totalProduced, byBrickType })
  }

  // ─────────────────────────────────────────────────────────────────
  // STOCK REPORT — source: Production - Sold (matches /api/stock/summary)
  // Frontend expects: { data: StockRow[], totalCurrentStock,
  //                     totalOpeningStock, lowStockItems }
  // StockRow = { id, brickType, openingStock, currentStock,
  //              lowStockAlert, stockValue }
  //   - openingStock   = previous-year production
  //   - currentStock   = total production − total sold (availableQuantity)
  //   - lowStockAlert  = currentStock < 100
  // ─────────────────────────────────────────────────────────────────
  case 'stock': {
    const [productions, dailySells] = await Promise.all([
      Production.find({}).sort({ date: -1 }).lean(),
      DailySell.find({}).lean(),
    ])

    const currentYear = new Date().getFullYear()
    const currentYearStart = `${currentYear}-01-01`

    // Single-pass per-field aggregation
    const totalByField     = new Array(PRODUCT_FIELDS.length).fill(0)
    const prevYearByField  = new Array(PRODUCT_FIELDS.length).fill(0)
    for (const p of productions) {
      const dateStr = String(p.date || '')
      const isPrevYear = dateStr && dateStr < currentYearStart
      for (let i = 0; i < PRODUCT_FIELDS.length; i++) {
        const v = Number((p as any)[PRODUCT_FIELDS[i].key]) || 0
        if (v <= 0) continue
        totalByField[i] += v
        if (isPrevYear) prevYearByField[i] += v
      }
    }

    // Sold by product name. Multi-product records store line items in
    // `products[]`; legacy single-product records only have `product`.
    // We iterate over `products[]` when present so EVERY sold item is
```

```
      // counted toward the correct product's stock — otherwise multi-
      // product sales would only reduce the first product's stock.
      const soldByField: Record<string, number> = {}
      for (const d of dailySells) {
        const prods = Array.isArray((d as any).products) ? (d as any).products : []
        if (prods.length > 0) {
          for (const p of prods) {
            const prodName = String((p as any).product || '').trim()
            if (!prodName) continue
            const qty = Number((p as any).quantity) || 0
            soldByField[prodName] = (soldByField[prodName] || 0) + qty
          }
        } else {
          const prod = String((d as any).product || '').trim()
          if (!prod) continue
          const qty = Number((d as any).quantity) || 0
          soldByField[prod] = (soldByField[prod] || 0) + qty
        }
      }

      const rows = PRODUCT_FIELDS.map((f, i) => {
        const openingStock  = prevYearByField[i]
        const totalProd     = totalByField[i]
        const sold          = soldByField[f.name] || 0
        const currentStock  = Math.max(0, totalProd - sold)
        const lowStockAlert = currentStock < 100
        return {
          id: f.key,
          brickType: f.name,
          openingStock,
          currentStock,
          lowStockAlert,
          stockValue: 0, // could be filled if rate data is available
        }
      })
      const totalCurrentStock = rows.reduce((s, r) => s + r.currentStock, 0)
      const totalOpeningStock = rows.reduce((s, r) => s + r.openingStock, 0)
      const lowStockItems = rows.filter((r) => r.lowStockAlert)
      return NextResponse.json({
        data: rows,
        totalCurrentStock,
        totalOpeningStock,
        lowStockItems,
      })
    }

    // ─────────────────────────────────────────────────────────────
    // PROFIT & LOSS — source: DailySell (revenue) + Expense (expenses)
    // Frontend expects ProfitLossData = { reportType, month, totalRevenue,
    //   totalExpenses, netProfit, expensesByCategory }
    // ─────────────────────────────────────────────────────────────
    case 'profit-loss': {
      const effectiveMonth = monthFilter || new Date().toISOString().substring(0, 7)
      const [dailySells, expenses] = await Promise.all([
        DailySell.find({}).lean(),
        Expense.find({}).lean(),
      ])

      const filteredSales = dailySells.filter((d: any) =>
        dateMatches(String(d.date || ''))
      )
      const filteredExpenses = expenses.filter((e: any) =>
        dateMatches(String(e.date || ''))
      )

      const totalRevenue = filteredSales.reduce((sum, s: any) => sum + (Number(s.amount) || 0), 0)
      const totalExpenses = filteredExpenses.reduce((sum, e: any) => sum + (Number(e.amount) || 0), 0)
      const expensesByCategory: Record<string, number> = {}
      for (const e of filteredExpenses) {
        const cat = String((e as any).category || 'Other')
        expensesByCategory[cat] = (expensesByCategory[cat] || 0) + (Number((e as any).amount) || 0)
      }
      const netProfit = totalRevenue - totalExpenses
      return NextResponse.json({
        reportType: 'profit-loss',
        month: effectiveMonth,
        totalRevenue,
        totalExpenses,
        netProfit,
        expensesByCategory,
        // Extras (not in TS type but useful for the UI):
        totalPaymentsReceived: filteredSales.reduce((s, x: any) => s + (Number(x.receivedAmount) || 0), 0),
        outstanding: filteredSales.reduce((s, x: any) => s + (Number(x.pendingAmount) || 0), 0),
      })
    }
```

```
    // ──────────────────────────────────────────────────────────
    // OUTSTANDING REPORT — per-customer pending payments
    // Frontend expects: { data: OutstandingRow[], totalOutstanding }
    // OutstandingRow = { customerId, customer: {id, name} | null,
    //                    totalOrders, totalPayments, outstanding }
    // Source: DailySell grouped by customerName. We use customerName as
    // the primary grouping key because that's what the user types into
    // the Daily Sell form; customerId may be null on older entries.
    // ──────────────────────────────────────────────────────────
    case 'outstanding': {
      const dailySells = await DailySell.find({}).lean()
      // Group by customerName → { totalOrders, totalPayments, outstanding, customerId }
      const byCustomer = new Map<string, {
        customerId: string
        totalOrders: number
        totalPayments: number
        outstanding: number
      }>()
      for (const d of dailySells) {
        const name = String((d as any).customerName || '').trim()
        if (!name) continue
        if (!dateMatches(String((d as any).date || ''))) continue
        const amt = Number((d as any).amount) || 0
        const rec = Number((d as any).receivedAmount) || 0
        const pend = Number((d as any).pendingAmount ?? Math.max(0, amt - rec)) || 0
        const cid = (d as any).customerId ? String((d as any).customerId) : ''
        const existing = byCustomer.get(name) || {
          customerId: cid,
          totalOrders: 0,
          totalPayments: 0,
          outstanding: 0,
        }
        existing.totalOrders += amt
        existing.totalPayments += rec
        existing.outstanding += pend
        // Prefer a non-empty customerId if we see one
        if (!existing.customerId && cid) existing.customerId = cid
        byCustomer.set(name, existing)
      }
      const data = Array.from(byCustomer.entries())
        .map(([name, v]) => ({
          id: v.customerId || `name:${name}`, // stable React key
          customerId: v.customerId,
          customer: { id: v.customerId, name },
          totalOrders: v.totalOrders,
          totalPayments: v.totalPayments,
          outstanding: v.outstanding,
        }))
        // Show customers with outstanding > 0 first, then by totalOrders desc
        .sort((a, b) => {
          if ((b.outstanding > 0 ? 1 : 0) !== (a.outstanding > 0 ? 1 : 0)) {
            return (b.outstanding > 0 ? 1 : 0) - (a.outstanding > 0 ? 1 : 0)
          }
          return b.totalOrders - a.totalOrders
        })
      const totalOutstanding = data.reduce((s, r) => s + r.outstanding, 0)
      return NextResponse.json({ data, totalOutstanding })
    }

    // ──────────────────────────────────────────────────────────
    // CUSTOMER LEDGER — kept for backward compat (older reports UI)
    // Same shape as outstanding but groups by Customer collection ID.
    // ──────────────────────────────────────────────────────────
    case 'customer-ledger': {
      const [customers, dailySells] = await Promise.all([
        Customer.find({}).lean(),
        DailySell.find({}).lean(),
      ])
      // Build by customerId (fallback to customerName)
      const byKey = new Map<string, {
        customerId: string
        customerName: string
        totalOrders: number
        totalPayments: number
      }>()
      for (const d of dailySells) {
        const cid = (d as any).customerId ? String((d as any).customerId) : ''
        const name = String((d as any).customerName || '').trim()
        const key = cid || `name:${name}`
        const amt = Number((d as any).amount) || 0
        const rec = Number((d as any).receivedAmount) || 0
        const existing = byKey.get(key) || {
          customerId: cid,
          customerName: name,
          totalOrders: 0,
          totalPayments: 0,
```

```
          }
          existing.totalOrders += amt
          existing.totalPayments += rec
          byKey.set(key, existing)
        }
        const ledger = Array.from(byKey.values()).map((v) => ({
          customerId: v.customerId,
          customer: { id: v.customerId, name: v.customerName },
          totalOrders: v.totalOrders,
          totalPayments: v.totalPayments,
          outstanding: Math.max(0, v.totalOrders - v.totalPayments),
        }))
        return NextResponse.json({ customerLedger: ledger })
      }

      default:
        return NextResponse.json({ error: 'Invalid report type' }, { status: 400 })
    }
  } catch (error) {
    console.error('Error generating report:', error)
    return NextResponse.json({ error: 'Failed to generate report' }, { status: 500 })
  }
}
```

/api/route.ts 6 lines

```ts
import { NextResponse } from 'next/server'

export async function GET() {
  return NextResponse.json({ status: 'ok', message: 'Veda Enterprises ERP API is running' })
}
```

/api/stock/[id]/route.ts 122 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB, toObject } from '@/lib/db'
import { Stock } from '@/lib/models'
import { requireSession, requireRole } from '@/lib/auth'
import { normalizeDate } from '@/lib/date-utils'

// Force dynamic — never cache individual stock responses
export const dynamic = 'force-dynamic'
export const revalidate = 0

// GET /api/stock/[id] — fetch a single stock entry
export async function GET(
  _request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const session = await requireSession()
    if (session instanceof NextResponse) return session

    await connectDB()
    const { id } = await params
    const stock = await Stock.findById(id).lean()
    if (!stock) {
      return NextResponse.json({ error: 'Stock entry not found' }, { status: 404 })
    }
    return NextResponse.json({ stock: toObject(stock) })
  } catch (error) {
    console.error('Error fetching stock entry:', error)
    return NextResponse.json({ error: 'Failed to fetch stock entry' }, { status: 500 })
  }
}

// PUT /api/stock/[id] — update a single stock entry
//
// Whitelist of updatable fields. Must match StockSchema in src/lib/models.ts.
// Old UI used keys with "mm" suffix (zigZagGrey80mm etc.) which silently
// failed — that bug is now fixed in stock-module.tsx but we still accept
// both spellings here for backward compatibility (in case any old client
// caches the JS).
const STOCK_FIELDS: Array<{ canonical: string; aliases: string[] }> = [
  { canonical: 'date',           aliases: [] },
  { canonical: 'cement',         aliases: [] },
  { canonical: 'zigZagGrey80',   aliases: ['zigZagGrey80mm'] },
  { canonical: 'zigZagRed80',    aliases: ['zigZagRed80mm'] },
  { canonical: 'zigZagYellow80', aliases: ['zigZagYellow80mm'] },
  { canonical: 'zigZagGrey60',   aliases: ['zigZagGrey60mm'] },
  { canonical: 'zigZagRed60',    aliases: ['zigZagRed60mm'] },
  { canonical: 'zigZagYellow60', aliases: ['zigZagYellow60mm'] },
  { canonical: 'chequreTile',    aliases: [] },
  { canonical: 'curveStone',     aliases: [] },
  { canonical: 'dumbleGrey80',   aliases: ['dumbleGrey80mm'] },
  { canonical: 'dumbleRed80',    aliases: ['dumbleRed80mm'] },
  { canonical: 'dumbleYellow80', aliases: ['dumbleYellow80mm'] },
]

export async function PUT(
  request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const session = await requireRole(['admin', 'operator'])
    if (session instanceof NextResponse) return session

    await connectDB()
    const { id } = await params
    const body = await request.json()
    // Normalize date to canonical YYYY-MM-DD
    // (handles dd-mm-yyyy, dd/mm/yyyy, Excel serials, Date objects, etc.)
    if (body.date) body.date = normalizeDate(body.date)

    const updateData: Record<string, unknown> = {}
    for (const { canonical, aliases } of STOCK_FIELDS) {
      // Check canonical name first
      if (body[canonical] !== undefined) {
        updateData[canonical] = body[canonical]
        continue
      }
      // Then aliases (backward compat)
      for (const alias of aliases) {
        if (body[alias] !== undefined) {
          updateData[canonical] = body[alias]
          break
        }
```

```
      }
    }

    const stock = await Stock.findByIdAndUpdate(id, updateData, { new: true })
    if (!stock) {
      return NextResponse.json({ error: 'Stock entry not found' }, { status: 404 })
    }

    return NextResponse.json({ stock: toObject(stock) })
  } catch (error) {
    console.error('Error updating stock entry:', error)
    return NextResponse.json({ error: 'Failed to update stock entry' }, { status: 500 })
  }
}

// DELETE /api/stock/[id] — delete a single stock entry
export async function DELETE(
  _request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const session = await requireRole(['admin', 'operator'])
    if (session instanceof NextResponse) return session

    await connectDB()
    const { id } = await params

    const stock = await Stock.findByIdAndDelete(id)
    if (!stock) {
      return NextResponse.json({ error: 'Stock entry not found' }, { status: 404 })
    }

    return NextResponse.json({ message: 'Stock entry deleted successfully' })
  } catch (error) {
    console.error('Error deleting stock entry:', error)
    return NextResponse.json({ error: 'Failed to delete stock entry' }, { status: 500 })
  }
}
```

/api/stock/route.ts 107 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB, toObject } from '@/lib/db'
import { Stock } from '@/lib/models'
import { requireSession, requireAdmin } from '@/lib/auth'
import { normalizeDate } from '@/lib/date-utils'

// Force dynamic — never cache list responses
export const dynamic = 'force-dynamic'
export const revalidate = 0

export async function GET() {
  try {
    const session = await requireSession()
    if (session instanceof NextResponse) return session

    await connectDB()
    const stocks = await Stock.find({}).sort({ date: -1 }).lean()
    return NextResponse.json({ stocks: stocks.map(toObject) })
  } catch (error) {
    console.error('Error fetching stock:', error)
    return NextResponse.json({ error: 'Failed to fetch stock entries' }, { status: 500 })
  }
}

export async function POST(request: Request) {
  try {
    // All POST routes (bulk-delete AND create) require admin session
    const session = await requireAdmin()
    if (session instanceof NextResponse) return session

    await connectDB()
    const body = await request.json()
    // Normalize date to canonical YYYY-MM-DD
    // (handles dd-mm-yyyy, dd/mm/yyyy, Excel serials, Date objects, etc.)
    if (body.date) body.date = normalizeDate(body.date)

    // ── Bulk delete: POST /api/stock with { ids: [...] } ──────────────
    // Mirrors the production bulk-delete API so the same client-side
    // pattern works for both modules.
    if (body && Array.isArray(body.ids)) {
      const ids = body.ids.filter((id: unknown) => typeof id === 'string' && id.length > 0)
      if (ids.length === 0) {
        return NextResponse.json({ error: 'No ids provided' }, { status: 400 })
      }
      const result = await Stock.deleteMany({ _id: { $in: ids } })
      return NextResponse.json({
        message: 'Stock entries deleted successfully',
        deletedCount: result.deletedCount || 0,
      })
    }

    if (!body.date) {
      return NextResponse.json({ error: 'Date is required' }, { status: 400 })
    }
    const stock = await Stock.create({
      date: body.date,
      cement: Number(body.cement) || 0,
      zigZagGrey80: Number(body.zigZagGrey80) || 0,
      zigZagRed80: Number(body.zigZagRed80) || 0,
      zigZagYellow80: Number(body.zigZagYellow80) || 0,
      zigZagGrey60: Number(body.zigZagGrey60) || 0,
      zigZagRed60: Number(body.zigZagRed60) || 0,
      zigZagYellow60: Number(body.zigZagYellow60) || 0,
      chequreTile: Number(body.chequreTile) || 0,
      curveStone: Number(body.curveStone) || 0,
      dumbleGrey80: Number(body.dumbleGrey80) || 0,
      dumbleRed80: Number(body.dumbleRed80) || 0,
      dumbleYellow80: Number(body.dumbleYellow80) || 0,
    })
    return NextResponse.json({ stock: toObject(stock) }, { status: 201 })
  } catch (error) {
    console.error('Error creating stock:', error)
    return NextResponse.json({ error: 'Failed to create stock entry' }, { status: 500 })
  }
}

// DELETE /api/stock?all=true — delete every stock entry (Delete All button).
// Admin-only — destructive bulk operation.
export async function DELETE(request: Request) {
  try {
    const session = await requireAdmin()
    if (session instanceof NextResponse) return session
```

```
    await connectDB()
    const { searchParams } = new URL(request.url)
    const all = searchParams.get('all')

    if (all === 'true') {
      const result = await Stock.deleteMany({})
      return NextResponse.json({
        message: 'All stock entries deleted successfully',
        deletedCount: result.deletedCount || 0,
      })
    }

    // Without ?all=true this route is not used for single deletes —
    // those go through /api/stock/[id]. Return a clear error.
    return NextResponse.json(
      { error: 'Use DELETE /api/stock/[id] for single deletes, or ?all=true to delete every entry.' },
      { status: 400 }
    )
  } catch (error) {
    console.error('Error deleting stock entries:', error)
    return NextResponse.json({ error: 'Failed to delete stock entries' }, { status: 500 })
  }
}
```

/api/stock/summary/route.ts 186 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB } from '@/lib/db'
import { Stock, Production, DailySell } from '@/lib/models'
import { requireSession } from '@/lib/auth'

// Force dynamic — never cache summary responses
export const dynamic = 'force-dynamic'
export const revalidate = 0

// ─── Item-wise Stock Summary ──────────────────────────────────────
//
// Returns ONE row per product item (Cement, Zig Zag Grey 80mm, etc.) with:
//   • id                  — stable key derived from the field name (used by React)
//   • key                 — the schema field name (cement, zigZagGrey80, …)
//   • name                — human-readable label (used as the EXACT match key
//                           for DailySell.product, since the Daily Sell form
//                           now uses a dropdown of these exact names)
//   • totalProduction     — sum of this field across EVERY Production record
//                           (matches the column total shown in the Production
//                           module — this is the canonical "how much have we
//                           produced" number)
//   • sellItem            — sum of DailySell.quantity where DailySell.product
//                           EXACTLY matches this item's name (case-insensitive).
//                           Because the Daily Sell form now uses a dropdown
//                           populated with these same names, the match is exact
//                           — no fuzzy text guesswork.
//   • availableQuantity   — Total Production − Sell Item
//                           (the formula the user explicitly asked for)
//   • previousYearStock   — sum of Production[field] for entries whose date
//                           falls BEFORE the current calendar year.
//   • latestDate          — most recent date with non-zero production
//   • latestQuantity      — production value on that latest date
//   • productionDays      — count of UNIQUE dates with non-zero production
//
// The frontend renders a 5-column summary table:
//   Item Name | Available Qty | Sell Item | Total Production | Previous Year Stock
//
// All aggregation happens server-side so the client gets a small, fixed
// payload (one row per product, ~12 rows total) regardless of how many
// production / sales records exist.

interface ProductField {
  key: string
  name: string
}

const PRODUCT_FIELDS: ProductField[] = [
  { key: 'cement',         name: 'Cement' },
  { key: 'zigZagGrey80',   name: 'Zig Zag Grey 80mm' },
  { key: 'zigZagRed80',    name: 'Zig Zag Red 80mm' },
  { key: 'zigZagYellow80', name: 'Zig Zag Yellow 80mm' },
  { key: 'zigZagGrey60',   name: 'Zig Zag Grey 60mm' },
  { key: 'zigZagRed60',    name: 'Zig Zag Red 60mm' },
  { key: 'zigZagYellow60', name: 'Zig Zag Yellow 60mm' },
  { key: 'chequreTile',    name: 'Chequre Tile' },
  { key: 'curveStone',     name: 'Curve Stone' },
  { key: 'dumbleGrey80',   name: 'Dumble Grey 80mm' },
  { key: 'dumbleRed80',    name: 'Dumble Red 80mm' },
  { key: 'dumbleYellow80', name: 'Dumble Yellow 80mm' },
]

export async function GET() {
  try {
    const session = await requireSession()
    if (session instanceof NextResponse) return session

    await connectDB()

    // ── Fetch all source data in parallel ──────────────────────
    // Three queries total: Production (canonical source), Stock (per-date
    // snapshot for finding "latest" fast), and DailySell (sales records).
    const [productions, stocks, dailySells] = await Promise.all([
      Production.find({}).sort({ date: -1 }).lean(),
      Stock.find({}).sort({ date: -1 }).lean(),
      DailySell.find({}).lean(),
    ])

    // ── Determine current year boundary (YYYY-01-01) ──────────
    const currentYear = new Date().getFullYear()
    const currentYearStart = `${currentYear}-01-01`

    // ── SINGLE PASS over Production ────────────────────────────
    // Previously the code walked `productions` 4 times per field (total /
```

```
// prevYear / latest / uniqueDates), giving 48 passes over the array
// for 12 product fields. We now walk it ONCE and accumulate per-field
// tallies in parallel arrays.
const totalProductionByField   = new Array(PRODUCT_FIELDS.length).fill(0)
const previousYearByField      = new Array(PRODUCT_FIELDS.length).fill(0)
const latestDateByField        = new Array<string>(PRODUCT_FIELDS.length).fill('')
const latestQtyByField         = new Array<number>(PRODUCT_FIELDS.length).fill(0)
const uniqueDatesByField: Set<string>[] = PRODUCT_FIELDS.map(() => new Set<string>())

for (const p of productions) {
  const dateStr = String((p as Record<string, unknown>).date || '')
  const isPrevYear = dateStr && dateStr < currentYearStart
  for (let i = 0; i < PRODUCT_FIELDS.length; i++) {
    const key = PRODUCT_FIELDS[i].key
    const v = Number((p as Record<string, unknown>)[key]) || 0
    if (v <= 0) continue
    totalProductionByField[i] += v
    if (isPrevYear) previousYearByField[i] += v
    // productions is sorted date-desc, so the FIRST non-zero value we
    // encounter for a field is the latest date for that field.
    if (!latestDateByField[i]) {
      latestDateByField[i] = dateStr
      latestQtyByField[i] = v
    }
    if (dateStr) uniqueDatesByField[i].add(dateStr)
  }
}

// ── SINGLE PASS over Stock (for latest date fallback) ──────────
// Stocks are also sorted date-desc. Used only when Production had no
// entries for a field (rare edge case).
const stockLatestDate  = new Array<string>(PRODUCT_FIELDS.length).fill('')
const stockLatestQty   = new Array<number>(PRODUCT_FIELDS.length).fill(0)
for (const s of stocks) {
  const dateStr = String((s as Record<string, unknown>).date || '')
  for (let i = 0; i < PRODUCT_FIELDS.length; i++) {
    if (stockLatestDate[i]) continue // already found latest for this field
    const key = PRODUCT_FIELDS[i].key
    const v = Number((s as Record<string, unknown>)[key]) || 0
    if (v > 0) {
      stockLatestDate[i] = dateStr
      stockLatestQty[i] = v
    }
  }
}

// ── SINGLE PASS over DailySell (sell quantity by product name) ─────
// Multi-product records store line items in `products[]`; legacy single-
// product records only have `product` + `quantity`. We iterate over
// `products[]` when present so EVERY sold item is counted toward the
// correct product's stock — otherwise multi-product sales would only
// reduce the first product's available quantity.
const sellByProductName = new Map<string, number>()
const addSell = (rawName: string, qty: number) => {
  const name = rawName.toLowerCase().trim()
  if (!name) return
  sellByProductName.set(name, (sellByProductName.get(name) || 0) + qty)
}
for (const d of dailySells) {
  const prods = Array.isArray((d as any).products) ? (d as any).products : []
  if (prods.length > 0) {
    for (const p of prods) {
      addSell(String((p as any).product || ''), Number((p as any).quantity) || 0)
    }
  } else {
    addSell(
      String((d as Record<string, unknown>).product || ''),
      Number((d as Record<string, unknown>).quantity) || 0,
    )
  }
}

// ── Assemble per-field summary rows ────────────────────────────
const summaries = PRODUCT_FIELDS.map((field, i) => {
  const totalProduction   = totalProductionByField[i]
  const previousYearStock = previousYearByField[i]
  const sellItem          = sellByProductName.get(field.name.toLowerCase()) || 0
  const availableQuantity = totalProduction - sellItem
  const latestDate        = latestDateByField[i] || stockLatestDate[i]
  const latestQuantity    = latestDateByField[i] ? latestQtyByField[i] : stockLatestQty[i]
  const productionDays    = uniqueDatesByField[i].size

  return {
    id: field.key,
    key: field.key,
    name: field.name,
```

```
        totalProduction,
        sellItem,
        availableQuantity,
        previousYearStock,
        latestDate,
        latestQuantity,
        productionDays,
      }
    })

    return NextResponse.json({ summary: summaries })
  } catch (error) {
    console.error('Error fetching stock summary:', error)
    return NextResponse.json({ error: 'Failed to fetch stock summary' }, { status: 500 })
  }
}
```

/api/tractor-payment/[id]/route.ts 105 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB, toObject } from '@/lib/db'
import { TractorPayment } from '@/lib/models'
import { requireSession, requireRole } from '@/lib/auth'
import { normalizeDate } from '@/lib/date-utils'

// Force dynamic — never cache individual responses
export const dynamic = 'force-dynamic'
export const revalidate = 0

// Whitelist of updatable fields. Must match TractorPaymentSchema in src/lib/models.ts.
const FIELDS = [
  'date',
  'vendorName',
  'quantityTon',
  'rate',
  'totalAmount',
  'paidAmount',
  'remainingAmount',
  'remarks',
  'type',
  'linkedDailySellId',
] as const

// GET /api/tractor-payment/[id] — fetch a single tractor-payment entry
export async function GET(
  _request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const session = await requireSession()
    if (session instanceof NextResponse) return session

    await connectDB()
    const { id } = await params
    const record = await TractorPayment.findById(id).lean()
    if (!record) {
      return NextResponse.json({ error: 'tractorPayment entry not found' }, { status: 404 })
    }
    return NextResponse.json({ tractorPayment: toObject(record) })
  } catch (error) {
    console.error('Error fetching tractor-payment entry:', error)
    return NextResponse.json({ error: 'Failed to fetch tractor-payment entry' }, { status: 500 })
  }
}

// PUT /api/tractor-payment/[id] — update a single tractor-payment entry
export async function PUT(
  request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const session = await requireRole(['admin', 'operator', 'accountant'])
    if (session instanceof NextResponse) return session

    await connectDB()
    const { id } = await params
    const body = await request.json()
    // Normalize date to canonical YYYY-MM-DD
    // (handles dd-mm-yyyy, dd/mm/yyyy, Excel serials, Date objects, etc.)
    if (body.date) body.date = normalizeDate(body.date)

    const updateData: Record<string, unknown> = {}
    for (const field of FIELDS) {
      if (body[field] !== undefined) {
        updateData[field] = body[field]
      }
    }

    const record = await TractorPayment.findByIdAndUpdate(id, updateData, { new: true })
    if (!record) {
      return NextResponse.json({ error: 'tractorPayment entry not found' }, { status: 404 })
    }

    return NextResponse.json({ tractorPayment: toObject(record) })
  } catch (error) {
    console.error('Error updating tractor-payment entry:', error)
    return NextResponse.json({ error: 'Failed to update tractor-payment entry' }, { status: 500 })
  }
}

// DELETE /api/tractor-payment/[id] — delete a single tractor-payment entry
export async function DELETE(
```

```
  _request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const session = await requireRole(['admin', 'operator', 'accountant'])
    if (session instanceof NextResponse) return session

    await connectDB()
    const { id } = await params

    const record = await TractorPayment.findByIdAndDelete(id)
    if (!record) {
      return NextResponse.json({ error: 'tractorPayment entry not found' }, { status: 404 })
    }

    return NextResponse.json({ message: 'tractorPayment entry deleted successfully' })
  } catch (error) {
    console.error('Error deleting tractor-payment entry:', error)
    return NextResponse.json({ error: 'Failed to delete tractor-payment entry' }, { status: 500 })
  }
}
```

/api/tractor-payment/bulk-delete/route.ts 63 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB } from '@/lib/db'
import { TractorPayment } from '@/lib/models'
import { getSession } from '@/lib/auth'

// Force dynamic — never cache
export const dynamic = 'force-dynamic'

// POST /api/tractor-payment/bulk-delete
// Body: { ids: string[] }
// Deletes tractor payment entries whose `_id` is in `ids`.
// Gated behind admin/operator session — accountants are read-only.
export async function POST(request: Request) {
  try {
    await connectDB()

    const session = await getSession()
    if (!session) {
      return NextResponse.json(
        { error: 'Unauthorized — please log in' },
        { status: 401 }
      )
    }
    if (session.role === 'accountant') {
      return NextResponse.json(
        { error: 'Forbidden — accountants cannot delete tractor payment entries' },
        { status: 403 }
      )
    }

    const body = await request.json()
    const { ids } = body as { ids?: string[] }

    if (!Array.isArray(ids) || ids.length === 0) {
      return NextResponse.json(
        { error: 'ids array is required and must be non-empty' },
        { status: 400 }
      )
    }

    const result = await TractorPayment.deleteMany({ _id: { $in: ids } })

    if (result.deletedCount === 0) {
      return NextResponse.json(
        { error: 'No matching tractor payment entries found — they may have been already deleted' },
        { status: 404 }
      )
    }

    return NextResponse.json({
      message: `${result.deletedCount} tractor payment entr${result.deletedCount === 1 ? 'y' : 'ies'} deleted`,
      deletedCount: result.deletedCount,
      requestedCount: ids.length,
    })
  } catch (error) {
    console.error('Error bulk-deleting tractor payments:', error)
    return NextResponse.json(
      { error: 'Failed to bulk-delete tractor payment entries' },
      { status: 500 }
    )
  }
}
```

/api/tractor-payment/route.ts 62 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB, toObject } from '@/lib/db'
import { TractorPayment } from '@/lib/models'
import { requireSession, requireRole } from '@/lib/auth'
import { normalizeDate } from '@/lib/date-utils'

// Force dynamic — never cache list responses
export const dynamic = 'force-dynamic'
export const revalidate = 0

export async function GET() {
  try {
    const session = await requireSession()
    if (session instanceof NextResponse) return session

    await connectDB()
    const records = await TractorPayment.find({}).sort({ date: -1 }).lean()
    return NextResponse.json({ tractorPayments: records.map(toObject) })
  } catch (error) {
    console.error('Error fetching tractor payments:', error)
    return NextResponse.json({ error: 'Failed to fetch tractor payments' }, { status: 500 })
  }
}

export async function POST(request: Request) {
  try {
    const session = await requireRole(['admin', 'operator', 'accountant'])
    if (session instanceof NextResponse) return session

    await connectDB()
    const body = await request.json()
    // Normalize date to canonical YYYY-MM-DD
    // (handles dd-mm-yyyy, dd/mm/yyyy, Excel serials, Date objects, etc.)
    if (body.date) body.date = normalizeDate(body.date)
    if (!body.date || !body.vendorName || !body.quantityTon || !body.rate) {
      return NextResponse.json({ error: 'Date, vendor name, quantity and rate are required' }, { status: 400 })
    }
    const totalAmount = Number(body.quantityTon) * Number(body.rate)
    const paidAmount = Number(body.paidAmount) || 0
    // 'type' field: 'tractor' (default, classic vendor payment) or
    // 'transporter' (auto-synced from Daily Sell). When the user creates
    // a record through the normal UI it's always 'tractor'.
    const type = body.type === 'transporter' ? 'transporter' : 'tractor'
    const record = await TractorPayment.create({
      date: body.date,
      vendorName: body.vendorName,
      quantityTon: Number(body.quantityTon),
      rate: Number(body.rate),
      totalAmount,
      paidAmount,
      remainingAmount: totalAmount - paidAmount,
      remarks: body.remarks || '',
      type,
      linkedDailySellId: body.linkedDailySellId || null,
    })
    return NextResponse.json({ tractorPayment: toObject(record) }, { status: 201 })
  } catch (error) {
    console.error('Error creating tractor payment:', error)
    return NextResponse.json({ error: 'Failed to create tractor payment' }, { status: 500 })
  }
}
```

/api/users/[id]/route.ts 123 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB, toObject } from '@/lib/db'
import { User } from '@/lib/models'
import { requireAdmin } from '@/lib/auth'

const VALID_ROLES = new Set(['admin', 'operator', 'accountant'])

export async function GET(request: Request, { params }: { params: Promise<{ id: string }> }) {
  try {
    const auth = await requireAdmin()
    if (auth instanceof NextResponse) return auth

    await connectDB()
    const { id } = await params
    const user = await User.findById(id)
    if (!user) {
      return NextResponse.json({ error: 'User not found' }, { status: 404 })
    }
    const obj = toObject(user)
    delete obj.password
    return NextResponse.json({ user: obj })
  } catch (error) {
    console.error('Error fetching user:', error)
    return NextResponse.json({ error: 'Failed to fetch user' }, { status: 500 })
  }
}

export async function PUT(request: Request, { params }: { params: Promise<{ id: string }> }) {
  try {
    const auth = await requireAdmin()
    if (auth instanceof NextResponse) return auth

    await connectDB()
    const { id } = await params
    const body = await request.json()

    const updateData: Record<string, unknown> = {}

    if (body.name !== undefined) {
      if (typeof body.name !== 'string' || body.name.trim().length === 0) {
        return NextResponse.json({ error: 'Name cannot be empty' }, { status: 400 })
      }
      updateData.name = body.name.trim()
    }
    if (body.email !== undefined) {
      if (typeof body.email !== 'string' || !/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(body.email)) {
        return NextResponse.json({ error: 'Invalid email format' }, { status: 400 })
      }
      updateData.email = body.email.trim()
    }
    if (body.role !== undefined) {
      if (!VALID_ROLES.has(body.role)) {
        return NextResponse.json(
          { error: `Invalid role. Must be one of: ${Array.from(VALID_ROLES).join(', ')}` },
          { status: 400 }
        )
      }
      updateData.role = body.role
    }
    if (body.active !== undefined) {
      updateData.active = Boolean(body.active)
    }

    // Handle password change separately
    if (body.password) {
      if (typeof body.password !== 'string' || body.password.length < 6) {
        return NextResponse.json({ error: 'Password must be at least 6 characters' }, { status: 400 })
      }
      const bcrypt = await import('bcryptjs')
      updateData.password = await bcrypt.default.hash(body.password, 12)
    }

    const user = await User.findByIdAndUpdate(id, updateData, { new: true })
    if (!user) {
      return NextResponse.json({ error: 'User not found' }, { status: 404 })
    }

    const obj = toObject(user)
    delete obj.password
    return NextResponse.json({ user: obj })
  } catch (error) {
    console.error('Error updating user:', error)
    return NextResponse.json({ error: 'Failed to update user' }, { status: 500 })
```

```
  }
}

export async function DELETE(request: Request, { params }: { params: Promise<{ id: string }> }) {
  try {
    const auth = await requireAdmin()
    if (auth instanceof NextResponse) return auth

    await connectDB()
    const { id } = await params

    // Block self-delete — prevent admin from locking themselves out
    if (auth && typeof auth === 'object' && 'userId' in auth && auth.userId === id) {
      return NextResponse.json(
        { error: 'Cannot delete your own account' },
        { status: 400 }
      )
    }

    // Block deleting the LAST admin
    const adminCount = await User.countDocuments({ role: 'admin', active: true })
    const target = await User.findById(id).lean()
    if (target && target.role === 'admin' && target.active && adminCount <= 1) {
      return NextResponse.json(
        { error: 'Cannot delete the last active admin account. Promote another user to admin first.' },
        { status: 400 }
      )
    }

    const user = await User.findByIdAndDelete(id)
    if (!user) {
      return NextResponse.json({ error: 'User not found' }, { status: 404 })
    }
    return NextResponse.json({ message: 'User deleted successfully' })
  } catch (error) {
    console.error('Error deleting user:', error)
    return NextResponse.json({ error: 'Failed to delete user' }, { status: 500 })
  }
}
```

/api/users/bulk-delete/route.ts 84 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB } from '@/lib/db'
import { User } from '@/lib/models'
import { getSession } from '@/lib/auth'

export const dynamic = 'force-dynamic'

// POST /api/users/bulk-delete
// Body: { ids: string[] }
// Admin-only — operators and accountants cannot delete users.
// Self-delete is blocked (you can't delete your own account).
export async function POST(request: Request) {
  try {
    await connectDB()

    const session = await getSession()
    if (!session) {
      return NextResponse.json(
        { error: 'Unauthorized — please log in' },
        { status: 401 }
      )
    }
    if (session.role !== 'admin') {
      return NextResponse.json(
        { error: 'Forbidden — only admins can delete users' },
        { status: 403 }
      )
    }

    const body = await request.json()
    const { ids } = body as { ids?: string[] }

    if (!Array.isArray(ids) || ids.length === 0) {
      return NextResponse.json(
        { error: 'ids array is required and must be non-empty' },
        { status: 400 }
      )
    }

    // Block self-delete — prevent admin from locking themselves out
    if (session.userId && ids.includes(session.userId)) {
      return NextResponse.json(
        { error: 'Cannot delete your own account. Remove yourself from the selection first.' },
        { status: 400 }
      )
    }

    // Also block deleting the LAST admin — would lock everyone out of admin panel
    const adminCount = await User.countDocuments({ role: 'admin', active: true })
    const adminsBeingDeleted = await User.countDocuments({
      _id: { $in: ids },
      role: 'admin',
      active: true,
    })
    if (adminCount > 0 && adminsBeingDeleted >= adminCount) {
      return NextResponse.json(
        { error: 'Cannot delete the last active admin account. Promote another user to admin first.' },
        { status: 400 }
      )
    }

    const result = await User.deleteMany({ _id: { $in: ids } })

    if (result.deletedCount === 0) {
      return NextResponse.json(
        { error: 'No matching users found — they may have been already deleted' },
        { status: 404 }
      )
    }

    return NextResponse.json({
      message: `${result.deletedCount} user${result.deletedCount === 1 ? '' : 's'} deleted`,
      deletedCount: result.deletedCount,
      requestedCount: ids.length,
    })
  } catch (error) {
    console.error('Error bulk-deleting users:', error)
    return NextResponse.json(
      { error: 'Failed to bulk-delete users' },
      { status: 500 }
    )
  }
```

```
  }
}
```

/api/users/bulk-update/route.ts 96 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB } from '@/lib/db'
import { User } from '@/lib/models'
import { getSession } from '@/lib/auth'

export const dynamic = 'force-dynamic'

// POST /api/users/bulk-update
// Body: { ids: string[], active: boolean }
// Admin-only — bulk activate / deactivate users in one call.
// Self-deactivation is blocked (you can't lock yourself out by accident).
export async function POST(request: Request) {
  try {
    await connectDB()

    const session = await getSession()
    if (!session) {
      return NextResponse.json(
        { error: 'Unauthorized — please log in' },
        { status: 401 }
      )
    }
    if (session.role !== 'admin') {
      return NextResponse.json(
        { error: 'Forbidden — only admins can bulk-update users' },
        { status: 403 }
      )
    }

    const body = await request.json()
    const { ids, active } = body as { ids?: string[]; active?: boolean }

    if (!Array.isArray(ids) || ids.length === 0) {
      return NextResponse.json(
        { error: 'ids array is required and must be non-empty' },
        { status: 400 }
      )
    }
    if (typeof active !== 'boolean') {
      return NextResponse.json(
        { error: 'active (boolean) is required' },
        { status: 400 }
      )
    }

    // If deactivating, block self-deactivation AND block deactivating the last admin
    if (active === false) {
      if (session.userId && ids.includes(session.userId)) {
        return NextResponse.json(
          { error: 'Cannot deactivate your own account. Remove yourself from the selection first.' },
          { status: 400 }
        )
      }
      const adminCount = await User.countDocuments({ role: 'admin', active: true })
      const adminsBeingDeactivated = await User.countDocuments({
        _id: { $in: ids },
        role: 'admin',
        active: true,
      })
      if (adminCount > 0 && adminsBeingDeactivated >= adminCount) {
        return NextResponse.json(
          { error: 'Cannot deactivate the last active admin. Promote another user to admin first.' },
          { status: 400 }
        )
      }
    }

    const result = await User.updateMany(
      { _id: { $in: ids } },
      { $set: { active } }
    )

    if (result.modifiedCount === 0 && result.matchedCount === 0) {
      return NextResponse.json(
        { error: 'No matching users found' },
        { status: 404 }
      )
    }

    const action = active ? 'activated' : 'deactivated'
    return NextResponse.json({
      message: `${result.modifiedCount} user${result.modifiedCount === 1 ? '' : 's'} ${action}`,
      modifiedCount: result.modifiedCount,
```

```
      matchedCount: result.matchedCount,
      requestedCount: ids.length,
      active,
    })
  } catch (error) {
    console.error('Error bulk-updating users:', error)
    return NextResponse.json(
      { error: 'Failed to bulk-update users' },
      { status: 500 }
    )
  }
}
```

/api/users/route.ts 78 lines

```ts
import { NextResponse } from 'next/server'
import { connectDB, toObject } from '@/lib/db'
import { User } from '@/lib/models'
import { requireAdmin } from '@/lib/auth'

const VALID_ROLES = new Set(['admin', 'operator', 'accountant'])

export async function GET() {
  try {
    const auth = await requireAdmin()
    if (auth instanceof NextResponse) return auth

    await connectDB()
    const users = await User.find({}).sort({ createdAt: -1 }).lean()

    // Don't return passwords
    const result = users.map((u: any) => {
      const obj = toObject(u)
      delete obj.password
      return obj
    })

    return NextResponse.json({ users: result })
  } catch (error) {
    console.error('Error fetching users:', error)
    return NextResponse.json({ error: 'Failed to fetch users' }, { status: 500 })
  }
}

export async function POST(request: Request) {
  try {
    const auth = await requireAdmin()
    if (auth instanceof NextResponse) return auth

    await connectDB()
    const bcrypt = await import('bcryptjs')
    const body = await request.json()

    if (!body.name || !body.email || !body.password || !body.role) {
      return NextResponse.json({ error: 'Missing required fields' }, { status: 400 })
    }
    if (!VALID_ROLES.has(body.role)) {
      return NextResponse.json(
        { error: `Invalid role. Must be one of: ${Array.from(VALID_ROLES).join(', ')}` },
        { status: 400 }
      )
    }
    if (typeof body.password !== 'string' || body.password.length < 6) {
      return NextResponse.json({ error: 'Password must be at least 6 characters' }, { status: 400 })
    }
    if (typeof body.email !== 'string' || !/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(body.email)) {
      return NextResponse.json({ error: 'Invalid email format' }, { status: 400 })
    }

    // Check if email already exists
    const existing = await User.findOne({ email: body.email })
    if (existing) {
      return NextResponse.json({ error: 'Email already exists' }, { status: 400 })
    }

    const hashedPassword = await bcrypt.default.hash(body.password, 12)
    const user = await User.create({
      name: body.name,
      email: body.email,
      password: hashedPassword,
      role: body.role,
      active: body.active !== undefined ? body.active : true,
    })

    const obj = toObject(user)
    delete obj.password
    return NextResponse.json({ user: obj }, { status: 201 })
  } catch (error) {
    console.error('Error creating user:', error)
    return NextResponse.json({ error: 'Failed to create user' }, { status: 500 })
  }
}
```